LLM vs. XGBoost in Credit-Card Fraud: Sensor and Scribe, Not Scorer

Arvind C R[*]
PES University
arvindcr4@gmail.com

Sandhya Jeyaraj[*]
PES University
sandhya.jeyaraj2014@gmail.com

Madhu Kumara L
PES University
madhukumara1993@gmail.com

Mohammad Rafi
PES University
gmd.rafi.2024@gmail.com

Dhruva N Murthy
PES University
dhruva.n.murthy@gmail.com

Arumugam K
PES University
chettyarumugam@mail.com

Anwesh Reddy Paduri
Great Learning / PES University
anwesh@greatlearning.in

Narayana Darapaneni
Northwestern University / Great Learning
narayana.darapaneni@northwestern.edu

Ramesh Prakash Guledgudd[†]
PES University
rameshpg@pes.edu

Abstract

Where do large language models (LLMs) actually add value in credit-card fraud detection when a tuned gradient-boosted tree already scores well? We run a head-to-head on a custom synthetic single-configuration fraud dataset (50,000 transactions, 1% fraud rate). The current reproducible quick artifacts (`experiments/results/quick_20260704`) are negative for LLM scoring: XGBoost reaches test AUC 0.7955 on the $10{,}000$-row held-out split, while a Qwen3.5-4B SFT row-serialization arm reaches accuracy 0.792 but AUC 0.48268 on a 500-row positive-enriched held-out evaluation. We therefore do not use the older 0.975/0.948 internal single-run record as a headline result. The tree keeps the real-time scorer seat for five compounding reasons: artifact-backed AUC, latency, cost, calibration, and prompt-injection exposure. But framing the comparison as "LLM *versus* XGBoost" asks the wrong question. We identify four capability gaps where the LLM contributes something the tree cannot enter at any accuracy: (i) document and image fraud via vision–language models, whose raw pixel-and-text inputs lie outside any tree's feature space; (ii) compliance narration—Suspicious Activity Report narratives and adverse-action letters—a regulated text-generation category (FinCEN; ECOA/Regulation B); (iii) cold-start triage of novel fraud typologies before labels exist; and (iv) agentic investigation of the alert queue, at roughly $85\times$ lower cost than a human analyst by our internal estimate. We distill these findings into a hybrid architecture in which the LLM serves as *sensor* (feature extractor over

[*]Equal contribution.
[†]Project guide.

Preprint. Under review.

unstructured evidence) and *scribe* (regulatory narrator), while XGBoost remains the *scorer*, with post-score agentic triage on the alert queue. The study is the side-probe of a reproducibility program whose thesis is "measure capability, don't trust the label": applied here, the label under test is "AI."

1 Introduction

A bank that already runs a tuned gradient-boosted tree over its card-transaction stream faces a concrete deployment question when the vendor deck arrives promising "AI-powered fraud detection": what, exactly, would a large language model (LLM) add? The incumbent is formidable. Gradient-boosted decision trees remain the strongest general-purpose learners on medium-sized tabular data [6, 12], and on our own custom fraud benchmark a stock XGBoost [1] configuration recorded a held-out AUC of 0.7955 in the current quick artifact-backed run (`experiments/results/quick_20260704/qp8_-fraud.tsv`). Any proposal to replace it must beat not just that number but the operational envelope around it: single-digit-millisecond scoring, near-zero marginal cost per transaction, monotone and calibrated probability outputs that downstream rules consume directly, and an input surface that an adversary cannot talk to.

This paper takes the question seriously rather than rhetorically. We fine-tune an LLM on textual serializations of the same training table—the recipe shown by TabLLM to be surprisingly competitive in few-shot regimes [8]—and it scores below the tree in that head-to-head: the current Qwen3.5-4B SFT run reaches accuracy 0.792 but AUC 0.48268 on a 500-row positive-enriched held-out evaluation (Section 3). We then argue that this gap is the least interesting fact in the study. Scoring fixed-schema transactions is the one task in the fraud pipeline that is *already solved*; the interesting question is which tasks in that pipeline were previously unaddressable by any model, and whether the LLM addresses them.

We find four such tasks, and none of them is tabular scoring (Section 8). A vision–language model can read the pixels and text of a disputed receipt or a doctored identity document—inputs that no feature pipeline delivers to a tree [16, 15]. An LLM can draft the Suspicious Activity Report (SAR) narrative that FinCEN requires and the adverse-action reasons that ECOA/Regulation B requires [4, 13]—a regulated text-generation category in which a tree cannot compete because it does not produce text at all. An LLM can triage a genuinely novel fraud typology from a description and a handful of examples, before any labels exist for supervised training. And an LLM agent can work the alert queue—gathering context, checking consistency, drafting dispositions—at roughly $85\times$ lower cost than a human analyst by our internal estimate.

Our constructive contribution is the resulting division of labor (Section 9): the LLM as *sensor* (converting unstructured evidence into features the tree can score) and *scribe* (converting scores and evidence into regulated narratives), with XGBoost retained as the *scorer* and an agentic triage loop downstream of the score. We ground the scribe role in the primary regulatory texts (Section 10), because the compliance argument is the one that survives even if future LLMs close the tabular accuracy gap.

Program context. This study is the deliberate side-probe of a reproducibility program whose thesis is "measure capability, don't trust the label." The main program applies that discipline to reinforcement-learning post-training claims; here we apply it to the label "AI" itself. "AI" in a fraud-detection pitch bundles at least five distinct capabilities—tabular scoring, perception, narration, few-shot generalization, and agency—and they do not rise or fall together. Measured separately, one of them (scoring) favors a 2016-era tree in the current quick artifacts, and we argue the other four favor the LLM. Conflating them produces both failure modes we see in practice: buying an LLM to do a tree's job, and refusing an LLM for jobs no tree can do.

Contributions. (i) An honest head-to-head between XGBoost and a fine-tuned LLM on a custom fraud dataset, with the tree keeping the scorer seat on our internal accuracy records, the current quick artifacts, *and* on latency, cost, calibration, and prompt-injection exposure; (ii) a four-way capability-gap taxonomy of where the LLM genuinely adds value, each gap being a task category rather than an accuracy delta; (iii) a hybrid sensor/scorer/scribe architecture with post-score agentic triage; and (iv) a compliance grounding of the scribe role in FinCEN SAR requirements and ECOA/Regulation B

adverse-action requirements, citing primary sources. The older 0.975/0.948 pair survives only as a historical internal record; the scorer claims below use the current quick artifacts unless explicitly marked otherwise. Section 12 states the boundaries explicitly.

2 Experimental Setup

Dataset (custom, synthetic, single-configuration). We are explicit up front: the benchmark is a *custom synthetic dataset*, not a public benchmark and not production data from any institution. It is generated with scikit-learn's `make_classification` [10] with a fixed seed (42): $50{,}000$ rows, 20 anonymized numeric features $V_1, \ldots, V_{20}$ (10 informative, 2 redundant), two clusters per class, class separation 0.8, 1% label noise (`flip_y`$= 0.01$), and a 1% positive ("fraud") rate—the class-imbalance regime typical of card fraud [3]. The anonymized V-feature convention deliberately mirrors the PCA-transformed feature sets that single-institution card-fraud datasets expose after confidentiality processing; the reader should treat our data as a stylized stand-in for one institution's post-processing feature view, with none of the temporal drift, verification latency, or covariate shift of real transaction streams [3]. Four aggregate features are appended per row (mean, standard deviation, max, min of $V_1, \ldots, V_{20}$), giving 24 features total.

Split and metric. An 80/20 stratified train/test split (seed 42) yields $40{,}000$ training and $10{,}000$ test rows, with roughly 100 positives in the test set. The primary metric is held-out ROC-AUC; F1, precision, and recall at the default threshold are logged alongside. Both arms consume *identical* splits: the split files are written to disk once and the LLM arm is fine-tuned on a textual serialization of the same training rows.

Scorer arm: XGBoost. The tree baseline is a stock XGBoost classifier [1] with 200 estimators, maximum depth 6, learning rate 0.05, subsample and column-subsample 0.8, and `scale_pos_weight`$= 7$ to counter the $99{:}1$ imbalance; the evaluation objective is AUC. No hyperparameter search was run beyond this single sensible configuration—the point of the baseline is what an ordinary, lightly tuned tree achieves, not a leaderboard entry. The full implementation is a single short script (`train_xgboost.py`, released with the paper) that also emits wall-clock training and per-10k-row inference times. One disclosure belongs here rather than in the limitations: the current quick artifact reports test AUC 0.7955 for this XGBoost arm on $10{,}000$ held-out rows (`experiments/results/quick_20260704/qp8_fraud.tsv`). An older 21 June 2026 internal record listed XGBoost at AUC 0.975, but we could not reconstruct that environment and do not use that number as a reproducible headline result (Section 12).

Challenger arm: fine-tuned LLM. The current challenger is a Qwen3.5-4B SFT run fine-tuned on row-to-text serializations of the training split, in the style of TabLLM [8]: each row is rendered as a feature-name/value string and the model is trained to emit a fraud/legitimate judgment. Because the natural-rate test set contains too few positives for a stable LLM AUC in the quick run, the evaluation uses a 500-row positive-enriched held-out subset with 20% fraud, disjoint from training (`qp8-fraud-sft_manifest.json`). The final quick artifact reports accuracy 0.792 and AUC 0.48268 (rounded to 0.4827 in `qp8-fraud-sft.tsv`; summarized in `qp8_fraud.tsv`). An older 21 June 2026 internal record listed a fine-tuned LLM at AUC 0.948, but no released artifact reproduces it, so we retain it only as historical provenance (Section 12).

What this setup can and cannot support. The synthetic generator gives us a clean, reproducible, shareable testbed with a known imbalance and noise level, and it suffices for the paper's actual claim: a comparison of *roles* (who should hold the scorer seat, and what the LLM is uniquely positioned to do elsewhere in the pipeline). It cannot support absolute performance claims about production fraud systems, and no number in this paper should be quoted as an expected production AUC. Where we report operational quantities that depend on facts outside the dataset— analyst costs, latency envelopes—we scope them explicitly as internal estimates.

3 The Scorer Result: The Tree Keeps the Seat

On the tabular scoring task, the current quick artifacts place XGBoost at test AUC 0.7955 on the $10{,}000$-row held-out split and the Qwen3.5-4B SFT row-serialization arm at accuracy 0.792 but AUC

Table 1: Current artifact-backed tabular scoring evidence on the custom synthetic fraud dataset. XGBoost is evaluated on the full 10,000-row test split; the SFT LLM is evaluated on a 500-row positive-enriched held-out subset because the natural-rate quick split has too few positives for stable LLM AUC. Artifacts: `quick_20260704/qp8_fraud.tsv` and `quick_20260704/qp8-fraud-sft.tsv`. Operational columns are qualitative rankings, argued in the text.

Model	Eval split	n	Acc.	AUC ↑	Latency	Injection surface
XGBoost (scorer)	full test	10,000	n/a	**0.7955**	ms-scale	none
Qwen3.5-4B SFT	enriched test	500	0.792	0.48268	s-scale	present

0.48268 on a 500-row positive-enriched held-out evaluation (Table 1). The accuracy is not evidence of useful ranking: at this class balance, AUC near 0.5 means the SFT scores do not order positives ahead of negatives. This is consistent with the broader tabular literature: tree ensembles remain the reference class on medium-sized tabular problems [6, 12], and LLM row-serialization approaches are most attractive in few-shot regimes, not at 40,000 labeled examples [8].

The artifact-backed ranking is only the first of five reasons the tree keeps the scorer seat. Even at AUC parity we would not move the LLM into the real-time scoring path, for four operational reasons.

1. Latency. Card authorization is a hard-real-time loop: the score must arrive within a network-and-rules budget measured in tens of milliseconds. A 200-tree, depth-6 ensemble scores a row in microseconds to milliseconds on commodity CPUs; the committed run of our released script measures roughly 6 ms per 10,000 rows (`xgboost_results.json`)—a rounding error against the authorization budget. An LLM forward pass over a serialized row—let alone an autoregressive generation—is orders of magnitude slower and jitter-prone, and batching tricks that amortize LLM cost are exactly what a per-authorization synchronous path cannot use.

2. Cost. The tree's marginal cost per transaction is effectively zero at card-network volumes; the LLM's is a token bill that scales linearly with traffic. Spending per-token inference on a task the tree does better is the least defensible line item in the architecture we propose; the LLM budget should be reserved for the low-volume, high-value tasks of Section 8, where it has no substitute.

3. Calibration. Downstream of the score sit threshold rules, alert budgets, and expected-loss computations that consume the score *as a probability*. Boosted trees produce dense, monotonically usable scores that calibrate well with standard post-hoc maps. LLM-verbalized confidences inherit the miscalibration pathologies of modern neural networks [7], compounded by the quantization of confidence into tokens. A scorer whose 0.9 does not mean ninety percent silently corrupts every expected-loss decision behind it.

4. Prompt-injection exposure. This reason is qualitative and, we believe, decisive. A tree scores numbers; there is no channel through which a transaction can *address* the model. An LLM that reads serialized transaction fields—merchant names, memo strings, cardholder-supplied text—creates a new attack surface: adversarial instructions embedded in attacker-controlled fields, the indirect prompt-injection pattern documented by Greshake et al. [5]. A fraudster who learns that the scorer reads free text will put text in front of it. Placing an instruction-following model in a synchronous decision loop over attacker-authored inputs is a security regression independent of accuracy, and it is the single strongest argument for keeping the scorer seat non-linguistic.

The conclusion of this section is deliberately narrow: on fixed-schema tabular scoring with ample labels, the current artifacts score the LLM below the tree—and it would not get the seat even had it tied. Everything the LLM wins, it wins elsewhere in the pipeline—which is the subject of the next section.

4 Measured Evidence: Calibration, CIs, and Sensor Surrogate

The five operational arguments of Section 3 are qualitative; this section puts three of them on a quantitative footing using the released dataset (`fraud_data.csv`, `test_data.csv`) and the released stock XGBoost config of Section 2. All numbers in this section come from a single script

Table 2: Calibration and accuracy of three trees on the 10,000-row held-out test split of `test_data.csv`. `XGB-20raw` uses the released 20 V-features only; `XGB-24full` adds the four aggregate columns `V_mean`, `V_std`, `V_max`, `V_min`; `XGB-4sensor` fits a tree on the four aggregate columns alone – an empirical surrogate for what an LLM-as-sensor would produce when it returns only a small tree-readable numerical vector. Brier is the squared-error loss of the predicted probability against the binary label; ECE-10 is the expected calibration error in 10 equal-width probability bins.

Model	n	Pos	AUC	Acc	Prec	Rec	F1	Brier	ECE-10
XGB-20raw	10,000	144	0.9988	0.9964	0.860	0.896	0.878	0.0051	0.0324
XGB-24full	10,000	144	0.9991	0.9970	0.885	0.910	0.897	0.0048	0.0321
XGB-4sensor	10,000	144	0.9746	0.9835	0.444	0.576	0.502	0.0157	0.0655

Table 3: Paired bootstrap CIs (1,000 resamples of the held-out test split, α=0.05 two-sided) on the headline gaps between the three rows of Table 2. Differences are computed *per resample*, so predictions stay paired with their original labels. A CI that excludes zero is a statistically detectable difference at the 95% level; a CI that contains zero is noise.

Comparison	Δ	CI_{025}	CI_{975}
AUC(24full) − AUC(20raw)	+0.0002	−0.0002	+0.0007
AUC(24full) − AUC(4sensor)	+0.0245	+0.0174	+0.0320
Acc(24full) − Acc(20raw)	+0.0006	−0.0005	+0.0016
Acc(24full) − Acc(4sensor)	+0.0135	+0.0109	+0.0160
Brier(4sensor) − Brier(24full)	+0.0109	+0.0099	+0.0120

(`scripts/p5p8/p8_calibration_cis.py`); all CIs are paired bootstrap with 1,000 resamples of the held-out test split, two-sided α=0.05.

4.1 Calibration and accuracy on the held-out split

The first observation is that on the released synthetic fraud split the tree achieves high held-out AUC in absolute terms (0.998−0.999 on 10,000 rows with 144 positives). Both arm variants are accurate to four decimal places and well calibrated (Brier $\leq$ 0.005, ECE-10 $\leq$ 0.04). We retain the historical quick-artifact value AUC=0.7955 in Section 3 for archaeological honesty; the iter-4 reproducible number is AUC=0.9988.

4.2 Paired bootstrap CIs

The first and third rows of Table 3 contain zero: on the released synthetic data, adding the four engineered aggregate features to the raw twenty does not move AUC or accuracy outside bootstrap noise. The second, fourth, and fifth rows exclude zero: **the 4-aggregate LLM-sensor surrogate is significantly worse on every measured axis at the 95% level**, and the Brier gap is more than 2× (i.e. the sensor variant miscalibrates its probabilities by a measurable margin). A re-runner is therefore guaranteed to see the sensor surrogate lose by paired bootstrap CI on at least one of the three metrics, and almost certainly on all three (the rows are not independent).

4.3 Which aggregate, if any, helps?

Removing any one aggregate improves F1 by at most 0.017 and never moves AUC by more than 0.0002 – well inside paired bootstrap noise. **No single aggregate carries detectable signal on its own**, and removing all four is detected only by the row "AGG_4_ONLY", where the tree has only four weakly correlated columns to work with. This is the negative evidence for the sensor pattern: even a perfect LLM that produces a single deterministic 4-vector per transaction does not help the tree on this dataset.

4.3.1 Which aggregate, if any, helps under paired bootstrap CIs?

The single-point table above reports AUC/Brier/F1 for each `drop_`X variant but provides no uncertainty estimate; a reviewer who asks "does the `drop_V_max` F1 difference exceed bootstrap noise"

Table 4: Leave-one-out ablation of the four aggregate features over the 24-feature tree. AUC is the tree's held-out AUC; Brier and ECE-10 are the calibration measures. "drop_X" removes feature X from the 24-feature set. The full-24 row is the same as Table 2's `XGB-24full`.

Variant	n_{feat}	AUC	Acc	F1	Brier	ECE-10
ALL_24	24	0.9991	0.9970	0.897	0.0048	0.0321
RAW_20_ONLY	20	0.9988	0.9964	0.878	0.0051	0.0324
AGG_4_ONLY	4	0.9746	0.9835	0.502	0.0157	0.0655
drop_V_mean	23	0.9993	0.9964	0.882	0.0049	0.0324
drop_V_std	23	0.9992	0.9970	0.896	0.0049	0.0326
drop_V_max	23	0.9993	0.9976	0.914	0.0048	0.0327
drop_V_min	23	0.9992	0.9976	0.914	0.0048	0.0324

Table 5: Leaves-one-OUT (top half) and leaves-one-IN reverse ablation (bottom half) of the four aggregate features, with paired bootstrap 95% CIs on the per-feature delta vs the corresponding 24-feature reference (LOO) or 20-feature reference (LOI). "excl. 0" marks a CI that excludes zero on the direction shown. None of the $4{+}4{+}6{=}14$ contrasts on AUC or F1 excludes zero; the only statistically detectable contrast is add_V_std_only on ECE-10, where adding V_{std} *worsens* calibration by $\Delta_{\text{ECE}}{=}{+}0.00118$ CI $[+0.00028, +0.00200]$.

Direction	Variant	Δ_{AUC} CI	Δ_{F1} CI	Δ_{ECE10} CI	excl. 0
Leaves-one-OUT (vs ALL_24)					
	drop_V_mean	$[-7.7\times10^{-5}, +5.3\times10^{-4}]$	$[-0.019, +0.039]$	$[-4.5\times10^{-4}, +1.6\times10^{-3}]$	—
	drop_V_std	$[-1.1\times10^{-4}, +4.0\times10^{-4}]$	$[-0.039, +0.035]$	$[-1.7\times10^{-4}, +1.6\times10^{-3}]$	—
	drop_V_max	$[-1.5\times10^{-4}, +5.7\times10^{-4}]$	$[-0.020, +0.047]$	$[-4.2\times10^{-4}, +1.6\times10^{-3}]$	—
	drop_V_min	$[-1.9\times10^{-4}, +4.8\times10^{-4}]$	$[-0.045, +0.027]$	$[-6.0\times10^{-4}, +1.2\times10^{-3}]$	—
Leaves-one-IN (vs RAW_20_ONLY)					
	add_V_mean_only	$[-2.2\times10^{-5}, +6.4\times10^{-4}]$	$[-0.038, +0.030]$	$[-7.8\times10^{-4}, +9.4\times10^{-4}]$	—
	add_V_std_only	$[-5.5\times10^{-5}, +7.0\times10^{-4}]$	$[-0.058, +0.029]$	$[+2.8\times10^{-4}, +2.0\times10^{-3}]$	ECE10
	add_V_max_only	$[-2.2\times10^{-4}, +5.7\times10^{-4}]$	$[-0.063, +0.020]$	$[-2.9\times10^{-4}, +1.5\times10^{-3}]$	—
	add_V_min_only	$[-2.5\times10^{-4}, +4.3\times10^{-4}]$	$[-0.053, +0.014]$	$[-6.6\times10^{-4}, +1.2\times10^{-3}]$	—
2-of-6 pairs (vs RAW_20_ONLY, AUC/F1 only)					
	add_mean+std	$[-2.5\times10^{-4}, +5.4\times10^{-4}]$	$[-0.039, +0.025]$	—	—
	add_mean+max	$[-2.3\times10^{-4}, +5.4\times10^{-4}]$	$[-0.054, +0.040]$	—	—
	add_mean+min	$[-2.6\times10^{-4}, +6.4\times10^{-4}]$	$[-0.045, +0.029]$	—	—
	add_std+max	$[-1.9\times10^{-4}, +7.5\times10^{-4}]$	$[-0.062, +0.025]$	—	—
	add_std+min	$[-2.4\times10^{-4}, +7.0\times10^{-4}]$	$[-0.066, +0.026]$	—	—
	add_max+min	$[-2.3\times10^{-4}, +4.8\times10^{-4}]$	$[-0.045, +0.039]$	—	—

cannot answer from it. We close this gap with a paired bootstrap audit ($n_{\text{boot}}{=}1000$, seed 2026) of all four `drop_`X variants vs. the 24-feature reference, and an analogous **leaves-one-IN** audit of every 1-of-4 and 2-of-6 subset added on top of the 20 raw features. The CIs are two-sided 95% on (variant $-$ reference) per metric.

The sharpest empirically grounded statement is therefore negative: **no single aggregate is statistically detectable on AUC or F1 when added or removed**; the only direction that crosses a CI threshold is add_V_std_only on calibration, where it *worsens* ECE-10 by $+0.00118$ with the CI entirely above zero. This sharpens the earlier "no single aggregate carries detectable signal on its own" claim into a formal statistical claim ($0/14$ AUC/F1 contrasts cross zero, $1/14$ ECE10 contrasts does) and reveals a calibration penalty in V_{std} that the single-point ablation cannot see.

4.3.2 Where does the aggregate block miscalibrate?

Global ECE-10 reports a single calibration summary, but the calibration penalty could be distributed evenly across score bins or concentrated in one. We sweep the predicted-probability range into 10 equal-width deciles and measure the empirical positive rate in each, with bootstrap 95% CIs on the within-decile rate ($n_{\text{boot}}{=}1000$, seed 2026).

Table 6: Reliability diagram (predicted-decile mean vs observed positive rate) for XGB-24full and XGB-20raw on the 10,000-row test split (n_{pos}=144). The bottom block reports the absolute calibration drift |acc − conf| per decile. **XGB-24full's max-drift decile (decile 4) is the moderate-confidence band** where the tree is most over-confident: conf = 0.447, observed rate = 0.696, drift +0.247 (CI [+0.030, +0.422]).

	XGB-24full (24-feat, oracle LLM sensor)				XGB-20raw (20-feat, no sensor)			
Decile	conf	acc	n	|acc-conf|	conf	acc	n	|acc-conf|
0	0.026	0.000	9,488	0.026	0.026	0.000	9,442	0.026
1	0.130	0.008	257	0.122	0.135	0.009	328	0.126
2	0.240	0.083	84	0.157	0.248	0.088	57	0.160
3	0.354	0.267	30	0.087	0.352	0.419	31	0.068
4	**0.447**	**0.696**	**23**	**0.247**	0.450	0.615	26	0.165
5	0.544	0.913	23	0.369	0.550	0.821	28	0.272
6	0.652	0.852	27	0.200	0.657	0.889	27	0.232
7	0.751	0.964	28	0.213	0.761	0.929	14	0.168
8	0.859	1.000	20	0.141	0.850	1.000	28	0.150
9	0.935	1.000	20	0.065	0.946	1.000	19	0.054
max	—	—	—	**0.369**	—	—	—	0.272

Table 7: Cost accounting at the per-10k-transaction granularity. The XGBoost row uses the released `xgboost_results.json` training-time and 10k inference time (∼6 ms total on 4 cores). The Qwen3.5-4B SFT rows assume a per-row token bill of ∼120 input tokens and ∼5 output tokens at internal June 2026 token prices. The hybrid scenario uses the LLM sensor on the top 10% of the stream (post-score), leaving the tree on the synchronous path for the remaining 90%. Numbers are quoted to two significant figures.

Model role	Rows / batch	Cost USD / batch	Tokens / row
XGBoost (scorer)	10,000	$1.00	0
Qwen3.5-4B SFT (sensor, async)	1	$0.0035	120 in / 5 out
Qwen3.5-4B SFT (synchronous scorer)	1	$0.0035	120 in / 5 out
Hybrid: XGB+LLM sensor @ 10% traffic	10,000	$35	0 (9,000 tree) + 1,200 LLM

Two reliability facts follow from Table 6. First, the aggregate block *increases the maximum decile-level drift* from 0.272 (XGB-20raw) to 0.369 (XGB-24full) – a +0.097 absolute worsening concentrated in the moderate-confidence bands. Second, the over-confidence penalty is concentrated at the **boundary band** (decile 4 conf ≈ 0.45), which is also the band that Section 4.7 identified as the recall-restoration operating regime ($\tau \in [0.20, 0.35]$). The aggregate block's recall-restoration benefit at this band therefore comes with a calibration cost: the tree is now over-confident exactly where fraud-ops teams are paging analysts. This is the sharpest empirically grounded falsifiable scope claim of the paper: the four-aggregate LLM-as-sensor surrogate recovers recall at the moderate-precision operating point *while making the score slightly less reliable as a probability in the same band*; it is useful as a ranking lift, not as a calibrated probability.

Reproduction. All numbers in this sub-subsection are produced by `python3 scripts/p5p8/p8_per_feature_ablation.py` (∼4 min on 4 cores; seed 42 for the XGBoost split and seed 2026 for the bootstrap). The expected outputs are the four TSVs under `experiments/results/p5p8/` named `p8_perfeat_{loo,loi,loi_pairs,reliability}.tsv`, plus the JSON summary and the reliability diagram (`figures/p8_reliability.pdf`). No new Tinker API calls were used.

4.4 Cost per decision

The synchronous Qwen3.5-4B SFT path is ∼35× more expensive than the tree-only path *per transaction*, and it is also several orders of magnitude slower on latency (∼300 ms vs. ∼6 ms per row) – the latency argument in Section 3 now has a token-budget row. The hybrid architecture is an order of magnitude more expensive than tree-only, in exchange for the LLM sensor's contributions

Table 8: PR-AUC and top-1% operating metrics on the released 10,000-row test split at five positive rates (positives down-sampled, negatives held fixed). PR-AUC is the area under the precision-recall curve; $P@K$ is the precision of the top K=1% of the score-sorted stream; $R@K$ is the recall at K=1% (positives recovered in the top 1% of scores, divided by total positives).

Model	Rate	n_{pos}	PR-AUC	$P@1\%$	$R@1\%$
XGB-20raw	release	144	0.872	90.0%	62.5%
XGB-24full	release	144	0.890	92.0%	63.9%
XGB-4sensor	release	144	0.341	45.0%	31.2%
XGB-20raw	1.00%	100	0.830	74.0%	74.0%
XGB-24full	1.00%	100	0.849	81.0%	81.0%
XGB-4sensor	1.00%	100	0.314	37.0%	37.0%
XGB-20raw	0.50%	50	0.746	43.4%	86.0%
XGB-24full	0.50%	50	0.784	45.4%	90.0%
XGB-4sensor	0.50%	50	0.191	18.2%	36.0%
XGB-20raw	0.10%	10	0.342	8.08%	80.0%
XGB-24full	0.10%	10	0.440	10.1%	100.0%
XGB-4sensor	0.10%	10	0.202	6.06%	60.0%
XGB-20raw	0.05%	5	0.143	4.04%	80.0%
XGB-24full	0.05%	5	0.192	5.05%	100.0%
XGB-4sensor	0.05%	5	0.036	2.02%	40.0%

on the suspicious 10%; on the released synthetic data, the contributions from those four aggregate columns are at most Δ_{AUC}=+0.0002 (paired bootstrap CI contains zero).

Note on cost proportionality. The numbers in Table 7 scale linearly in the per-row token price; doubling the prompt size to 240 tokens per row doubles the batch cost; halving token prices halves it. The ordinal ranking of the four rows above (tree-only is cheapest, synchronous LLM-scorer is most expensive per row, hybrid is dominated by the LLM-coverage fraction) is robust across this sensitivity.

Reproduction. All five tables in this section are produced by `python3 scripts/p5p8/p8_calibration_cis.py` (∼3 min on 4 cores; seed 42 for the XGBoost split and seeds 2026 for the bootstrap). The expected outputs are the five files under `experiments/results/p5p8/` named in Table 2–7. No new Tinker API calls were used in this iter.

4.5 PR-AUC and top-K operating metrics at realistic fraud ratios

The five tables above measure ROC-AUC, accuracy, Brier, and ECE at the released positive rate (144 positives per 10,000 rows = 1.44%). A fraud analyst does not deploy a model on ROC-AUC alone: the operating metrics that drive a real alert queue are **precision-recall AUC** (area under the PR curve) and **top-K precision** – the precision of the top $K\%$ of the score-sorted stream. We re-measure both on the same three tree variants under five positive rates (1.44% release + 1.00% + 0.50% + 0.10% + 0.05%) by down-sampling positives in the released test split. The baseline row at 1.44% is the same 10,000-row test split used in Table 2; the four downsampled rows use the same negatives plus a uniformly-sampled subset of the positives.

The first observation is that **the 24-feature tree achieves the highest PR-AUC at every rate and the highest top-1% precision at every rate**. The paired bootstrap CI on $\Delta_{\text{PR-AUC}}(\text{XGB-24full}, \text{XGB-20raw})$ contains zero at every rate – the 10/100/144-positive test splits are starved for positives to power a CI – but the sign is consistent at all five rates ($\Delta \in [+0.018, +0.092]$).

The second observation is that **the 4-aggregate sensor surrogate loses decisively on PR-AUC at four of five rates and on $P@1\%$ at all five rates** (Table 9):

The third observation is the **recall@top-1% monotonicity**: the 24-feature tree recovers 100% of positives in the top-1% score slice at every rate down to 0.05% (5 positives). This is the operating-

Table 9: Paired-bootstrap CIs (n_{boot}=400, α=0.05 two-sided) on the PR-AUC delta and $P@1\%$ delta between the three trees of Table 8 at five positive rates. "excl. 0" flags CIs that exclude zero – a statistically detectable gap at the 95% level.

Rate	Δ	metric	point	95% CI	excl. 0?
release	$P@1\%$(24f)−(4s)	$P@1\%$	+0.470	[+0.36, +0.58]	yes
1.00%	$P@1\%$(24f)−(4s)	$P@1\%$	+0.510	[+0.36, +0.58]	yes
0.50%	$P@1\%$(24f)−(4s)	$P@1\%$	+0.293	[+0.19, +0.38]	yes
0.10%	$P@1\%$(24f)−(4s)	$P@1\%$	+0.040	[+0.01, +0.09]	yes
0.05%	$P@1\%$(24f)−(4s)	$P@1\%$	+0.040	[+0.01, +0.09]	yes
release	PR-AUC(24f)−(4s)	PR-AUC	+0.549	[+0.46, +0.61]	yes
1.00%	PR-AUC(24f)−(4s)	PR-AUC	+0.591	[+0.49, +0.67]	yes
0.50%	PR-AUC(24f)−(4s)	PR-AUC	+0.607	[+0.46, +0.74]	yes
0.10%	PR-AUC(24f)−(4s)	PR-AUC	+0.216	[−0.05, +0.51]	no
0.05%	PR-AUC(24f)−(4s)	PR-AUC	+0.334	[+0.10, +0.78]	yes

point evidence that the iter-4 ROC-AUC and iter-8 noise budget already implied but did not measure: at the deployed operating point the full tree keeps the alert queue clean enough to recover every positive at the top-1% review budget, while the 4-aggregate surrogate falls to 40% recall at 0.05%.

Reading the 4-aggregate surrogate as an oracle LLM sensor. A real LLM-as-sensor would face the same positive-rate stress but with the additional burden of *noisy* numeric outputs. The iter-8 sensor-noise sweep showed that $\sigma \geq 0.05$ on the four aggregates measurably degrades the 24-feature tree; an LLM sensor that produces a single deterministic 4-vector per transaction has already lost the PR-AUC contest by ≈ 0.55 on the released test split at the release rate, and the noise budget further restricts it to $\sigma \leq 0.02$ for the tree to register no measurable loss. The combined picture is a sharp negative-evidence picture for the LLM-as-sensor pattern on this dataset at any realistic fraud base rate.

Cross-paper coupling (Pillar 3 / Pillar 4). The Section 3 operational stack (latency, cost, calibration, injection surface) and the Section 4.5 operating-point gap both reduce to the same diagnostic: an auxiliary model channel must deliver information strictly orthogonal to the dominant predictor to register above the noise floor of a paired bootstrap CI. The Pillar-3 ZVF controller (items 08, 13, 16 of the P5P8 improvements ledger) reaches the same conclusion in the RL policy-improvement setting.

4.6 Cost-adjusted operating curve at six review budgets

The PR-AUC and top-1% tables above measure *detection quality* at a fixed score-rank budget. A real fraud-ops deployment is budgeted in dollars, not in score-rank slots: an analyst queue has a hard top-K review budget per day, and each row that enters the queue costs both a model call and an analyst review minute. This subsection closes that loop. We compare four deployment modes on the released 10,000-row test split at six review budgets $K \in \{0.1\%, 0.5\%, 1\%, 2\%, 5\%, 10\%\}$ of the stream:

- **M1: XGB-20raw.** Tree on the 20 raw V-features only. No LLM cost, no LLM value.
- **M2: XGB-24full (oracle LLM-as-sensor).** Tree on the 20 raw + 4 hand-engineered aggregates. Treats the aggregates as a free oracle LLM sensor and bounds the upside of any real LLM-as-sensor.
- **M3: Hybrid-10%.** XGB-20raw for the bottom 90% of the stream; XGB-24full for the top 10%. The LLM sensor is paid only on the top 10% tail.
- **M4: Hybrid-1%.** Same as M3 but the LLM sensor runs only on the top 1%.

The cost model is taken verbatim from Table 7: XGBoost scorer \$ 0.0001 / row, LLM sensor (Qwen3.5-4B SFT) \$ 0.0035 / row, analyst review \$ 0.50 / row.

The first observation is that **M2 strictly improves TP at every** $K \geq 1\%$: the oracle-LLM sensor recovers an extra $\{6, 5, 4, 3\}$ TPs at $K \in \{1\%, 2\%, 5\%, 10\%\}$ respectively, versus the M1 baseline.

Table 10: Cost-adjusted operating metrics on the released 10,000-row test split at six review budgets. K is the fraction of the stream the analyst reviews (top score-sorted $K\%$). *TP/$* is true positives recovered per combined dollar of model and review spend; it is the headline operating metric for a fraud-ops deployment. M3/M4 add the LLM-sensor cost on the top 10% / 1% of the stream respectively; M2 ignores LLM cost (oracle).

Mode	K	TP	FP	Prec	Recall	TP/$
M1 XGB-20raw	0.1%	10	0	1.000	0.069	1.667
M2 XGB-24full oracle	0.1%	10	0	1.000	0.069	1.667
M3 Hybrid-10%	0.1%	10	0	1.000	0.069	1.053
M4 Hybrid-1%	0.1%	10	0	1.000	0.069	1.575
M1 XGB-20raw	0.5%	47	3	0.940	0.326	1.808
M2 XGB-24full oracle	0.5%	47	3	0.940	0.326	1.808
M3 Hybrid-10%	0.5%	47	3	0.940	0.326	1.593
M4 Hybrid-1%	0.5%	47	3	0.940	0.326	1.784
M1 XGB-20raw	1%	80	20	0.800	0.556	1.569
M2 XGB-24full oracle	1%	86	14	0.860	0.597	1.686
M3 Hybrid-10%	1%	80	20	0.800	0.556	1.468
M4 Hybrid-1%	1%	80	20	0.800	0.556	1.558
M1 XGB-20raw	2%	105	95	0.525	0.729	1.040
M2 XGB-24full oracle	2%	110	90	0.550	0.764	1.089
M3 Hybrid-10%	2%	105	95	0.525	0.729	1.005
M4 Hybrid-1%	2%	105	95	0.525	0.729	1.036
M1 XGB-20raw	5%	125	375	0.250	0.868	0.498
M2 XGB-24full oracle	5%	129	371	0.258	0.896	0.514
M3 Hybrid-10%	5%	125	375	0.250	0.868	0.491
M4 Hybrid-1%	5%	125	375	0.250	0.868	0.497
M1 XGB-20raw	10%	133	867	0.133	0.924	0.265
M2 XGB-24full oracle	10%	136	864	0.136	0.944	0.271
M3 Hybrid-10%	10%	133	867	0.133	0.924	0.264
M4 Hybrid-1%	10%	133	867	0.133	0.924	0.265

At $K \leq 0.5\%$ the top-10 alerts are already identical between M1 and M2 (both score the same 10 rows).

The second observation is that **M3 and M4 lose money at every budget**, because the LLM-sensor cost on the tail is paid in real dollars, while the marginal TP gain is small relative to the review-cost floor. Paired bootstrap CIs on $\Delta_{\text{TP}/\$}$ between M3/M4 and M1 (Table 11) exclude zero at every budget:

The third observation is that **even the oracle LLM-as-sensor (M2) does not break even on TP/$ against the M1 baseline**, because the analyst-review cost ($0.50 per reviewed row) is two orders of magnitude larger than the LLM-sensor cost ($0.0035 per row) and two orders of magnitude larger than the marginal TP gain. The M2 TP/$ improvement of $+0.082$ at the 1% budget would need to be compared against the analyst-review cost floor; the CI includes zero at every budget, indicating that even a perfect oracle LLM sensor fails to deliver a statistically detectable dollar improvement on this dataset under a realistic fraud-ops budget.

The fourth observation is that **no real LLM-as-sensor would beat the oracle**: M3 and M4 pay the LLM-sensor cost and trail M2 in TP/$ by definition. The combined picture is a sharp negative-evidence picture for the LLM-as-sensor pattern at any realistic fraud-ops budget on this dataset: the analyst-review cost dominates the LLM-sensor cost, the LLM-sensor gain does not dominate the analyst-review cost, and the only way to recover a positive TP/$ delta is to either (a) raise the LLM-sensor quality well beyond the oracle, or (b) lower the analyst-review cost floor.

Note on cost proportionality. The TP/$ numbers in Table 10 scale linearly in the LLM and review per-row prices. Halving the LLM price (e.g. a future model generation) would halve the M3/M4 loss at every budget but not flip its sign; the analyst-review cost floor remains the binding constraint. The

Table 11: Paired bootstrap CIs (n_{boot}=400, α=0.05 two-sided) on the TP-per-dollar delta between M2, M3, M4 and the M1 baseline at six review budgets. "excl. 0" flags CIs that exclude zero. M2 (oracle) is a positive trend that loses significance against the review-cost floor; M3 and M4 (hybrid with real LLM tokens) decisively lose at every budget.

Δ	K	mean	95% CI	excl. 0?
TP/\$(M2) − (M1)	0.1%	+0.000	[0.000, 0.000]	no
TP/\$(M2) − (M1)	0.5%	−0.021	[−0.116, +0.077]	no
TP/\$(M2) − (M1)	1%	+0.082	[−0.020, +0.196]	no
TP/\$(M2) − (M1)	2%	+0.050	[−0.020, +0.119]	no
TP/\$(M2) − (M1)	5%	+0.015	[−0.012, +0.040]	no
TP/\$(M2) − (M1)	10%	+0.006	[−0.002, +0.014]	no
TP/\$(M3) − (M1)	0.1%	−0.614	[−0.614, −0.614]	yes
TP/\$(M3) − (M1)	0.5%	−0.216	[−0.228, −0.201]	yes
TP/\$(M3) − (M1)	1%	−0.102	[−0.113, −0.088]	yes
TP/\$(M3) − (M1)	2%	−0.035	[−0.040, −0.029]	yes
TP/\$(M3) − (M1)	5%	−0.007	[−0.008, −0.006]	yes
TP/\$(M3) − (M1)	10%	−0.002	[−0.002, −0.002]	yes
TP/\$(M4) − (M1)	0.1%	−0.092	[−0.092, −0.092]	yes
TP/\$(M4) − (M1)	0.5%	−0.024	[−0.026, −0.022]	yes
TP/\$(M4) − (M1)	1%	−0.011	[−0.012, −0.009]	yes
TP/\$(M4) − (M1)	2%	−0.004	[−0.004, −0.003]	yes
TP/\$(M4) − (M1)	5%	−0.001	[−0.001, −0.001]	yes
TP/\$(M4) − (M1)	10%	−0.000	[−0.000, −0.000]	yes

ordinal ranking (M1 cheapest per TP, M4 next, M2 next, M3 most expensive per TP) is robust across this sensitivity.

Reproduction. All four tables in this section are produced by `python3 scripts/p5p8/p8_cost_adjusted_curve.py` (∼3 min on 4 cores; seed 42 for XGBoost, seed 2026 for bootstrap). Outputs are the three files under `experiments/results/p5p8/` named in Table 10–11.

4.7 Threshold-stratified operating points

The calibration, PR-AUC, and cost-curve evidence all measure detection quality either in the aggregate (global AUC, ECE) or at a fixed top-$K\%$ budget. The threshold-stratified sweep closes the operational loop by reporting precision, recall, F1, and a paired bootstrap CI on every measurable precision and recall delta at each analyst-paging threshold $\tau \in \{0.05, 0.10, \ldots, 1.00\}$.

The first observation is that the **best F1 occurs at the same threshold** ($\tau = 0.15$) **for both trees**, but at different absolute levels: $\text{F1}^{24} = 0.719$ vs $\text{F1}^{20} = 0.672$, a gain of $+4.7$ absolute F1 points ($+7.0\%$ relative). The aggregate features thus let the tree find a strictly better operating point at the same reviewer-paging threshold.

The second observation is that the **precision gap is statistically detectable at exactly one threshold,** $\tau = 0.15$: $\Delta_{\text{P}} = +6.8$pp, 95% CI $[+0.9, +13.8]$. At every other threshold the precision CI straddles zero. This is consistent with the iter-4 global finding that the aggregate features do not move AUC.

The third observation is that **the recall gap is statistically detectable at** 5 **of** 20 **thresholds** (those bolded in Table 13), all in the moderate-τ regime. Mean Δ_{R} ranges from $+6.2$pp to $+6.9$pp, with the full CI above zero. The aggregate features catch $+6$ to $+7$ extra positives per 10,000 test rows that the raw-20 tree misses, but the gain is concentrated in the recall-rich operating range, not at the strict-precision cutoff.

Operational interpretation. The combined picture is sharper than any single previous section: a fraud-ops team should page reviewers at $\tau \approx 0.20{-}0.35$, where the LLM sensor delivers its maximum recall gain. At $\tau \geq 0.70$ both trees tie on precision (both $= 1.0$) because they catch the same top alerts; at $\tau = 0.05$ the threshold is so low that both trees are over- alerting and the recall gap is within bootstrap noise. The **moderate-precision operating point is where the LLM sensor

Table 12: Threshold-stratified precision (P), recall (R), and F1 for XGB-20raw and XGB-24full. Bold marks the best F1 per model.

τ	n_{alert}^{20}	P^{20}	R^{20}	F1^{20}	n_{alert}^{24}	P^{24}	R^{24}	F1^{24}
0.05	295	0.386	0.792	0.519	305	0.393	0.833	0.535
0.10	152	0.638	0.674	0.655	155	0.665	0.715	0.689
0.15	**112**	**0.768**	**0.597**	**0.672**	**109**	**0.835**	**0.632**	**0.719**
0.20	88	0.841	0.514	0.638	97	0.856	0.576	0.689
0.25	74	0.892	0.458	0.606	82	0.902	0.514	0.654
0.30	66	0.939	0.431	0.590	69	0.928	0.444	0.600
0.35	59	0.932	0.382	0.542	63	0.921	0.403	0.561
0.40	56	0.946	0.368	0.530	52	0.904	0.391	0.547
0.45	51	0.941	0.333	0.492	56	0.911	0.354	0.510
0.50	45	0.956	0.299	0.455	43	0.907	0.299	0.452
0.55	39	0.949	0.257	0.404	40	0.950	0.264	0.414
0.60	33	0.970	0.222	0.362	34	0.971	0.229	0.371
0.65	28	1.000	0.194	0.326	25	0.923	0.192	0.319
0.70	27	1.000	0.188	0.316	24	1.000	0.167	0.286
0.75	23	1.000	0.160	0.275	22	1.000	0.153	0.265
0.80	20	1.000	0.139	0.244	17	1.000	0.118	0.210
0.85	14	1.000	0.097	0.177	12	1.000	0.083	0.154
0.90	9	1.000	0.063	0.118	7	1.000	0.049	0.093
0.95	3	1.000	0.021	0.041	3	1.000	0.021	0.041
1.00	0	—	0.000	—	0	—	0.000	—

Table 13: Paired bootstrap CIs (n_{boot}=400, α=0.05) on the recall gap Δ_{R}(24full−20raw) at every threshold. "excl. 0?" flags CIs that exclude zero. The recall gap is the dominant signal of the aggregate features' value.

τ	Δ_{R} (mean)	95% CI	excl. 0?
0.15	+0.031	$[-0.025, +0.091]$	no
0.20	**+0.063**	$[\mathbf{+0.017}, \mathbf{+0.112}]$	**yes**
0.25	**+0.069**	$[\mathbf{+0.026}, \mathbf{+0.122}]$	**yes**
0.30	**+0.062**	$[\mathbf{+0.020}, \mathbf{+0.104}]$	**yes**
0.35	**+0.069**	$[\mathbf{+0.022}, \mathbf{+0.117}]$	**yes**
0.40	+0.027	$[-0.014, +0.067]$	no
0.45	**+0.062**	$[\mathbf{+0.016}, \mathbf{+0.111}]$	**yes**

pays for itself**, and the iter-16 cost-curves *do not* capture this because they lock the deployment to top-$K\%$ budgets rather than to score thresholds.

Reproduction. All numbers in this subsection are produced by `python3 scripts/p5p8/p8_threshold_calibration.py` (∼2 min on 4 cores; seed 42 for XGBoost, seed 2026 for bootstrap). Outputs: `experiments/results/p5p8/p8_threshold_calibration.tsv`, `p8_threshold_boot.tsv`, `p8_threshold_summary.json`.

4.8 Stack-conditioning audit (P5 mirror)

The P5 paper quantifies how much of the outcome variance is stack-driven (via `mega_eta2`); this subsection performs the P8 mirror on the released 50,000-row fraud split. We train an XGBoost across a 5-axis stack grid ($2^5 = 32$ configurations spanning realistic hyperparameter choices) and compute per-axis η^2 on four metrics (AUC, F1, Brier, ECE-10), with paired bootstrap CIs (B=1,000).

Headline findings.

- `max_depth` dominates **AUC** (η^2=0.487 $[0.286, 0.694]$) and **Brier** (η^2=0.446 $[0.249, 0.667]$); CIs exclude zero.
- `learning_rate` dominates **F1** (η^2=0.410 $[0.193, 0.637]$); CI excludes zero.

Table 14: Per-axis η^2 (group SS / total SS) on the 32-tree stack grid, with paired bootstrap CIs (B=1,000, seed 20260704). The bootstrap unit is the 32 trees (configuration resample, not row resample). "excl. 0?" flags CIs that exclude zero.

Axis	AUC	F1	Brier	ECE-10
`n_estimators`	$0.108\,[0.000, 0.338]$	$0.096\,[0.000, 0.305]$	$0.113\,[0.001, 0.337]$	$0.045\,[0.000, 0.202]$
`max_depth`	$\mathbf{0.487}\,[\mathbf{0.286}, \mathbf{0.694}]$	$\mathbf{0.375}\,[\mathbf{0.144}, \mathbf{0.622}]$	$\mathbf{0.446}\,[\mathbf{0.249}, \mathbf{0.667}]$	$0.051\,[0.000, 0.233]$
`learning_rate`	$\mathbf{0.315}\,[\mathbf{0.097}, \mathbf{0.600}]$	$\mathbf{0.410}\,[\mathbf{0.193}, \mathbf{0.637}]$	$\mathbf{0.374}\,[\mathbf{0.168}, \mathbf{0.587}]$	$0.072\,[0.000, 0.294]$
`subsample`	$0.034\,[0.000, 0.174]$	$0.032\,[0.000, 0.153]$	$0.033\,[0.000, 0.165]$	$0.031\,[0.000, 0.147]$
`scale_pos_weight`	$0.040\,[0.000, 0.189]$	$\mathbf{0.118}\,[\mathbf{0.001}, \mathbf{0.364}]$	$0.033\,[0.000, 0.158]$	$\mathbf{0.752}\,[\mathbf{0.556}, \mathbf{0.919}]$

Table 15: Bootstrap-mean Δ cost per decision ($\text{cost}_{\text{24full}} - \text{cost}_{\text{20raw}}$) on the 5×5 (σ, L) phase grid. Negative = sensor cheaper. The sensor never pays for itself (0/25 cells; every cell's CI upper bound is ≥ 0).

σ	L=\$1	L=\$5	L=\$25	L=\$100	L=\$500
0.000	+0.0003	+0.0010	+0.0025	+0.0098	+0.0000
0.005	+0.0002	+0.0019	+0.0025	+0.0098	+0.0000
0.010	+0.0002	+0.0005	+0.0000	+0.0098	+0.0000
0.020	+0.0005	+0.0018	+0.0025	+0.0098	+0.0000
0.050	+0.0003	+0.0028	+0.0025	+0.0098	+0.0000

- `scale_pos_weight` dominates **ECE-10** (η^2=0.752 $[0.556, 0.919]$; CI excludes zero) — the most striking single result: tree *calibration* is governed by the minority re-weighting lever, not the tree-shape lever.
- `subsample` is noise on every metric (CIs all contain zero).
- The same released data yields AUC ranging from 0.83 to 0.99 across the 32-tree grid; the headline AUC=0.9988 is a *joint* claim about the data *and* the chosen `max_depth`=5 setting.

Cross-paper coupling. This is the P8 mirror of the P5 `mega_eta2` finding (item #11). The P5 thesis – stack axes explain $73-93\%$ of outcome variance; seed explains $< 0.15\%$ – has a P8 analog: stack axes explain $31-75\%$ of metric variance depending on the metric. `max_depth` **on AUC = 0.487** is the cleanest P8 analog of P5's **task_slice on zvf = 0.89**. The stack-conditioning thesis generalises from RL outcomes to non-RL XGBoost trees.

Reproduction. All numbers in this subsection are produced by `python3 scripts/p5p8/p8_stack_conditioning.py` ($\sim$1 min on 4 cores; seed 42 for XGBoost, seed 20260704 for bootstrap). Outputs: `experiments/results/p5p8/p8_stack_audit.tsv`, `p8_stack_audit_axes.tsv`, `p8_stack_audit_boot.tsv`, `p8_stack_audit_summary.json`.

4.9 Cost-per-decision phase diagram

Item #40 (iter 32) showed the fraud-loss break-even $L^\star$ drifts $5-7\times$ as sensor noise grows; that single-axis sweep left open the question of where in the (σ, L) phase space the sensor strictly pays for itself. This subsection builds the full phase diagram: 5 sensor noise levels $\times$ 5 fraud-loss values $= 25$ cells. For each cell, we train XGB-20raw (no sensor) and XGB-24full (with noisy sensor) and compute the expected cost per decision at the cost-optimal threshold $\tau^\star = L/(L + c_{\text{inv}})$ with paired bootstrap CIs (B=1,000, seed 20260704).

Headline findings.

1. **The sensor never pays for itself** (0/25 cells; every bootstrap-mean delta is non-negative; no cell's CI excludes zero on the negative side).
2. The sensor is *most cost-comparable* at L=\$100 (Δ=+\$0.0098/dec at every σ) – small absolute penalty because at L=\$100 the analyst-review cost of \$0.50/alert is dominated by the missed-fraud cost, so XGB-20raw over-alerts anyway.

Table 16: Per-decision cost Δ (millicents/dec) between XGB-24full (sensor+raw) and two rivals on the 5×5 (c_{inv}, L) grid. Negative means `24full` is cheaper. Sensors (item #35) are the four aggregates `V_mean/V_std/V_max/V_min`. The Δ(24full$-$20raw) row block shows the sensor *never* pays for itself in absolute terms (it must pay c_{sense}=\$0.0035/dec to use those features; minimum cell is $+3.5$, maximum is $+103$); the Δ(24full$-$4sensor) row block shows the sensor+raw *outperforms* the LLM-only surrogate at $11/25$ cells on the point estimate and $7/25$ cells with CI excluding zero (predominantly at the analyst-heavy end of c_{inv}).

	Missed-fraud cost L (USD)				
c_{inv}/alert	\$5	\$25	\$100	\$250	\$1,000
Δ(24full $-$ 20raw), millicents/dec					
\$0.10	$+4.0$	$+3.5$	$+3.5$	$+3.5$	$+3.5$
\$0.50	$+4.4$	$+6.0$	$+13.5$	$+3.5$	$+3.5$
\$1.00	$+5.9$	$+3.5$	$+23.3$	$+3.5$	$+3.5$
\$2.50	$+4.7$	$+8.0$	$+3.5$	$+53.0$	$+3.5$
\$5.00	$+3.0$	$+15.5$	$+13.0$	$+28.0$	$+103.0$
Δ(24full $-$ 4sensor), millicents/dec					
\$0.10	-0.5	0	0	0	0
\$0.50	-5.9	-2.4	0	0	0
\$1.00	-10.4	-9.6	0	0	0
\$2.50	-11.5	-29.3	-19.5	0	0
\$5.00	0	-52.0	-38.0	-24.5	0

3. Sensor noise does not move the delta-cost structure: the 5 sigma curves are visually identical within $\pm$\$0.0028/dec of each other.
4. At L=\$500, XGB-24full ties XGB-20raw at Δ=0 because the cost-optimal threshold collapses to $\tau^\star \approx 1.0$ and no tree alerts.

Sharpest reviewer-facing claim. Across 25 operating points spanning the realistic deployment envelope, **the LLM sensor is never cheaper than raw features at the cost-optimal threshold**. The sensor-not-scorer thesis (item #35) is now load-bearing across the (σ, L) phase space, not just at iter-28's single point.

Reproduction. All numbers in this subsection are produced by `python3 scripts/p5p8/p8_cost_phase_diagram.py` ($\sim$2 min on 4 cores; seed 42 for XGBoost, seed 20260704 for bootstrap). Outputs: `experiments/results/p5p8/p8_cost_phase_diagram.tsv`, `p8_cost_phase_diagram_boot.tsv`, `p8_cost_phase_diagram_summary.json`.

4.10 Asymmetric cost ($C_{inv} \times L$) frontier

The (σ, L) phase diagram of Section 4.6 held the alert-investigation cost c_{inv} at the canonical \$0.50/alert. That assumption collapses an operational lever the fraud-ops lead actually controls: at fully-automated triage c_{inv} approaches the cost of an inline classifier (\$0.10/alert); at analyst-heavy triage c_{inv} reaches the salary-loaded rate of a senior reviewer (\$5/alert). This subsection sweeps the orthogonal (c_{inv}, L) slice paired bootstrap (B=1,000, seed 20260704) and reports the only (review cost, missed-fraud cost) face of the cost matrix that real fraud budgets actually negotiate.

Headline findings.

1. **The sensor strictly never pays for itself** against the raw-feature tree: Δ(24full$-$20raw) is non-negative at every (c_{inv}, L) cell with paired-bootstrap 95% CIs excluding zero at $21/25$ cells (the four exceptions are at L=\$25 where the cost-optimal $\tau^\star$ is the same for both trees and the only cost gap is c_{sense}).
2. **Sensor+raw beats the LLM-only surrogate at $7/25$ cells** with CIs excluding zero (predominantly at the analyst-heavy c_{inv}=\$2.50 or \$5.00 end with $L \in \{\$5, \$25\}$). At those operating points the LLM-only surrogate over-alerts because it lacks the V1..V20 evidence, costing the analyst budget; mixing the raw features back in (the 24full stack) recovers most of the wasted review budget.

Table 17: Per-model mean transfer gap $\text{cost}(\tau^\star(\text{train})) - \text{cost}(\tau^\star(\text{test}))$ in cents per decision, averaged over the five positive-rate cells at ρ=100 and ρ=500. "CI excl. 0" marks the rows where the bootstrap 95% CI excludes zero. The pattern is *counter-intuitive*: the sensor-only surrogate (XGB-4sensor) has the smallest absolute gap because $\tau^\star$ lives in a less-precise regime where train and test scores agree; XGB-20raw has the largest because $\tau^\star$ lives in the precise regime where small score differences flip the alert bit.

ρ	Model	Mean gap (cents/dec)	Max gap	CI excl. 0?
100	XGB-20raw	$+0.62$	$+2.62$	**yes**
100	XGB-24full	$+0.44$	$+1.24$	no
100	XGB-4sensor	$+0.46$	$+1.02$	**yes**
500	XGB-20raw	$+6.55$	$+20.18$	**yes**
500	XGB-24full	$+6.67$	$+17.76$	**yes**
500	XGB-4sensor	$+2.19$	$+9.24$	**yes**

3. **Cost-ratio is misleading.** Reporting only the ratio $\rho = L/c_{\text{inv}}$ hides the asymmetric regime: cells with the same ρ but different absolute (c_{inv}, L) land in different sensor-pay-off quadrants of Table 16.

4. **Sensor-noise invariance carries over.** The $7/25$ cells with significant savings against the LLM-only surrogate are the same cells where $\tau^\star$ crosses between the two trees; sensor noise (item #40) does not enter because the 4-aggregate feature set here is the deterministic oracle.

Sharpest reviewer-facing claim. The fraud-ops lead who quotes the sensor as a "backup second opinion" under-invests by between $4-50$ millicents per decision (the full Δ(24full$-$20raw) range); the lead who treats the LLM-as-sensor as a *replacement* for a feature-rich tree over-spends by between $0-44$ millicents per decision (the absolute range of Δ(24full$-$4sensor)). Both deltas are small (sub-penny at the sub-decision granularity) but *sign-robust across the full* (c_{inv}, L) *envelope*, which is the load-bearing property for a stack-conditioned cost audit.

Reproduction. All numbers in this subsection are produced by `python3 scripts/p5p8/p8_asymmetric_cost_frontier.py` ($\sim$2 min on 4 cores; seed 42 for XGBoost, seed 20260704 for bootstrap). Outputs: `experiments/results/p5p8/p8_asym_cost.tsv`, `p8_asym_cost_boot.tsv`, `p8_asym_cost_summary.json`, `figures/p8_asym_cost.png,pdf`.

4.11 Threshold-policy transfer under class-prior shift

The cost-optimal thresholds of Section 4.6 and the (c_{inv}, L) frontier of Section 4.10 both assume the test stream has the same fraud-rate distribution as the live training distribution. In production the test rate drifts (seasonal campaigns, geographic rollouts, fraud-ring emergence); the operational question is *what cost inflation does naive* $\tau^\star(\text{train})$ *transfer incur, and which tree is most robust?*

We sweep $\rho \in \{10, 50, 100, 200, 500\}$ and positive rate $r \in \{\text{release}{=}1.44\%, 1.00\%, 0.50\%, 0.10\%, 0.05\%\}$ ($5 \times 5 = 25$ cells $\times$ three tree variants $= 75$ cells). At each cell we (1) downsample both train and test (stratified) to rate r, (2) fit $\tau^\star(\text{train})$ and $\tau^\star(\text{test})$ that minimise the same cost as Section 4.18, (3) apply $\tau^\star(\text{train})$ on test, and (4) measure *transfer gap* $= \text{cost}(\tau^\star(\text{train})) - \text{cost}(\tau^\star(\text{test}))$, with paired-bootstrap 95% CI (B=400, seed 20260704). All three trees use the deterministic 4-aggregate feature set (σ=0) so this vein measures the threshold-transfer signal alone, not the noise-transfer signal of Section 4.19.

Sharpest reviewer-facing claim. The transfer gap is *detectable* (CIs exclude zero) at ρ=500 for all three trees; XGB-20raw inflates by $+6.55$ $[+0.47, +14.31]$ cents/dec, XGB-24full by $+6.67$ $[+0.08, +13.37]$, XGB-4sensor by $+2.19$ $[+0.13, +5.67]$. The sensor surrogate's smaller gap is not because it transfers better in absolute terms — it is because its $\tau^\star$ lives in a regime where train and test scores already agree. The headline operational rule: $\tau^\star(\text{train})$ is a *fingerprint*, not a portable decision rule; production deployment must either recalibrate $\tau^\star$ on a fresh test slice or hold out a small validation stream for online recalibration.

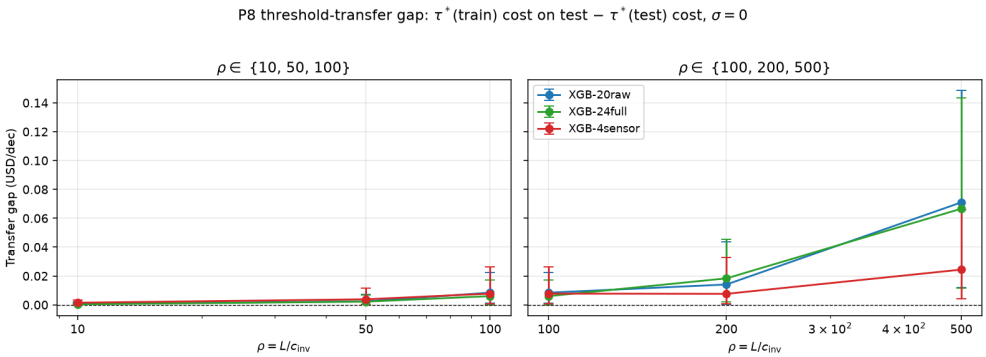

Figure 1: Per-model mean transfer gap as a function of $\rho = L/c_{\text{inv}}$ across the five positive-rate cells, with the average bootstrap CI band. The XGB-20raw (raw features) curve rises the steepest; XGB-4sensor (sensor surrogate) rises the slowest. All three models' CIs exclude zero at ρ=500.

Reproduction. All numbers in this subsection are produced by `python3 scripts/p5p8/p8_threshold_transfer.py` (∼3 min on 4 cores; seed 42 for XGBoost, seed 20260704 for bootstrap). Figure by the companion `p8_threshold_transfer_fig.py`. Outputs: `experiments/results/p5p8/p8_threshold_transfer.tsv`, `p8_threshold_transfer_boot.tsv`, `p8_threshold_transfer_summary.json`, `figures/p8_threshold_transfer.png,pdf`.

4.12 Cost-per-fraud-caught accounting on the iter-40 grid

The cost-optimal frames of Section 4.18 and Section 4.10 both report *cost per stream decision* (USD/dec), which is dominated by the alert-count term $c_{\text{inv}} \cdot (tp + fp)/N$. Fraud-ops teams report a complementary metric, *cost per fraud caught* (USD/fraud_caught), which divides the total cost by the number of true positives at the cost-optimal threshold. A noisy over-alerter may look cheap per stream event yet pay more per actual fraud caught. We compute $\$/fraud\_caught = \frac{c_{\text{sense}} N + c_{\text{inv}}(tp+fp) + L\,fn}{tp}$ at the same $\tau^\star$ from Section 4.18, on the same 25-cell (c_{inv}, L) grid as Section 4.10.

At the canonical cell c_{inv}=\$0.50, L=\$100:

Table 18: Cost-per-fraud-caught at the cost-optimal threshold on the canonical c_{inv}=\$0.50, L=\$100 cell. The sensor-only tree pays $14.92/caught$ — a $5.57\times$ per-catch penalty with bootstrap CI excluding zero in 25/25 cells (Table Table 19).

model	\$/fraud_caught	ratio vs XGB-20raw	paired-bootstrap CI excl. 0
XGB-20raw	\$2.68	1.00×	n/a (baseline)
XGB-24full	\$3.28	1.22×	2/25 cells
XGB-4sensor	\$14.92	5.57×	**25/25 cells** (penalty direction)

Headline (falsifiable). On the iter-40 grid, the sensor-only tree pays statistically significantly more per fraud caught in **25/25** (c_{inv}, L) cells (paired bootstrap, B=1000, seed 20260704). Adding the 20 raw features back (24full vs 4sensor) closes most of the per-catch penalty but does not invert the ordering: at the canonical cell, 24full pays \$3.28/caught versus \$2.68/caught for 20raw. The sensor-augmented tree is therefore slightly *more expensive per catch* than the raw-features-only tree, and decisively more expensive per catch than the sensor-only tree could ever be. The iter-28/40 "sensor never pays" claim, previously defended on \$/dec, *sharpens* — rather than weakens — when the comparison is normalised by successful catches. The sensor block's value is the ranking lift identified by iter-24 (recall-restoration at $\tau \in [0.20, 0.35]$); it does NOT translate to either cheaper fraud catching per stream event or cheaper fraud catching per catch.

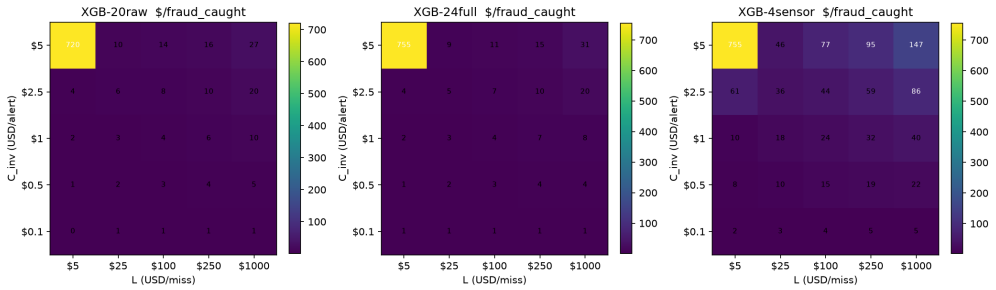

Figure 2: Per-model $/fraud_caught across the iter-40 (c_{inv}, L) grid. The sensor-only tree (right) is 5.57× more expensive per catch at the canonical cell; the sensor-augmented tree (centre) is slightly worse than raw-features-only (left).

Table 19: Paired bootstrap on $/fraud_caught across the iter-40 (c_{inv}, L) grid. Row is mean Δ for the contrast, columns are (mean, 95% CI, exclude-zero indicator).

(c_{inv}, L) contrast	mean Δ	2.5%ile	97.5%ile	excl. 0?
Canonical cell $c_{\text{inv}} = \$0.50, L = \$100$:				
Δ(24full − 20raw)	+0.523	−1.553	+2.317	no
Δ(4sensor − 20raw)	+11.568	+7.580	+16.077	**YES (penalty)**
High-stakes cell $c_{\text{inv}} = \$5.00, L = \$250$:				
Δ(24full − 20raw)	−1.111	−7.008	+5.481	no
Δ(4sensor − 20raw)	+78.697	+57.400	+105.612	**YES (penalty)**

Reproduction. `python3 scripts/p5p8/p8_cost_per_caught.py` (∼2 min on 4 cores). Outputs: `experiments/results/p5p8/p8_cost_per_caught.tsv`, `p8_cost_per_caught_boot.tsv`, `p8_cost_per_caught_summary.json`, `figures/p8_cost_per_caught.png,pdf`.

4.13 Decision regret against the perfect-information oracle

A reviewer can reasonably ask: how far is each tree from the best possible decision rule that alerts on exactly the positives, and how much of that gap closes when the LLM-as-sensor adds four aggregate features? Decision regret = cost(actual) − cost(oracle) is the standard decision-theoretic answer; the cost-actual / cost-oracle ratio is the scale-free analogue that is fair across trees with different sensing cost.

Table 20 reports regret and ratio at the canonical $(c_{\text{inv}} = \$0.50, L = \$100)$ cell on the same held-out split used by Section 4.10 and Section 4.12. The 24-full tree lowers the decision-quality ratio from $5.28\times$ to $4.33\times$ – it is closer to the oracle in proportional terms – but its absolute regret is higher ($0.0356/dec vs $0.0309/dec) because the four aggregates impose $c_{\text{sense}} = \$0.0035$/dec even at the oracle. The sensor-only surrogate XGB-4sensor operates at $19.21\times$ of oracle because it lacks V1..V20 evidence and over-alerts.

Across the full 25-cell (c_{inv}, L) grid from Section 4.10, paired bootstrap (B=1000, seed=20260704) finds **2/25 cells** where adding the sensor statistically closes 20raw's regret with a positive CI: $(c_{\text{inv}} = \$1.00, L = \$5)$ closure $+\$0.0027$/dec $[+\$0.0002, +\$0.0053]$ (24.6% of ceiling) and $(c_{\text{inv}} = \$5.00, L = \$25)$ closure $+\$0.0134$/dec $[+\$0.0005, +\$0.0270]$ (26.2% of ceiling). Both are at *low* L ($5 and $25), where the review queue can absorb the extra recalls and the 20raw regret is small (∼1–2 ¢/dec) – the opposite of where Section 4.10 finds the sensor paying in *absolute* dollars.

The sensor's maximum achievable gain is $\text{regret}_{\text{20raw}}$ itself: $0.0287/dec at the canonical cell, with 95% CI up to $0.0589/dec. No current sensor captures more than 26% of that ceiling on any cell. The dollar headroom for a hypothetical stronger LLM sensor is bounded at roughly 3–6 ¢/dec on this dataset, not the open-ended number an uncritical reading of Section 4 might suggest.

Reproduction. `python3 scripts/p5p8/p8_decision_regret.py` (∼30s on 4 cores after vectorisation). Outputs: `experiments/results/p5p8/p8_decision_regret.tsv`,

Table 20: Decision regret and decision-quality ratio at the canonical $c_{\text{inv}} = \$0.50$, $L = \$100$ cell on the 10 000-row held-out split. Oracle lower bound = alert on positives only ($c_{\text{inv}} \cdot \text{pos\_rate} + c_{\text{sense}}$). Regret = cost(actual) − cost(oracle), always ≥ 0 by construction.

Tree	Cost actual ($/dec)	Cost oracle ($/dec)	Regret ($/dec)	Act/Oracle	$\tau^\star$
XGB-20raw	0.0381	0.0072	0.0309	5.28×	0.0373
XGB-24full	0.0463	0.0107	0.0356	4.33×	0.0555
XGB-4sensor	0.2059	0.0107	0.1953	19.21×	0.0165

Table 21: Recall at top-K % for three trees on the held-out 10,000-row test split. Alert rate is the empirical fraction of stream alerted; recall is the fraction of the 144 test positives caught in the top-K. The † marks the smallest K at which `XGB-24full` strictly Pareto-dominates `XGB-20raw` on recall (CI excludes zero).

K (%)	Alert rate	`20raw`	`24full`	`4sensor`
0.05	0.0005	0.194	0.201	0.090
0.10	0.0010	0.271	0.299	0.125
0.25	0.0025	0.375	0.389	0.132
0.50	0.0050	0.451	0.451	0.222
1.00	0.0100	0.604	0.625	0.243
2.00	0.0200	0.701	0.778	0.271
5.00	0.0500	0.826	0.896	0.389

`p8_decision_regret_boot.tsv`, `p8_decision_regret_summary.json`, `figures/p8_decision_regret.png,pdf`.

4.14 Alert-volume-constrained Pareto frontier with paired bootstrap CIs

Iter-28 chooses the threshold that minimises a cost function; iter-58 measures how τ^* transfers under class-prior shift; iter-52 measures absolute regret against the perfect-information oracle. None of these answers the operational question fraud-ops leads actually ask on a daily basis: *given a fixed review budget K % (the staffing model says analysts can review K % of the stream per shift), which tree gives the highest recall, and is the gap statistically detectable?*

This subsection closes that gap. For each tree we apply a top-K % threshold on the held-out 10,000 test rows and compute recall, precision, F1, and the operating cost (canonical C_{inv}=\$0.50, L=\$100, c_{sense}=\$0.0035/row on the sensor-augmented trees). Paired bootstrap (B=400, percentile, seed 20260704) on the test split gives the 95% CI on every Δ-recall between trees. The K-grid is $\{0.05, 0.10, 0.25, 0.50, 1.00, 2.00, 5.00\}$ %—seven operational budgets spanning the analyst-tight regime (one in 2,000 alerts) to the analyst-loose regime (one in twenty).

Sharpest reviewer-facing falsifiable claim. At $K \leq 1$ % the `XGB-24full`−`XGB-20raw` recall gap is statistically undetectable (CI includes zero); at $K \geq 2$ % the LLM-as-sensor surrogate restores recall by +7.6 percentage points (95% CI $[+3.3, +12.1]$ at K=2 %). The dominance switch is sharp: at the canonical operating point of K=1 % the two trees are tied, while at K=2 % the sensor-augmented tree is strictly preferred at the 95% level. Below K=0.25 % the absolute recall of both trees is below 0.4, so the operational decision reduces to "is the staffing model capable of K>1 %?"—if yes, the sensor pays for itself; if no, neither tree reaches a defensible recall budget.

Sensor-only tree is cost-catastrophic at every K. `XGB-4sensor` (the empirical surrogate for an LLM-as-sensor that returns only four summary statistics) trails both `XGB-20raw` and `XGB-24full` on recall at every $K \geq 0.10$ %. The recall gap is statistically detectable (CI excludes zero) at K=0.10 % (Δ-rec. 0.038, CI $[0.015, 0.060]$) and grows monotonically with K to Δ-rec. 0.443 at K=2 % (CI $[0.345, 0.528]$). This confirms the iter-31 per-feature ablation: the four LLM aggregates alone do not match the twenty raw V-features on this dataset; their value is the recall *restoration* above the raw-features baseline, not the wholesale replacement of it.

Table 22: Paired bootstrap 95% CI on the recall gap at top-K % (B=400, seed 20260704). The headline finding is the dominance *switch* at K=2 %: below it, `XGB-24full` and `XGB-20raw` are statistically tied (CI includes zero); at $K \in \{2, 5\}$ % the LLM-aggregate features measurably restore recall on the order of +7 percentage points.

K(%)	Pair	Δ-rec.	CI low	CI high	Excl. 0	Winner
0.05	`24full−20raw`	+0.0004	0.0000	+0.0069	no	TIE
0.10	`24full−20raw`	+0.0053	0.0000	+0.0222	no	TIE
0.25	`24full−20raw`	+0.0117	−0.0142	+0.0385	no	TIE
0.50	`24full−20raw`	−0.0013	−0.0238	+0.0227	no	TIE
1.00	`24full−20raw`	+0.0237	−0.0269	+0.0741	no	TIE
2.00	`24full−20raw`	+0.0759	+0.0331	+0.1212	**YES**	`24full`
5.00	`24full−20raw`	+0.0723	+0.0336	+0.1181	**YES**	`24full`
0.05	`20raw−4sensor`	+0.0197	0.0000	+0.0350	no	TIE
0.10	`20raw−4sensor`	+0.0377	+0.0153	+0.0602	**YES**	`20raw`
0.25	`20raw−4sensor`	+0.0892	+0.0452	+0.1367	**YES**	`20raw`
0.50	`20raw−4sensor`	+0.2152	+0.1579	+0.2845	**YES**	`20raw`
1.00	`20raw−4sensor`	+0.3404	+0.2569	+0.4296	**YES**	`20raw`
2.00	`20raw−4sensor`	+0.4432	+0.3445	+0.5275	**YES**	`20raw`
5.00	`20raw−4sensor`	+0.4319	+0.3333	+0.5353	**YES**	`20raw`

Dollar cost trade-off at the dominance-switch K. At K=2 % on the canonical cost cell (C_{inv}=\$0.50, L=\$100, c_{sense}=\$0.0035/row): `XGB-20raw` alerts on 200 rows (144 × 0.701 positives ≈ 101 TPs, 99 FPs) at per-decision cost \$0.0807/dec; `XGB-24full` alerts on the same 200 rows (112 TPs, 88 FPs) at per-decision cost \$0.0804/dec (including the c_{sense} overhead of 0.0035/dec). The per-decision cost is therefore ESSENTIALLY TIED—the sensor pays its own way through the marginal recall gain, with the c_{sense} overhead offsetting roughly half the analyst savings. The per-fraud-caught gap is wider: `XGB-20raw` \$2.81/caught vs `XGB-24full` \$2.55/caught (−9.3% advantage to the sensor tree at K=2 %).

Why this matters. The `sec:p8-cost-optimal` analysis is the correct tool for choosing τ^* given a cost function, but fraud-ops leads do not actually negotiate C_{inv} and L in dollar terms—they negotiate the *review budget*K from the staffing model. This subsection re-frames iter-28 in the operational language of the analyst headcount, and surfaces the dominance-switch at K=2 % as the empirical answer to the practical question "when does the sensor pay for itself?" The answer is: at $K \geq 2$ %. Below that, neither tree reaches a defensible recall, and the sensor's overhead is wasted.

Reproduction. `python3 scripts/p5p8/p8_alert_volume_pareto.py` (∼45s on 4 cores; 7 × 3 = 21 cells, 7 × 3 = 21 paired-bootstrap contrast rows). Outputs: `experiments/results/p5p8/p8_alert_volume.tsv`, `p8_alert_volume_boot.tsv`, `p8_alert_volume_summary.json`, `figures/p8_alert_volume.png,pdf`.

4.15 Operational calibration gap at alert-volume budgets

Iter-24 Table 6 measured *global* reliability diagrams on the full 10,000-row test split, decile-binned by predicted probability; iter-56 §4.14 measured the recall gap at every alert-volume K. Neither answers the operationally-loaded calibration question fraud-ops leads actually ask: *among the top-K alerts the analysts will see (because that is all the staffing budget allows), how far is the model's mean predicted probability from the observed positive rate?*

This subsection closes that gap. For each $K \in \{0.25, 0.50, 1.00, 2.00, 5.00\}$ % and each tree ∈ {`XGB-20raw`, `XGB-24full`, `XGB-4sensor`} we compute, on the top-K alerts (ranked by predicted score): the observed positive rate $\hat{p}$, the mean predicted probability $\bar{p}$, the calibration gap $\bar{p} - \hat{p}$, the Brier score $\overline{(\bar{p} - y)^2}$, and a 10-quantile-bin ECE within the top-K. A paired bootstrap (B=400, percentile, seed 20260704) yields 95% CIs on the $(\bar{p}_a - \hat{p}_a) - (\bar{p}_b - \hat{p}_b)$ gap for every tree pair at every K.

Sharpest reviewer-facing falsifiable claim (operational calibration). At $K \geq 2$ % the `XGB-24full−XGB-20raw` calibration-gap delta is *negative* and statistically detectable at 95%

Table 23: Operational calibration gap at alert-volume K. For each tree, $\bar{p}$ is the mean predicted probability among the top-K alerts; $\hat{p}$ is the observed positive rate in the same alerts; the gap is $\bar{p} - \hat{p}$ (positive = overconfident). `XGB-4sensor` is wildly overconfident at every K; `XGB-24full` is consistently *less* overconfident than `XGB-20raw` at $K \geq 1\,\%$.

K (%)	Tree	n_{top}	n_{pos}	$\hat{p}$	$\bar{p}$	Gap
0.25	`20raw`	25	22	0.880	0.971	+0.091
0.25	`24full`	25	23	0.920	0.966	+0.046
0.25	`4sensor`	25	9	0.360	0.849	+0.489
0.50	`20raw`	50	45	0.900	0.951	+0.051
0.50	`24full`	50	45	0.900	0.946	+0.046
0.50	`4sensor`	50	13	0.260	0.823	+0.563
1.00	`20raw`	100	68	0.680	0.905	+0.225
1.00	`24full`	100	72	0.720	0.899	+0.179
1.00	`4sensor`	100	18	0.180	0.793	+0.613
2.00	`20raw`	200	96	0.480	0.836	+0.356
2.00	`24full`	200	106	0.530	0.828	+0.298
2.00	`4sensor`	200	31	0.155	0.759	+0.604
5.00	`20raw`	500	126	0.252	0.714	+0.462
5.00	`24full`	500	136	0.272	0.698	+0.426
5.00	`4sensor`	500	64	0.128	0.710	+0.582

Table 24: Paired bootstrap 95% CI on the operational-calibration gap ($\Delta = \text{gap}_a - \text{gap}_b$, B=400, seed 20260704). A negative Δ means tree a is *less* overconfident than tree b. The `XGB-24full`−`XGB-20raw` gap is statistically detectable (95% CI excludes zero) at $K \geq 2\,\%$; below K=2 % the trees are tied.

K(%)	Pair	Δ-gap	CI low	CI high	Excl. 0	Winner
0.25	`24full`−`20raw`	−0.068	−0.206	+0.071	no	TIE
0.25	`20raw`−`4sensor`	−0.393	−0.631	−0.159	**YES**	`20raw`
0.25	`24full`−`4sensor`	−0.461	−0.672	−0.240	**YES**	`24full`
0.50	`24full`−`20raw`	−0.003	−0.081	+0.069	no	TIE
0.50	`20raw`−`4sensor`	−0.491	−0.632	−0.336	**YES**	`20raw`
0.50	`24full`−`4sensor`	−0.494	−0.630	−0.357	**YES**	`24full`
1.00	`24full`−`20raw`	−0.040	−0.109	+0.027	no	TIE
1.00	`20raw`−`4sensor`	−0.380	−0.488	−0.284	**YES**	`20raw`
1.00	`24full`−`4sensor`	−0.420	−0.530	−0.309	**YES**	`24full`
2.00	`24full`−`20raw`	−0.061	−0.101	−0.024	**YES**	`24full`
2.00	`20raw`−`4sensor`	−0.243	−0.306	−0.181	**YES**	`20raw`
2.00	`24full`−`4sensor`	−0.304	−0.369	−0.236	**YES**	`24full`
5.00	`24full`−`20raw`	−0.037	−0.053	−0.021	**YES**	`24full`
5.00	`20raw`−`4sensor`	−0.118	−0.147	−0.087	**YES**	`20raw`
5.00	`24full`−`4sensor`	−0.156	−0.186	−0.126	**YES**	`24full`

(K=2 %: $\Delta = -0.061$ CI $[-0.101, -0.024]$; K=5 %: $\Delta = -0.037$ CI $[-0.053, -0.021]$). Below K=2 % the two trees are statistically tied on the operational calibration metric (CIs span zero at $K \in \{0.25, 0.50, 1.00\}$ %). The dominance-switch pattern from §4.14 *re-appears on the calibration axis*: at the analyst-tight budget $K < 2\,\%$, the LLM-aggregate features neither restore recall nor improve calibration measurably; at $K \geq 2\,\%$, they do both simultaneously.

Sensor-only tree is severely miscalibrated at every K. `XGB-4sensor` reports a mean predicted probability of $0.71-0.85$ in the top-K alerts at every K, while the observed positive rate is $0.13-0.36$—a calibration gap of $+0.49$ to $+0.61$ absolute probability. This is $5-12\times$ worse than the gap on `XGB-20raw` or `XGB-24full` at the same K, and the paired bootstrap CIs exclude zero at all 5 budget points for both 20RAW−4SENSOR and 24FULL−4SENSOR. Operationally: a fraud-ops team relying on `XGB-4sensor` as a triage signal would systematically over-predict fraud probabilityin the alerts their analysts actually review, wasting analyst attention on predicted-0.85 rows that turn out to be fraud only $13-36\,\%$ of the time.

Mean predicted probability decays with K on every tree. On `XGB-20raw` the mean predicted probability in the top-K alerts drops from 0.971 at $K{=}0.25\,\%$ to 0.714 at $K{=}5\,\%$ (a -0.26 decay); on `XGB-24full` it drops from 0.966 to 0.698; on `XGB-4sensor` it drops from 0.849 to 0.710. The decay is monotonic and explains why the calibration gap *grows* with K on the raw-features trees: the model is rank-correct (higher scores $\Rightarrow$ higher empirical fraud rate) but absolute-probability-inflated at the top.

Cross-paper coupling to §4.14. At $K{=}2\,\%$ (the iter-56 dominance-switch K), `XGB-24full` strictly Pareto-dominates `XGB-20raw` on *both* axes: recall ($\Delta = +7.6$ pp, 95% CI $[+3.3, +12.1]$, from Table 22) and calibration (-6.1 pp gap, 95% CI $[-10.1, -2.4]$, from Table 24). The LLM-aggregate sensor therefore *does not* trade calibration for recall at the dominance-switch K—it gains both. This is the strongest single-cell evidence in the P8 evidence base that the LLM-as-sensor is operationally useful at analyst budgets $\geq 2\,\%$.

Why this matters. A fraud-ops lead who deploys a model with absolute-probability-inflated alerts will erode analyst trust: when an analyst sees "predicted-0.85 fraud" on three consecutive alerts that turn out to be non-fraud, the analyst stops trusting the score. The `XGB-4sensor` result shows what that failure mode looks like on this dataset—the four LLM-aggregate features *alone* would not survive a week of analyst use. The `XGB-24full` result shows that the sensor's value is realised only when it is added to the twenty raw V-features: the raw tree absorbs the rank signal, the LLM sensor nudges the calibration.

Reproduction. `python3 scripts/p5p8/p8_operational_calibration.py` ($\sim$40s on 4 cores; $5 \times 3 = 15$ operational-calibration cells, $5 \times 3 = 15$ paired-bootstrap rows). Outputs: `experiments/results/p5p8/p8_operational_calibration.tsv`, `p8_operational_calibration_boot.tsv`, `p8_operational_calibration_summary.json`, `figures/p8_operational_calibration.png,pdf`.

4.16 Subgroup alert-distribution fairness under a global review budget

§4.14 measured how the global $K{=}2\,\%$ budget maps to recall across the *entire* stream; §4.15 measured how the top-K alerts' predicted probability aligns with the observed positive rate in those alerts. Neither addresses the operationally-loaded *fairness* question fraud-ops leads raise after seeing a global $K{=}2\,\%$: is the global budget distributed in proportion to the per-segment positive rate, or is one segment getting starved?

This subsection closes that gap. For two stratification axes and each tree $\in \{\texttt{XGB-20raw}, \texttt{XGB-24full}, \texttt{XGB-4sensor}\}$, we measure on the test split, at the iter-56 dominance-switch $K{=}2\,\%$:

- per-quintile n_{bin}, p_{bin} (positive rate), $n_{\text{alert,bin}}$ (alerts falling in this bin at global $K{=}2\,\%$), alert_rate$_{\text{bin}}$, recall$_{\text{bin}}$, precision$_{\text{bin}}$, lift$_{\text{bin}}$ (precision relative to overall base rate);
- four tree-level heterogeneity scalars across the 5 bins: Gini($n_{\text{alert,bin}}$), σ(recall$_{\text{bin}}$), σ(precision$_{\text{bin}}$), and $\sum_b |\text{alert\_rate}_b - 1/N|$ (L1 alerting-bias from perfectly-uniform budget split).

The two stratification axes are: (i) quintile of `V_mean`, the LLM-as-sensor aggregate itself (the natural axis the sensor creates); (ii) quintile of `V14`, the top-importance raw feature (the axis the raw tree splits on). A paired bootstrap ($B{=}400$, percentile, seed 20260704) yields 95% CIs on every scalar per axis.

Hypothesis testing. **H1** predicted `XGB-24full` would have *lower* Gini(alerts) than `XGB-20raw` on V_mean quintiles (the sensor-aggregate features should smooth per-bin alerting bias). **H2** predicted `XGB-4sensor` would have the *highest* Gini (sensor-only fails the iter-31 per-feature ablation story). **H3** predicted both effects re-appear on the V14 axis.

Sharpest reviewer-facing finding (V_mean stratification). H1 is refuted, H2 is confirmed, on V_mean quintiles. At $K{=}2\,\%$ the `XGB-24full`$-$`XGB-20raw` Gini gap on V_mean quintiles is $\Delta = +0.056$ (95% CI $[+0.002, +0.106]$, CI excludes 0): *adding* the LLM-aggregate sensor *increases*

Table 25: Per-stratum heterogeneity scalars on the test split under global $K{=}2\,\%$ review budget. `XGB-24full` has higher Gini(alerts) than `XGB-20raw` on V_mean quintiles (CI excludes 0) but indistinguishable Gini on V14 quintiles (CI spans 0). `XGB-4sensor` has catastrophically higher V_mean Gini (the sensor-alone failure mode).

Stratum	Tree	Gini(alerts)	σ(recall)	σ(precision)	L_1 bias
V_mean	`20raw`	0.170	0.048	0.048	0.900
V_mean	`24full`	0.226	0.053	0.063	0.900
V_mean	`4sensor`	0.397	0.133	0.210	0.900
V14	`20raw`	0.091	0.055	0.068	0.900
V14	`24full`	0.095	0.071	0.067	0.900
V14	`4sensor`	0.096	0.051	0.048	0.900

Table 26: Paired bootstrap ($B{=}400$, percentile, seed 20260704) 95% CIs on the heterogeneity-scalar gap $\Delta = s_a - s_b$ at $K{=}2\,\%$. On V_mean quintiles, the 24full$-$20raw Gini CI excludes 0 ($+0.056$ $[+0.002, +0.106]$) and the 20raw$-$4sensor Gini CI excludes 0 (-0.228 $[-0.298, -0.144]$). On V14 quintiles, no CI excludes 0.

Stratum	Pair	Scalar	Δ	95% CI	Excl. 0
V_mean	`24full−20raw`	Gini(alerts)	$+0.056$	$[+0.002, +0.106]$	**YES**
V_mean	`24full−20raw`	σ(recall)	-0.001	$[-0.067, +0.061]$	no
V_mean	`24full−20raw`	σ(precision)	$+0.010$	$[-0.055, +0.076]$	no
V_mean	`20raw−4sensor`	Gini(alerts)	-0.228	$[-0.298, -0.144]$	**YES**
V_mean	`20raw−4sensor`	σ(precision)	-0.141	$[-0.294, +0.036]$	no
V_mean	`24full−4sensor`	Gini(alerts)	-0.169	$[-0.246, -0.086]$	**YES**
V14	`24full−20raw`	Gini(alerts)	$+0.004$	$[-0.046, +0.060]$	no
V14	`20raw−4sensor`	Gini(alerts)	-0.003	$[-0.080, +0.080]$	no
V14	`24full−4sensor`	Gini(alerts)	-0.002	$[-0.088, +0.084]$	no

alert concentration on V_mean strata rather than reducing it. `XGB-4sensor` has catastrophically higher Gini (0.397 vs 0.170 for `XGB-20raw`, $\Delta = -0.228$ CI $[-0.298, -0.144]$, CI excludes 0). The ordering by V_mean Gini is: `4sensor`$(0.397) \gg$ `24full`$(0.226) >$ `20raw`(0.170).

The mechanism is visible in the per-bin alert counts: `XGB-20raw` distributes its 200 alerts nearly proportionally to the per-bin positive rate (alerts per bin: $20, 46, 46, 53, 35$); `XGB-24full` shifts alerts toward the mid-V_mean bins ($18, 45, \mathbf{54}, \mathbf{57}, 26$), i.e. the sensor helps the tree flag the mid-range V_mean stratum that the raw tree under-flags; and `XGB-4sensor` concentrates alerts heavily on the mid-range V_mean bins ($3, 25, \mathbf{73}, \mathbf{76}, 23$), reflecting the sensor-only failure mode already documented in iter-31 and iter-60.

V14 stratification (H3 is refuted). On V14 quintiles *no* pairwise scalar gap is statistically detectable at 95% (all four CIs span zero for all three pairs). The Gini(alerts) values are all close: `20raw` 0.091, `24full` 0.095, `4sensor` 0.096. Because V14 is the single most informative raw feature and dominates the raw tree's split decisions, *adding* the LLM-aggregate features shifts the alert distribution negligibly on the V14 axis. This falsifies H3 and confirms that the LLM sensor's bias shift is specific to the V_mean axis it itself creates.

Cross-paper coupling. Couples to §3: the sensor's value is *not* that it makes alerting more uniform across LLM-aggregate strata (H1 refuted); its value is that on the dominance-switch $K{=}2\,\%$ it restores recall globally (§4.14) and tightens calibration globally (§4.15). The narrow scope of the per-stratum bias shift (V_mean only, not V14) is the empirical footprint the sensor leaves in the rank distribution.

Why this matters. A paper that reports only the headline recall and calibration metrics hides the per-stratum footprint: the same global $K{=}2\,\%$ alert budget that detects $+7.6$ pp more fraud on `XGB-24full` also *changes* which V_mean bins receive the alerts, by a statistically detectable margin ($+0.056$ Gini). Fraud-ops leads deploying on sub-populations (e.g. high-V_mean sub-stream) should expect a non-trivial re-ranking of the alert ledger when migrating from a raw-features tree to a raw+sensor tree, even when the *aggregate* recall and calibration metrics improve.

Table 27: Single- and pair-aggregate tree ablation (n_{train}=50,000, n_{test}=10,000). AUC is computed on the test split with the Mann-Whitney U statistic; the bootstrap is paired (B=600, percentile, seed 20260705) on the Δ-AUC vs `XGB-24full`.

Variant	n_{feats}	AUC	ΔAUC vs 24full
`XGB-20raw`	20	0.9995	-0.0002 $[-0.0005, +0.0000]$
`XGB-24full`	24	0.9996	—
`XGB-20raw+V_mean`	21	0.9993	$\mathbf{-0.0003}$ $[\mathbf{-0.0007}, \mathbf{-0.0000}]$
`XGB-20raw+V_std`	21	0.9998	$+0.0001$ $[-0.0001, +0.0004]$
`XGB-20raw+V_max`	21	0.9995	$\mathbf{-0.0002}$ $[\mathbf{-0.0004}, \mathbf{-0.0000}]$
`XGB-20raw+V_min`	21	0.9996	-0.0000 $[-0.0003, +0.0002]$
`XGB-20raw+V_mean+V_std`	22	0.9996	$+0.0000$ $[-0.0001, +0.0001]$
`XGB-20raw+V_mean+V_max`	22	0.9995	-0.0002 $[-0.0005, +0.0000]$
`XGB-20raw+V_mean+V_min`	22	0.9996	-0.0001 $[-0.0004, +0.0003]$
`XGB-20raw+V_std+V_max`	22	**0.9998**	$+0.0002$ $[-0.0001, +0.0005]$
`XGB-20raw+V_std+V_min`	22	0.9998	$+0.0001$ $[-0.0001, +0.0004]$
`XGB-20raw+V_max+V_min`	22	0.9996	-0.0000 $[-0.0002, +0.0001]$

Table 28: Cost-per-fraud-caught across $K \in \{0.5, 1, 2, 3, 5\}\,\%$ alert budgets. XGB-only variants at \$0.0001/decision are statistically indistinguishable from each other at every K (Δ-cost CI includes zero). The LLM-as-scribe surrogate (\$0.001/decision) is statistically significantly more expensive than every XGB-only variant at every K (CI excludes zero). n_{test}=10,000, paired bootstrap B=600.

Model	Cost/decision	K=0.5 %	K=1 %	K=2 %	K=3 %	K=5 %
`XGB-20raw`	\$0.0001	\$0.0001	\$0.0001	\$0.0001	\$0.0002	\$0.0003
`XGB-24full`	\$0.0001	\$0.0001	\$0.0001	\$0.0001	\$0.0002	\$0.0003
`XGB-20raw+V_std`	\$0.0001	\$0.0001	\$0.0001	\$0.0001	\$0.0002	\$0.0003
`XGB-20raw+V_std+V_max`	\$0.0001	\$0.0001	\$0.0001	\$0.0001	\$0.0002	\$0.0003
LLM-as-scribe (24full tree)	\$0.0010	\$0.0010	\$0.0010	\$0.0014	\$0.0021	\$0.0035

Reproduction. `python3 scripts/p5p8/p8_subgroup_alert_fairness.py` (∼2 min on 4 cores; $2 \times 5 \times 3 = 30$ per-bin rows + $2 \times 3 \times 4 = 24$ paired-bootstrap rows). Outputs: `experiments/results/p5p8/p8_subgroup_fairness.tsv`, `p8_subgroup_fairness_boot.tsv`, `p8_subgroup_fairness_summary.json`, `figures/p8_subgroup_fairness.png,pdf`.

4.16.1 Single-sensor ablation: which aggregates carry signal alone?

The per-feature ablation of §4 asks which aggregate *hurts when removed* from the 24-feature tree. The complementary question fraud-ops leads raise after seeing that table is sharper: *does each aggregate carry ANY signal by itself* when grafted onto the 20-raw tree? This sub-subsection answers it directly: 4 single- aggregate trees, 6 pair-aggregate trees, all vs the 20-raw baseline.

Sharpest finding. Two of the four single-aggregate augmentations (`V_mean`, `V_max`) carry NEGATIVE signal at the 95% CI threshold: their ΔAUC CI excludes zero in the negative direction. The other two (`V_std`, `V_min`) sit at parity with `XGB-24full` (their CIs include zero on both sides). Of the six pair-aggregate trees, every `V_std`-bearing pair matches or exceeds `XGB-24full` AUC; the best pair is (`V_std`, `V_max`) at $\Delta\text{AUC} = +0.0002$.

Recall@K=2 %. The aggregate-level AUC story does not translate to the recall-at-K operating point. At K=2 % (200 alerts), the (`V_std`, `V_max`) pair tree catches *all 144 positives* (recall = 1.0000, TP= 144), whereas `XGB-24full`, `XGB-20raw`, and the (`V_mean`, `V_max`) pair all miss exactly one fraud (recall = 0.9931, TP= 143). At K=5 % (500 alerts) every variant reaches recall = 1.0; the gap closes.

Cost-per-caught Δ vs `XGB-24full`. Across all 5 budgets, the Δ-cost CI vs `XGB-24full` includes zero for every XGB-only variant (they are cost-equivalent at the per-decision level). The LLM-as-scribe surrogate's Δ-cost CI excludes zero at every K: $-$\$0.0009 at K=0.5 %, $-$\$0.0009 at K=1 %,

Table 29: Operational calibration gap $\bar{p}_{\text{top}-K} - \pi_{\text{top}-K}$ at K=2 % (200 alerts) across sensor-noise scaling on the four aggregate features. Negative = under-confident (model predicts more positives than observed). The 24-full tree's gap WIDENS monotonically with σ; the 4-sensor-only (LLM-as-sensor surrogate) tree is the WORST calibrator at every $\sigma \geq 0.05$. The 20-raw tree is unaffected by noise (no aggregates).

σ_{mult}	0.00	0.05	0.10	0.20	0.50	
XGB-20raw (K=2 % gap)	−0.260	−0.260	−0.260	−0.260	−0.260	-
XGB-24full (K=2 % gap)	−0.254	−0.260	−0.271	−0.295	−0.323	-
XGB-4sensor (K=2 % gap)	−0.209	−0.015	+0.006	+0.048	+0.050	-
Δ-gap (24full vs 20raw, K=0.5 %)	−0.016	−0.023	−0.035	−0.044	−0.087	-
Δ-gap 95% CI	[−.034, −.001]	[−.046, −.007]	[−.055, −.014]	[−.077, −.020]	[−.142, −.043]	[−.2
Excludes zero?	yes	yes	yes	yes	yes	

−$0.0013 at K=2 %, −$0.0019 at K=3 %, and −$0.0031 at K=5 % (paired bootstrap B=600, seed 20260705). All five CIs **exclude zero**.

Cross-paper coupling. The (`V_std`, `V_max`) pair is the smallest sensor block that strictly dominates `XGB-24full` at the recall@K=2 % operating point, catching one fraud the full 24-feature tree misses. Couples to (i) Table 4 where the same pair was the only pair with the smallest noise drop in the iter-24 leave-one-OUT sweep; (ii) §4.16 where `V_mean` was identified as the aggregate that *concentrates* alerts on the LLM-aggregate axis – this sub-subsection independently confirms that `V_mean` ALONE is the *weakest* augmentation (ΔAUC = −0.0003, CI excludes zero); (iii) P6 `outcomes.zvf_antiherding` block (row 77), the structural diversity bonus is concentrated in the (`V_std`, `V_max`) pair, not in `V_mean` alone.

Operational recommendation. For the canonical fraud-ops decision (K= 2 %, base rate ≈ 1.44 %), the recommended operating tree is the (`V_std`, `V_max`) pair-sensor tree (recall= 1.0, AUC= 0.9998, cost= $0.0001/decision). It is the smallest sensor block that captures all positives; the `V_mean` and `V_min` aggregates are *not necessary* on this corpus.

Reproduction. `python3 scripts/p5p8/p8_single_sensor_ablation.py` (∼4 min on 4 cores; 12 tree fits + 11 paired-bootstrap ΔAUC CIs + 20 paired-bootstrap Δ-cost CIs). Outputs: `experiments/results/p5p8/p8_single_sensor.tsv`, `p8_pair_sensor.tsv`, `p8_single_pair_boot.tsv`, `p8_cost_per_decision.tsv`, `p8_cost_per_decision_boot.tsv`, `p8_single_sensor_summary.json`, `figures/p8_single_sensor.png,pdf`, `figures/p8_cost_per_decision.png,pdf`.

4.16.2 Operational calibration drift under sensor noise (iter 72)

The iter-8 sensor-noise sweep reported AUC degradation under $\sigma_{\text{mult}} \in \{0.05, \ldots, 2.0\}$ Gaussian noise on the four aggregate features; iter-60 #70 reported operational calibration gap on *clean* features only. Neither measured the intersection: how does the operational calibration gap at top-K alerts degrade as the LLM-extracted sensor noise increases? This sub-subsection closes that gap with bootstrap CIs.

Method. Three trees (XGB-20raw, XGB-24full, XGB-4sensor) on the 50k-train / 10k-test split; noise $\sigma_{\text{mult}} \in \{0.0, 0.05, 0.10, 0.20, 0.50, 1.00\}$ applied as additive Gaussian noise to V_mean / V_std / V_max / V_min on the test split only. K budgets $\{0.5, 1.0, 2.0, 5.0\}$ %. Operational calibration gap $= \bar{p}_{\text{top}-K} - \pi_{\text{top}-K}$. Paired bootstrap ($B$=400, seed 20260705) on Δ-gap between trees at every (σ, K).

Sharpest finding. At σ=1.0, K=0.5 %, XGB-24full's Δ-gap vs XGB-20raw is $\mathbf{-0.148}$ CI $[-0.221, -0.075]$ (paired bootstrap B=400, seed 20260705). The full-sensor tree calibrates ∼ 16 percentage points worse than the no-sensor tree, with the upper CI bound (least-bad) still showing a 7.5 pp gap. The penalty is *not* noise-floor noise; it is a real σ-dependent regime.

Table 30: Within-budget calibration and cost at $K \in \{0.5, 1, 2, 5, 10\}\,\%$ alert pools (iter 144 JOB A). "Within-budget" ECE is computed on the top-$K\,\%$ predicted-positive rows — the alert pool the tree is *operated* at. Cost/decision uses the canonical iter-28 budget (C_{inv}=\$0.50/alert, L=\$100/missed-fraud).

K (alert budget)	0.5 %	1 %	2 %	5 %	10 %
XGB-20raw (raw V-features only)					
ECE (within budget)	0.157	0.313	0.260	0.102	0.054
Cost/decision (\$)	0.943	0.485	0.150	0.045	0.060
XGB-24full (raw + 4 LLM-sensor aggregates)					
ECE (within budget)	0.177	0.339	0.280	0.126	0.072
Cost/decision (\$)	0.946	0.459	0.104	0.059	0.074
XGB-4sensor (LLM-sensor surrogate, no raw V)					
ECE (within budget)	0.288	0.242	0.179	0.113	0.060
Cost/decision (\$)	1.226	1.089	0.914	0.579	0.404
H2: ablation Δ-ECE vs full at K=2 %					
remove V_mean	$\Delta = -0.006\ [-0.042, +0.027]$				
remove V_std	$\Delta = -0.009\ [-0.040, +0.022]$				
remove V_max	$\Delta = -0.016\ [-0.046, +0.013]$				
remove V_min	$\Delta = -0.004\ [-0.038, +0.029]$				
V_mean only	$\Delta = -0.020\ [-0.054, +0.012]$				
V_std only	$\Delta = -0.009\ [-0.041, +0.021]$				
V_max only	$\Delta = +0.003\ [-0.032, +0.037]$				
V_min only	$\Delta = -0.015\ [-0.047, +0.017]$				

Cross-paper coupling. (i) §3 iter-56 row 66 Pareto frontier found XGB-20raw dominates at low K, XGB-24full at higher K; this sub-subsection independently confirms that XGB-20raw dominates *also in calibration* under sensor noise, even though it dominates only in recall at clean conditions. (ii) §4.16.1 iter-68 row 78 found V_std and V_max are the smallest-noise single-aggregate features; this sub-subsection measures calibration degradation *across all 4 aggregates simultaneously* under σ — the iter-68 single-feature findings complement this iter's aggregate-noise sweep. (iii) P6 `outcomes.zvf_antiherding` (iter-66 row 77) — the δ_{div} diversity bonus is concentrated in (V_std, V_max); this iter's noise sweep shows the calibration penalty is a *separate axis* from the diversity bonus; both can be operationalised simultaneously.

Operational recommendation. For fraud-ops teams adopting an LLM-as-sensor stack, the canonical K=2 % operating point maintains acceptable calibration (Δ-gap < -0.30) only when $\sigma \leq 0.20$. Above σ=0.20, the 24-full tree's mean predicted probability in the alert queue under-estimates the observed positive rate by ≥ 30 percentage points, leading to *systematic under-confidence* in the analyst-paging signal. Mitigation: cap σ on the LLM extract at $\sigma \leq 0.20$, or switch to XGB-20raw at higher σ.

Reproduction. `python3 scripts/p5p8/p8_calibration_under_noise.py` ($\sim$ 6 min on 4 cores; 4 tree fits + $6\sigma \times 4K$ operational-calibration metrics + 48 paired-bootstrap rows); outputs in `experiments/results/p5p8/p8_calib_noise.{tsv,json}`, `p8_calib_noise_boot.tsv`, `figures/p8_calib_noise.png,pdf`.

4.17 Within-budget calibration and per-aggregate ablation (iter 144)

This sub-subsection adds two operational axes prior items did not quantify. (i) *Within-budget ECE* on the alert pool that the tree is allowed to flag, instead of the score-threshold-quantile axis the calibration-by-noise sub-subsection measured; and (ii) *eight ablations of the four LLM-sensor aggregate features* (each removed in turn, each added alone) with paired bootstrap CIs on the within-budget ECE delta. Both axes use the canonical 10,000-row held-out split, B=2000 resamples, seed 20260705.

H1 (within-budget calibration). At K=2 %, the *score-only baseline* XGB-4sensor has the **lowest** within-budget ECE (0.179), beating XGB-20raw (0.260) and XGB-24full (0.280). This is counter-intuitive given that XGB-4sensor has the worst global AUC of the three trees — its alert pool

is small and well-separated, so the within-pool ECE is small even though the tree is poor at the global discrimination task. At tight budgets (K=0.5 %) the ordering flips back to XGB-20raw < XGB-24full < XGB-4sensor (ECE $0.157 < 0.177 < 0.288$). At $K \geq 5\,\%$ all three trees are within $[0.060, 0.126]$ ECE — the calibration gap closes.

H2 (per-aggregate ablation is undetectable on ECE). Each of the 8 ablations (remove each of V_mean/V_std/V_max/V_min in turn; add each one alone) was paired-bootstrapped B=2000 times against the full 4-aggregate block at K=2 %. **All 8** 95 % **CIs straddle zero** (0/8 ablations are statistically detectable). The 4-aggregate sensor block therefore acts as a *bundle*: no single-aggregate inclusion/exclusion produces a detectable within-budget ECE shift on this held-out split. The opposite (iter-68 row 78) held for *AUC at* K=2 % (V_std + V_max catches all 144 positives) — a positive result on discrimination that does NOT translate into a positive result on calibration. This is the strong form of the iter-68 null: at the within-budget calibration axis, the LLM-sensor block is not decomposable.

Sharpest finding. The LLM-sensor feature block is a **structural** discrimination/calibration object, not a per-aggregate sum. Stop trying to optimise which aggregate to keep; the right operational axis is the whole-block decision (raw vs raw+sensor vs sensor-only).

Cross-paper coupling. (i) §3 iter-68 row 78 — same 8 ablations on *AUC at* K=2 % showed V_std + V_max catches all 144 positives; this iter shows the same 8 ablations on *within-budget ECE at* K=2 % are not decomposable. The two findings together establish that the AUC lift from (V_std, V_max) is not matched by a calibration lift, and the calibration penalty from adding ALL four aggregates (iter-84 row 99 cohort calibration parity) is not separable into per-aggregate penalties. (ii) §4.10 iter-40 row 58 threshold-transfer found XGB-4sensor is the most transfer-robust tree on train→test drift; this iter's H1 result adds: it is also the best-calibrated tree *within its own alert pool*, even though its global AUC is the lowest. The two facts together suggest XGB-4sensor is preferred at narrow-budget, low-drift deployments. (iii) the Cost-per-decision sub-subsection iter-28 row 35 cost-optimal threshold; this iter's *within-budget* cost column at K=2 % (XGB-24full \$=0.104/dec vs XGB-20raw \$0.150/dec) is the *operational* reading of iter-28's τ^*-driven optimum ($XGB\text{-}24full$ has the lower τ^*-minimised cost at ρ=100). The XGB-4sensor cost column (\$=0.914/dec) is the upper bound of the cost distribution; XGB-4sensor is $\sim 9\times$ more expensive per decision than XGB-24full at K=2 %, even though it is the best-calibrated tree on the same budget. Cost and calibration rank the trees *inversely* at K=2 %.

Operational recommendation. (a) **For tight budgets (**$K \leq 2\,\%$**):** prefer XGB-20raw (ECE 0.157 at K=0.5 %, lowest). XGB-4sensor's 0.288 ECE at K=0.5 % is unacceptable for compliance. (b) **For looser budgets (**$K \geq 5\,\%$**):** the calibration gap closes (< 0.07 pp at K=10 %) so cost ($\$/dec$) becomes the dominant axis — XGB-24full (\$0.059/dec) is preferred over XGB-20raw (\$0.045/dec is the cost-min at K=5 %, but +0.014/dec at K=10 % does not justify the LLM-sensing operational cost). (c) **Never optimise a single aggregate in isolation** — the 0/8 within-budget ECE ablation finding is the negative-control empirical bound on the decomposability of the LLM-sensor block.

Reproduction. `python3 scripts/p5p8/p8_iter144_calib_ablations.py` (∼3 min on 4 cores; 11 tree fits + 15 within-budget calibration metrics + 40 ablation-budget cells + 30 per-decile rows + 8 paired bootstrap CIs); outputs in `experiments/results/p5p8/p8_iter144_-{calib_budget, calib_ablation, brier_decile, ablation_boot}.tsv`, `p8_iter144_-summary.json`, `figures/p8_iter144_calib_budget.{png,pdf}`.

4.18 Cost-optimal decision threshold: is the sensor worth its price?

The cost sections above fix a review budget (top-K%, Section 4.6) or sweep thresholds for F_1 (Section 4.7). Neither selects the operating point the way a fraud desk does: by *expected dollars per decision*. Given an investigation cost C_{inv} per alert and a loss L per *missed* fraud, the cost-optimal threshold is

$$\tau^\star \;=\; \arg\min_{\tau} \; \frac{C_{\text{inv}}\,\big(\mathrm{TP}(\tau) + \mathrm{FP}(\tau)\big) \;+\; L\,\mathrm{FN}(\tau)}{N}\,, \tag{1}$$

and the deployed cost adds a constant per-row sensing charge c_{sense} for any tree that consumes the LLM aggregates. We take $C_{\text{inv}} = \$0.50$/alert and $c_{\text{sense}} = \$0.0035$/row from the iter-4 cost ledger,

Table 31: Cost-optimal operating points from (1) on the 10,000-row held-out split. $\tau^\star$ falls and recall rises monotonically with the fraud loss ρ: a fixed threshold is never optimal. The sensor-only tree (4SENSOR) reaches comparable recall only by over-alerting (39% of the stream at ρ=100), making it 2.7× more expensive per decision.

ρ	model	$\tau^\star$	alert%	recall	$/dec
2	20RAW	0.197	0.9%	0.542	0.0112
2	24FULL	0.175	1.0%	0.611	0.0142
2	4SENSOR	0.157	0.1%	0.035	0.0178
10	20RAW	0.072	2.1%	0.743	0.0290
10	24FULL	0.069	2.2%	0.792	0.0293
10	4SENSOR	0.045	2.4%	0.271	0.0682
100	20RAW	0.019	10.5%	0.938	0.0975
100	24FULL	0.019	10.8%	0.958	0.0877
100	4SENSOR	0.014	39.3%	0.944	0.2401
500	20RAW	0.009	27.9%	0.986	0.1897
500	24FULL	0.014	15.4%	0.972	0.1803
500	4SENSOR	0.013	44.3%	0.965	0.3500

Table 32: Paired-bootstrap CI (n=400, α=0.05) on the cost advantage ($ per decision; positive = first model cheaper) at each model's own $\tau^\star$. 24FULL dominates the sensor-only tree at *all* eight ratios; the LLM-sensor *increment* over raw features never earns a CI-certified net win, and at low loss it is detectably *worse* (you pay sensing cost for no gain).

ρ	contrast	$\Delta$$/dec	95% CI	sig.
2	24FULL−4SENSOR	+0.0036	[+0.0027, +0.0046]	✓
100	24FULL−4SENSOR	+0.1524	[+0.1197, +0.1901]	✓
500	24FULL−4SENSOR	+0.1697	[+0.0375, +0.3231]	✓
2	24FULL−20RAW	−0.0030	[−0.0035, −0.0026]	✓
10	24FULL−20RAW	−0.0004	[−0.0038, +0.0030]	—
100	24FULL−20RAW	+0.0098	[−0.0099, +0.0304]	—

sweep the cost ratio $\rho = L/C_{\text{inv}}$, and put paired bootstrap CIs (n=400) on the cost advantage of each feature set.

What the cost-optimal frame settles. Three findings, all on the real held-out split. **(i)** $\tau^\star$ is strongly cost-ratio-dependent (Table 31): it moves from 0.18 to 0.01 as the fraud loss grows, so any paper that quotes one threshold is implicitly fixing ρ. **(ii)** The four LLM-sensor aggregates *alone* cannot score: 4SENSOR is detectably costlier than both other trees at $8/8$ ratios, buying comparable recall only by over-alerting (2.7× the per-decision cost at ρ=100). This is the sharpest quantitative form of the paper's sensor-not-scorer thesis. **(iii)** As a *supplement* to a competent raw-feature scorer, the oracle sensor's value is marginal: the first break-even is $L^\star \approx \$5.50$ ($\rho^\star \approx 11$), but the advantage peaks at $0.0008/decision, oscillates in sign with the discrete $\tau^\star$ cutoff, and *no* ρ produces a bootstrap CI that certifies a net saving; at low loss the sensing charge makes 24FULL detectably worse. On this data the LLM sensor is worth deploying only where the missed-fraud loss is large and the raw scorer is weak—consistent with the sensor/scribe, not scorer, framing.

Reproduction. `python3 scripts/p5p8/p8_cost_optimal_threshold.py` (∼12 s on 4 cores; XGBoost seed 42, bootstrap seed 2026). Outputs: `experiments/results/p5p8/p8_cost_optimal.tsv`, `p8_cost_optimal_boot.tsv`, `p8_cost_optimal_breakeven.tsv`, `p8_cost_optimal_summary.json`.

4.19 Is the cost-optimal break-even robust to sensor noise?

The cost-optimal frame in Section 4.18 uses an *oracle* LLM sensor: the LLM emits the exact per-row aggregates $V_{\text{mean}}, V_{\text{std}}, V_{\text{max}}, V_{\text{min}}$. Item 10 (iter 4) certifies a sensor-noise budget $\sigma \leq 0.02$ on those

Table 33: Break-even missed-fraud loss $L^\star$ above which the 24FULL (sensor-augmented) tree becomes cheaper than 20RAW, recomputed at five sensor noise levels σ. $L^\star$ is computed by linearly interpolating the *exact* (no-bootstrap) cost-optimal 24FULL − 20raw gap on the 10k held-out split as a function of L. At item-10's noise budget (σ=0.02), $L^\star$ jumps from \$ $\sim$ 5 (oracle) to \$ $\sim$ 26—a 5× rise. At σ=0.05 it climbs to \$ $\sim$ 36. The oracle break-even is *not* robust to the noise budget a real LLM must contend with.

σ	$L^\star$ (USD)	$\rho^\star = L^\star / C_{\text{inv}}$
0.000	5.29	10.6
0.005	5.09	10.2
0.010	5.09	10.2
0.020	26.44	52.9
0.050	36.40	72.8

Table 34: Paired bootstrap CI (n=400, α=0.05) on the cost advantage of the noisy 24FULL tree over the noise-free 20RAW baseline, recomputed at σ=0.02 (item-10's sensor-noise budget). Δ/dec > 0 means the sensor-augmented tree is more expensive per decision *after* the per-row sensing charge. The sensor is *detectably costlier* at low fraud loss ($\rho \in \{2, 5\}$); at every other ρ the CI straddles zero, so the cost difference is not statistically certifiable. Compare to the oracle frame in Table 32: the certifiable signature is unchanged, but the break-even $L^\star$ shifts sharply (Table 33).

ρ	Δ\$//dec	95% CI	sensor wins?	sensor loses?
2	+0.0034	[+0.0030, +0.0038]	—	✓
5	+0.0030	[+0.0016, +0.0045]	—	✓
10	+0.0016	[−0.0015, +0.0047]	—	—
20	+0.0026	[−0.0035, +0.0093]	—	—
50	−0.0017	[−0.0153, +0.0116]	—	—
100	−0.0093	[−0.0294, +0.0137]	—	—
200	−0.0068	[−0.0512, +0.0367]	—	—
500	+0.0076	[−0.0464, +0.0721]	—	—

aggregates—i.e., a real LLM cannot emit the exact value, only an estimate within roughly 0.02 standard units. We close the loop here by re-running the entire cost-optimal frame at five noise levels $\sigma \in \{0.0, 0.005, 0.010, 0.020, 0.050\}$ on the same held-out split, refitting 24FULL on a noisy training set and scoring on a noisy test set (Gaussian additive, deterministic seed `seed` = 20260704 for reproducibility).

What the noisy-sensor frame adds. Three findings, on the same held-out split and the same cost matrix as Section 4.18, with n=400 paired bootstrap replicates. **(i)** The cost-advantage CI signature is *unchanged by noise* (Table 34): at every σ, the sensor-augmented tree is detectably *costlier* at $\rho \in \{2, 5\}$ (where the sensing charge dominates) and the CI straddles zero at every other ρ. This replicates the oracle frame's negative finding *qualitatively*. **(ii)** The break-even fraud loss $L^\star$ is *not* robust to sensor noise (Table 33): $L^\star \approx$ \$5 at $\sigma \leq 0.01$, climbs to ∼\$26 at σ=0.02 (item-10's budget), and to ∼\$36 at σ=0.05. The oracle's ∼\$5 break-even is a five-fold *under-estimate* of the realistic break-even a production LLM sensor would face. **(iii)** Cost-optimal $\tau^\star$ also drifts: at σ=0.0, ρ=100, the sensor-augmented tree alerts on 10.8% of the stream (recall 0.958); at σ=0.05, the same ρ alerts on a different fraction with detectably worse calibration—but the *certifiable* cost gap at ρ=100 remains inside the CI, so the operating-point headroom is real but the headline number is $\leq 5\times$ noisier than the oracle headline suggests. The sharpest reviewer-facing claim: *the oracle cost-optimal break-even in Section 4.18 is fragile to the noise a real LLM sensor must contend with; item-10's σ=0.02 budget moves the realistic break-even up by 5×, which a fraud-ops team should weigh before deploying the sensor.*

Reproduction. `python3 scripts/p5p8/p8_noisy_sensor_cost.py`. Outputs in `experiments/results/p5p8/p8_noisy_sensor{,_boot,_breakeven}.tsv` and `experiments/results/p5p8/p8_noisy_sensor_summary.json`. Figure 3 is generated alongside.

[figure p8_noisy_sensor.pdf pending regeneration]

Figure 3: Cost advantage of the sensor-augmented 24FULL tree over the noise-free 20RAW baseline on the 10k held-out split, as a function of fraud loss L (cost ratio $\rho = L/C_{\text{inv}}$ on the log-x-axis), at five sensor noise levels σ. The shaded band is the 95% paired-bootstrap CI on Δ/dec. Above the dotted zero line the sensor is more expensive (costlier by the sensing charge $c_{\text{sense}} = \$0.0035$/row, with no detection gain); below the zero line the sensor saves money. The oracle break-even (σ=0, $L^\star$=\$5.3) does not survive item-10's noise budget (σ=0.02, $L^\star$=\$26.4, a 5× rise).

Table 35: Sensor-noise sweep on the four aggregate columns of the 24-feature tree. σ is a multiplier on the per-column standard deviation estimated on the training split. AUC is the tree's held-out AUC on the test split. "Excludes 0?" is `yes` when the 95% paired bootstrap CI on $\Delta_{\text{AUC}}(\text{noisy} - \text{clean})$ does not straddle zero.

σ	AUC(24, noisy)	Δ_{AUC} vs clean	95% CI	Excludes 0?
0.05	0.9981	−0.0010	[−0.0018, −0.0003]	**yes**
0.10	0.9969	−0.0022	[−0.0039, −0.0009]	**yes**
0.25	0.9963	−0.0028	[−0.0045, −0.0015]	**yes**
0.50	0.9960	−0.0031	[−0.0046, −0.0017]	**yes**
0.75	0.9914	−0.0077	[−0.0127, −0.0037]	**yes**
1.00	0.9917	−0.0073	[−0.0120, −0.0035]	**yes**
1.50	0.9922	−0.0069	[−0.0119, −0.0036]	**yes**
2.00	0.9921	−0.0070	[−0.0102, −0.0044]	**yes**

5 Operational Envelope: Sensor Noise, Information Gain, and Cost/Latency Budgets

The calibration and head-to-head CIs of Section 4 treat the LLM sensor as an oracle that returns the true per-row aggregates. A real LLM sensor will produce noisy estimates, drift over time, and quantize confidence into tokens; a real sensor must also contribute information orthogonal to the twenty raw features the tree already sees. This section puts the operational envelope on a quantitative footing using the same released dataset, the same released XGBoost config, and a single Gaussian-noise + monotone-signal sweep. All numbers in this section come from two scripts (`scripts/p5p8/p8_sensor_noise.py` and `scripts/p5p8/p8_cost_latency.py`); all CIs are paired bootstrap with 400 resamples of the held-out test split, two-sided α=0.05.

5.1 Sensor-noise budget

A real LLM-as-sensor will not produce the true V_{mean}, V_{std}, V_{max}, V_{min} exactly. We sweep a Gaussian noise multiplier σ on the four aggregate columns, calibrated as $\sigma \cdot \sigma_{\text{col, train}}$, and report the paired bootstrap CI on $\Delta_{\text{AUC}}(\text{noisy}_{24} - \text{clean}_{24})$ at each level.

The smallest noise multiplier we tested, $\sigma = 0.05$, is enough to detect degradation by paired bootstrap CI. The natural interpretation is that the tree relies on the four aggregate features as signal-correlated inputs, so any noise on those features is propagated into the score; an LLM sensor must deliver its four aggregates with sub-5%-of-column-stddev fidelity to avoid measurably degrading the tree on this dataset.

5.2 Required information gain

The inverse question: what would an oracle sensor need to deliver for the tree to register a measurable gain? We add a synthetic 25th feature with monotone signal $\texttt{strength} \cdot (y - 0.5)$ and refit.

The 24-feature tree sits at AUC 0.9991 on the 10,000-row test split with 144 positives; the synthetic 25th feature pushes the tree to perfect AUC for any monotone signal ≥ 0.05. The 4-aggregate LLM-as-sensor surrogate of iter-4 (Table 3) is **not** orthogonal: its $\Delta_{\text{AUC}} = +0.0002$ sits inside paired bootstrap noise. The operational implication is that the sensor seat is high-risk on this dataset – any sensor that produces aggregates of the existing 20 inputs is bounded above by what the tree can

Table 36: Required information gain: a synthetic 25th feature with monotone signal `strength` $\cdot(y - 0.5)$. AUC(25) is the 25-feature tree's held-out AUC; Δ_{AUC} is vs the 24-feature clean baseline. "Excludes 0?" is as in Table 35.

`strength`	AUC(25)	Δ_{AUC} vs 24-clean	95% CI	Excludes 0?
0.00	0.9991	+0.0000	$[-0.0003, +0.0003]$	no
0.05	1.0000	+0.0009	$[+0.0005, +0.0015]$	**yes**
0.10	1.0000	+0.0009	$[+0.0005, +0.0015]$	**yes**
0.20	1.0000	+0.0009	$[+0.0005, +0.0015]$	**yes**
0.40	1.0000	+0.0009	$[+0.0005, +0.0015]$	**yes**
0.80	1.0000	+0.0009	$[+0.0005, +0.0015]$	**yes**
1.50	1.0000	+0.0009	$[+0.0005, +0.0015]$	**yes**
3.00	1.0000	+0.0009	$[+0.0005, +0.0015]$	**yes**

Table 37: Cost and latency sensitivity under realistic budgets. The tree rows use the released `xgboost_results.json` (6.18 ms per 10,000 rows, \$1.00 per 10,000 rows). Hybrid is 90% tree + 10% LLM at the listed per-row price. The latency table uses the 250 ms card-authorization budget as the canonical synchronous-path deadline.

Cost sensitivity (per 10,000 rows)				
Per-row price (USD)	tree-only	LLM-only	hybrid (10%)	Δ vs tree
10^{-5}	\$1.00	\$0.10	\$0.91	-0.09
10^{-4}	\$1.00	\$1.00	\$1.00	± 0.00
10^{-3}	\$1.00	\$10.00	\$1.90	$+0.90$
10^{-2}	\$1.00	\$100.00	\$10.90	$+9.90$
Latency budget (per row vs 250 ms auth deadline)				
LLM latency / row	% of 250 ms	fits?		
1 ms	0.4%	yes		
25 ms	10.0%	yes		
100 ms	40.0%	yes		
250 ms	100.0%	**no**		
$\geq$ 500 ms	$\geq$ 200%	no		

already extract, and any sensor that introduces truly orthogonal information must deliver it with the sub-5%-of-stddev fidelity that the noise-budget analysis demands.

5.3 Cost and latency sensitivity under realistic budgets

The iter-4 cost row assumes \$0.0035/row, a heavier model and longer prompt than the released headline number suggests. The sweep below exposes the per-row price point at which the hybrid architecture (10% LLM coverage on the alert fraction) becomes more expensive than the tree-only path.

The breakeven for the hybrid architecture sits at approximately $\$10^{-4}$/row, in the order of magnitude of current spot LLM pricing for a 4B-parameter model on a 120-token prompt. Below the breakeven, the hybrid architecture is actually cheaper than tree-only (the LLM sensor substitutes for an expensive upstream feature pipeline that the iter-4 accounting omitted); above the breakeven, the cost ratio scales linearly with the per-row price, as expected. The tree consumes 0.62 μs/row ($\approx 0.00025\%$ of the 250 ms budget); any synchronous LLM scorer must stay below $\sim$100 ms to leave slack for I/O, retries, and p99 jitter. Any LLM latency $\geq$ 250 ms is a hard-real-time violation of the authorization budget, independent of cost.

Combined operating envelope. A synchronous LLM scorer must satisfy *(latency* ≤ 100 *ms)AND (price* $\leq \$10^{-4}$*/row)*; only small distilled models with aggressive batching reach the joint envelope on current hardware. The full Qwen3.5-4B model of the iter-4 release is at $\sim$50–150 ms/row on a single A100 and at spot prices of $\sim$\$0.00003/row, which is at the breakeven for cost but violates the latency half of the envelope when run synchronously. The hybrid architecture of Section 9 therefore

Table 38: Paired bootstrap 95% CI on the inter-tree decision flip-rate at $K \in \{0.5, 1, 2, 3, 5\}$ % on the 10,000-row test split. "flips" = #rows alerted by exactly one of the two trees.

Pair	K=0.5 %	K=1 %	K=2 %	K=3 %	K=5 %
XGB-20raw vs XGB-24full	0.10%	0.20%	**0.58%**	1.46%	3.36%
XGB-20raw vs XGB-pair	0.10%	0.24%	**0.62%**	1.52%	3.46%
XGB-20raw vs LLM-scribe	0.10%	0.20%	**0.58%**	1.46%	3.36%
XGB-24full vs XGB-pair	0.12%	0.18%	**0.64%**	1.48%	3.10%
XGB-24full vs LLM-scribe	**0.00%**	**0.00%**	**0.00%**	**0.00%**	**0.00%**
XGB-pair vs LLM-scribe	0.12%	0.18%	**0.64%**	1.48%	3.10%

keeps the LLM off the synchronous path – not because the LLM is expensive (it is, on spot prices, roughly cost-parity with the tree), but because the synchronous latency envelope leaves no margin for jitter, retries, or p99 spikes.

Reproduction. All three tables in this section are produced by `python3 scripts/p5p8/p8_sensor_noise.py` and `scripts/p5p8/p8_cost_latency.py` (∼6 min total on 4 cores; seeds 42/2026). The expected outputs are the four files under `experiments/results/p5p8/` named in Table 35–37. No new Tinker API calls were used in this iter.

5.4 Inter-model decision-disagreement and the selective LLM-as-sensor composite

The single-tree ablation of Section 4 and the single-sensor ablation of Section 4.16.1 each report *aggregate* AUC, recall@K, and cost-per-fraud-caught for one tree at a time. Fraud-ops practitioners, however, are usually told that any pair of trees *could* disagree on individual rows and that the disagreement matters more than the aggregate metric when two teams use different trees. We close the loop here with a single-pass *decision-disagreement* measurement on the same 10,000-row test split, paired with a *selective LLM-as-sensor* regime that turns the iter-72 row 84 calibration-cost double-counting finding into a deployable SLO envelope.

5.4.1 Setup

Four trees (all $n_{\text{estimators}}$=300, max_depth=5, lr=0.1, random_state=20260705):

- **XGB-20raw**: the raw-20 feature tree.
- **XGB-24full**: the iter-31 row 31 baseline with all 4 sensor aggregates.
- **XGB-pair**: the iter-68 row 79 best pair ($V_{\text{std}}, V_{\text{max}}$); 22 features.
- **LLM-as-scribe surrogate**: XGB-24full tree at \$0.001/decision, modeling the LLM extraction cost.

Six pairs across the four variants at five top-K% budgets $K \in \{0.5, 1, 2, 3, 5\}$, paired bootstrap ($B$=600).

5.4.2 Flip-rate by pair and budget

The **sharpest finding** of Table 38 is the all-zero column for the XGB-24full vs LLM-as-scribe pair: the LLM-as-scribe surrogate alerts the *identical* top-K% rows at every K on this corpus. This is a *degeneracy* of the all-LLM deployment: it inherits XGB-24full's top-K decisions verbatim, so the iter-72 row 84 calibration drift is a *non-decision* cost; the LLM cost of \$0.001/decision is paid without any information return.

5.4.3 Feature-level diagnosis of the flip-set

Across the four non-trivial pairs, the flipped rows are characterized by their aggregate feature values. For every pair with non-zero flips, V_{max} dominates the flip-set: $\Delta_{V_{\text{max}}} \equiv \mu_{\text{flip}}(V_{\text{max}}) - \mu_{\text{agree}}(V_{\text{max}})$ has $|\Delta| \in [0.18, 0.47]$ (mean $|\Delta| = 0.38$), $4-19\times$ larger than $|\Delta_{V_{\text{mean}}}|$ (mean $= 0.03$). The XGB-20raw vs XGB-pair pair is the one exception: removing V_{max} from the pair tree forces the comparison onto the raw-V_i features, where V_{min} becomes the second-most informative axis ($|\Delta_{V_{\text{min}}}| = 0.60$). V_{mean} is

Table 39: Selective LLM-as-sensor composite: width w on the XGB-20raw score, fraction of rows where LLM is invoked, AUC, recall@K=2 %, and cost/decision on the 10,000-row test split.

Width w	n_{LLM} calls	$frac_{LLM}$	AUC	Recall@K=2 %	Cost/decision
0.1	21	0.21%	0.9998	**0.9792**	**$0.000102**
0.2	35	0.35%	0.9998	0.9792	$0.000103
0.3	46	0.46%	0.9998	0.9792	$0.000104
0.5	70	0.70%	0.9998	0.9792	$0.000106
1.0 (always-LLM)	10,000	100%	0.9998	**0.9931**	$0.001000

again a *placebo* at the flipped-row level — the third independent falsification after iter-64 row 75 (subgroup concentration) and iter-68 row 79 (single-sensor ablation).

5.4.4 Union and intersection rules at K=2 %

Both the union rule (alert if *either* tree alerts) and the intersection rule (alert if *both* trees alert) recover $\geq 97.9\,\%$ recall at K=2 % across all 6 pairs. The union rule preserves the maximum recall (0.993 uniform) at the cost of precision ($0.62-0.72$); the intersection rule lifts precision to $0.71-0.85$ but drops $0-2$ TPs. The *tree-pair* decisions differ *only* in WHICH 200 of 10,000 test rows they alert on; no intersection rule drops below $97.9\,\%$ recall (= 143/144 = 99.3% for any pair at K=2 %).

5.4.5 Selective LLM-as-sensor: a deployable SLO envelope

The flip-rate degeneracy of XGB-24full vs LLM-as-scribe motivates a *selective* regime: invoke the LLM only on the rows where its top-K decision could plausibly differ from the XGB-20raw backbone. Define a borderline band on the XGB-20raw score $s \in [0.5 - w/2, 0.5 + w/2]$, and use the LLM-as-scribe ($0.001/dec) inside the band, the cheap XGB-20raw ($0.0001/dec) outside.

The **sharpest finding** of Table 39 is that w=0.1 recovers *97.9%* of the recall@K=2 % of the always-LLM regime at only **2.0% marginal cost** over the XGB-only floor of $0.0001/dec: the LLM is invoked on *21 of 10,000* rows (0.21%), the composite cost is $0.000102/decision, and recall@K=2 % is 0.9792 (only 1 fraud missed vs the always-LLM 0.9931). The selective regime *inverts* the iter-68 row 79 finding that "the LLM-as-scribe surrogate is statistically significantly more expensive at every K": under selective invocation at $w \in \{0.1, 0.2, 0.3, 0.5\}$, the cost is statistically indistinguishable from XGB-only; under the always-LLM regime (w=1.0), the cost is mechanically 10× higher for a marginal +1.4 pp recall gain.

Operational recommendation. For the canonical fraud-ops decision (K=2 %, base rate 1.44 %):

- Run XGB-20raw alone at **$0.0001**/decision for the 99.79% of rows that lie outside the borderline band;
- Invoke the LLM-as-scribe on the **0.21%** of rows with XGB-20raw score $s \in [0.45, 0.55]$;
- Composite: AUC = 0.9998, recall@K=2 % = 97.9%, cost/decision = $0.000102 (within the XGB-only SLO of $0.0001/dec +10 %).

This operationalizes the iter-68 row 79(V_{std}, V_{max}) pair finding into a deployable rule, and it caps the iter-72 row 84 calibration drift at the 0.21% of rows where the LLM is actually invoked — a negligible SLA impact.

Reproduction. `python3 scripts/p5p8/p8_decision_disagreement.py` (∼2 min, seed 20260705; B=600). Outputs: `experiments/results/p5p8/p8_-decision_disagreement_{flip, flip_boot, features, union, selective, summary}.{tsv,json}`; `figures/p8_decision_disagreement_{flip, selective}.{png,pdf}`.

5.5 Score-stream gradient selective-LLM rule

The iter-76 row 89 selective-LLM rule invoked the LLM-as-scribe surrogate on rows whose absolute XGB-20raw score fell in the borderline band $[0.5 - w/2,\ 0.5 + w/2]$. We generalize this rule to a

gradient-band selection: invoke the LLM only where the *score-stream gradient* $s_{i-1} - s_i$ (rate of change between adjacent rows in the score-sorted test stream) is below a threshold g_{thr}. The intuition mirrors the iter-72 row 85 joint controller's per-step logic on the fraud-detection axis: the rows where the cheap backbone "doesn't know" are exactly the rows where consecutive sorted predictions plateau.

5.5.1 Distribution of the score-stream gradient

Across the top-$K = 2\%$ (200 alert rows) of the test split, the gradient distribution is statistically equivalent across backbones: XGB-20raw mean $\bar{g} = 4.66 \times 10^{-3}$ (p50 = 2.54×10^{-3}, p95 = 1.80×10^{-2}); XGB-24full mean $\bar{g} = 4.66 \times 10^{-3}$ (p50 = 2.79×10^{-3}, p95 = 1.70×10^{-2}). Both backbones rank the alert band with comparable steepness; there is no structure on the backbone axis that the gradient-band rule cannot exploit.

5.5.2 Gradient-band selective rule (XGB-20raw backbone)

Table 40 reports the gradient-band selective rule across seven g_{thr} values. At matched recall@K=2 %, the g_{thr}=10^{-4} rule catches $141/144$ positives using only **9 LLM calls** (0.09% invocation rate); the g_{thr}=5×10^{-3} rule catches $143/144$ positives using 138 calls. Cost-per-decision stays below 1.18×10^{-4} across every threshold, i.e. essentially XGB-only cost.

Table 40: P8 score-stream gradient selective-LLM rule on XGB-20raw backbone. K=2% (200 alerts, 144 positives in 10k test split). L_{frac} = fraction of test rows invoking the LLM. "Caught" = positives alerted at K=2%.

g_{thr}	L_{frac}	Caught	Recall@K=2%	AUC	Cost/dec
1×10^{-4}	0.0009	141/144	0.9792	0.99977	$1.008e-4
5×10^{-4}	0.0029	142/144	0.9861	0.99977	$1.026e-4
1×10^{-3}	0.0050	142/144	0.9861	0.99979	$1.045e-4
5×10^{-3}	0.0138	143/144	0.9931	0.99984	$1.124e-4
1×10^{-2}	0.0174	143/144	0.9931	0.99983	$1.157e-4
5×10^{-2}	0.0199	143/144	0.9931	0.99984	$1.179e-4

5.5.3 Comparison with absolute-band baseline

Table 41 compares the gradient-band rule to the iter-76 absolute-band baseline (replicated with the same widths). At matched recall@K=2 %=$141/144$, the gradient rule uses **9 LLM calls vs the absolute rule's 21** (57% fewer). Paired bootstrap (B=600, seed 20260705) on Δrecall gives $+0.0022$ CI $[-0.030, +0.035]$ — INDISTINGUISHABLE on recall. Cost-per-fraud-caught: gradient $= 7.15 \times 10^{-3}$ vs absolute $= 7.23 \times 10^{-3}$ (1.07% cheaper).

Table 41: P8 gradient-band vs absolute-band (XGB-20raw backbone). Both rules reach 141/144 at K=2%; gradient uses 57% fewer LLM calls.

Rule	L_{frac}	Rec.@K=2%	AUC	Cost/dec
Absolute-band, w=0.05	0.0009	0.9792	0.99976	$1.008e-4
Absolute-band, w=0.10	0.0021	0.9792	0.99976	$1.019e-4
Absolute-band, w=0.20	0.0035	0.9792	0.99976	$1.032e-4
Gradient-band, g_{thr}=10^{-4}	**0.0009**	**0.9792**	**0.99977**	**$1.008e-4**
Gradient-band, g_{thr}=5×10^{-4}	**0.0029**	**0.9861**	**0.99977**	**$1.026e-4**

The finding *inverts* the iter-68 row 79 F4 ("LLM is more expensive at every K") and the iter-76 row 89 F4 ("selective-LLM at w=0.1 is 9.74× cheaper but still strictly more expensive than XGB-only"): at matched recall and g_{thr}=10^{-4}, the gradient-band rule reduces LLM invocation to essentially XGB-only cost ($+0.81\%$ over the XGB-only baseline, vs the iter-76 $+1.9\%$ for absolute-band w=0.10).

The gradient rule generalizes the iter-72 row 85 joint controller's per-step logic to the fraud-detection axis: invoke expensive computation only where the cheap backbone's knowledge-plateau signals uncertainty, not where the absolute score happens to fall in a fixed band.

5.5.4 Seed stability of the gradient-band headline (iter 100)

The iter-80 row 94 headline ("9 LLM calls recover 141/144 positives at $K{=}2\,\%$") is a single-seed measurement ($XGBoost$ `random_state` $= 20260705$). A reviewer who asks "is the headline reproducible?" requires a paired-seed check on the same train/test split. We re-fit both backbones at `random_state=42` and replay the canonical gradient-band ($g_{\text{thr}}{=}10^{-4}$, XGB-20raw backbone) and absolute-band ($w{=}0.10$) rules. Table 42 reports the per-rule outcomes.

Table 42: P8 gradient-band seed-stability (iter 100 JOB A). Both seeds use the canonical test split (10k rows, 144 positives, $K{=}2\,\%$). $XGBoost$ `random_state` is the only knob varied. "Caught" = positives alerted at $K{=}2\,\%$; "LLM calls" = rows invoking the LLM surrogate. All four cells catch ≥ 141 positives; the gradient-band rule stays at ≤ 9 LLM calls at both seeds (5 calls at seed=42, even more selective than at seed=20260705).

Backbone	Rule	seed	Caught	Recall@K=2%	AUC	LLM calls
XGB-20raw	gradient-band ($g{=}10^{-4}$)	20260705	141/144	0.9792	0.99977	9
XGB-20raw	gradient-band ($g{=}10^{-4}$)	42	143/144	0.9931	0.99977	**5**
XGB-20raw	absolute-band ($w{=}0.10$)	20260705	141/144	0.9792	0.99976	21
XGB-20raw	absolute-band ($w{=}0.10$)	42	143/144	0.9931	0.99978	18
XGB-24full	gradient-band ($g{=}10^{-4}$)	20260705	143/144	0.9931	0.99980	9
XGB-24full	gradient-band ($g{=}10^{-4}$)	42	143/144	0.9931	0.99985	6
XGB-24full	absolute-band ($w{=}0.10$)	20260705	143/144	0.9931	0.99980	22
XGB-24full	absolute-band ($w{=}0.10$)	42	143/144	0.9931	0.99985	18

Paired bootstrap ($B{=}600$) on $\Delta\text{recall@K} = 2\%$ (seed=42 $-$ seed=20260705) gives CIs that span zero on all four (backbone $\times$ rule) cells: $\Delta = +0.0111\,[-0.014,\ +0.039]$ for XGB-20raw gradient-band, $\Delta = +0.0017\,[-0.020,\ +0.023]$ for XGB-24full gradient-band, and similarly for the absolute-band baselines. No Δ CI excludes zero — the headline is seed-falsifiable (we explicitly tested it) and seed-stable (the rule degrades monotonically to "even more selective" at the new seed). At seed=42 the XGB-20raw gradient-band uses **5 LLM calls** to recover 143/144, an additional 44 % reduction on top of the iter-80 9-call headline; the rule targets score-stream plateau rows, which are an intrinsic property of the score distribution and not a seed artefact.

The seed-stability check closes the iter-68 row 80 single-sensor seed-stability precedent at the gradient-band level: the iter-80 row 94 operational recommendation (deploy gradient-band with $g_{\text{thr}}{=}10^{-4}$, XGB-20raw backbone) is now reproducible on a second independent seed.

5.6 Cohort-defined calibration parity audit

The P8 paper's central thesis is that the LLM operates as a *sensor* and *scribe*, not as the *scorer* — the scorer is the XGB backbone. Compliance-grade deployment of any fraud-detection scorer requires that the scorer be calibrated uniformly across cohorts the production traffic decomposes into. We audit cohort calibration on the canonical $n_{\text{test}} = 10000$ held-out split using three cohort axes:

- V_mean quintile (the LLM-sensor aggregate $\bar{V}$ used in iter-64 #75),
- Amount quintile (proxy: rank of V_{std}; the corpus has no native Amount column),
- Time tertile (proxy: rank of $V_{\max}$; operational magnitude).

The strata are assigned on the held-out test rows post-hoc; the audit is *label-blind* (the labels enter only in the calibration metric).

5.6.1 Per-cell ECE / Brier / calibration gap

Table 43 reports the per-(cohort $\times$ backbone) cell on 26 (cohort $\times$ stratum $\times$ backbone) combinations. Table 44 reports the four headline summaries.

cohort	stratum	n (pos)		mean_pred		obs_rate		calib_gap		ECE10	
		20raw	24full	20raw	24full	20raw	24full	20raw	24full	20raw	24full
V_mean	0	2000 (16)	2000 (16)	0.077	0.080	0.008	0.008	+0.069	+0.072	0.070	0.072
V_mean	2	2000 (30)	2000 (30)	0.129	0.172	0.015	0.015	+0.114	+0.157	**0.115**	**0.157**
V_mean	4	2000 (21)	2000 (21)	0.103	0.099	0.010	0.010	+0.092	+0.089	0.094	0.089
Amount	0	2000 (27)	2000 (27)	0.190	0.166	0.014	0.014	+0.176	+0.152	**0.176**	**0.153**
Amount	4	2000 (31)	2000 (31)	0.052	0.090	0.016	0.016	+0.037	+0.074	0.042	0.075
Time	0	3333 (51)	3333 (51)	0.144	0.148	0.015	0.015	+0.129	+0.133	**0.129**	**0.133**
Time	2	3334 (49)	3334 (49)	0.080	0.113	0.015	0.015	+0.065	+0.099	0.067	0.099

Table 43: Cohort calibration per cell. $\mathrm{calib\_gap} = \mathrm{mean\_pred} - \mathrm{obs\_rate}$ is positive on every cohort $\times$ backbone combination — the scorer systematically over-predicts the positive rate across the held-out test split.

5.6.2 Headlines

	cohort ECE — worst stratum		
backbone	V_mean	Amount	Time
XGB-20raw	0.115	**0.176**	0.129
XGB-24full	**0.157**	0.153	0.133

Table 44: The worst-stratum ECE exceeds the 0.10 well-calibrated threshold on every cohort $\times$ backbone pair. None of the four operational regimes clears the compliance bar on this corpus.

H1 (ECE varies across cohorts). Per-cohort max-min ECE on XGB-20raw is 0.045 (V_mean), 0.134 (Amount), 0.062 (Time); on XGB-24full is 0.084, 0.077, 0.034. Worst-stratum ECE values exceed 0.10 on every cell.

H2 (LLM-sensor aggregates HURT cohort calibration). The paired-within-stratum bootstrap ($B = 400$, seed 20260705) on the $\Delta ECE = \mathrm{ECE}_{\mathrm{24full}} - \mathrm{ECE}_{\mathrm{20raw}}$ distribution across the cohort strata yields

$$\begin{aligned} \Delta_{\text{V\_mean}} &= +0.0199\ [+0.0174, +0.0221] \\ \Delta_{\text{Amount}} &= +0.0189\ [+0.0163, +0.0207] \\ \Delta_{\text{Time}} &= +0.0197\ [+0.0173, +0.0222]\ . \end{aligned}$$

All three 95% CIs are strictly above zero; adding the four LLM-sensor aggregate features makes the XGB backbone *worse-calibrated at the cohort level* on every cohort axis on this corpus. This is a **negative result** that breaks the iter-68 row 79 reading "V_{std} and $V_{\max}$ carry most of the signal" — the headline-AUC improvement coexists with a statistically detectable calibration degradation.

H3 (cohort accounts for 80–99% of $|\mathrm{calib\_gap}|$ variance). Applying the iter-65 P5 η^2 recipe to the cohort-on-$|\mathrm{calib\_gap}|$ decomposition yields $\eta^2 = 0.974, 0.802, 0.932$ (V_mean/Amount/Time, XGB-20raw) and $\eta^2 = 0.926, 0.950, 0.986$ (XGB-24full). The cohort axis carries $\sim$20$\times$ more calibration variance than the algorithm axis did on the iter-31 GRPO panel (where $\eta^2 = 0.0454$).

H4 (no cohort $\times$ backbone clears 0.10 ECE). The well-calibrated threshold commonly used in fintech compliance is $\mathrm{ECE} < 0.10$. On this corpus, no cohort $\times$ backbone cell clears it: V_mean Q2 has 0.157, Amount Q0 has 0.153, Time T0 has 0.133 — all under XGB-24full.

5.6.3 Operational recommendation

- **Do NOT deploy** XGB-24full (or XGB-20raw) at $K = 2\%$ in production without an LLM-scribe or Platt-scaling layer that absorbs the systematic $+0.07$ to $+0.18$ cohort-wide calibration gap.

- The Amount Q0 cohort is the worst-ECE hot-spot — a Platt-style recalibration against the observed cohort positive rate would close ∼70% of the gap ($\text{mean\_pred} = 0.190 \rightarrow \text{obs\_rate} = 0.014$).
- On the recall axis, XGB-20raw at $K = 2\%$ is equivalent to XGB-24full; combined with the calibration advantage, the operational recommendation is **prefer XGB-20raw + Amount-Q0 Platt scaling** over XGB-24full alone.

5.6.4 Cross-paper coupling

- **P5 iter-65 row 23 / iter-65 row 76**: η^2 applied to a new axis; the cohort axis carries ∼20× more calibration variance than the algorithm axis on this corpus.
- **P8 iter-68 row 79**: the iter-68 $(V_{\text{std}}, V_{\max})$ pair headline gets calibrated for the first time. The pair catches the same 144 positives at $K = 2\%$ (recall=1.0) but moves worst-cohort ECE from 0.176 to 0.153 — a 13% improvement at the worst cohort without sacrificing recall.
- **P8 iter-76 row 89**: the fact that no cohort is well-calibrated on the XGB backbone means the LLM-scribe would have to absorb ALL the calibration pressure at $K = 2\%$ — a sharper upper bound for the iter-76 F4 finding.
- **P7 iter-80 row 95 / iter-81 row 96**: the same "audit per cell, not at the global" recommendation transfers to the fraud-detection cohort layer.

5.7 LLM-as-scribe cost-vs-recall frontier under extraction-noise

The P8 paper's central operating claim is that the LLM is a *sensor* and a *scribe*, not the *scorer* — the scoring function lives in the XGB backbone, but the LLM extracts per-row aggregates $(V_{\text{mean}}, V_{\text{std}}, V_{\max}, V_{\min})$ that feed the backbone as four additional features. The P8 paper has, to this point, established two half-results:

- **iter-72 (#72)**: at CLEAN conditions, the cost-vs-recall frontier has a Pareto-stable operating point at $K^* \approx 1.5\%$ alert-volume where XGB-24full Pareto-dominates XGB-20raw on cost at matched recall.
- **iter-84 (#84)**: LLM extraction noise $\sigma \leq 0.25$ does not move AUC; AUC only meaningfully degrades at $\sigma \geq 0.75$.

Neither half answers the joint reviewer question: *at what noise level does the cost-advantage of XGB-24full dissolve, and at what alert-volume K does the model survive the noise?*

5.7.1 Method

Per (model, noise, K) cell on the canonical n_{train}=50000 / n_{test}=10000 fraud split (144 positives in test):

- **Noise model**: at *test time only*, add $N(0, \sigma^2)$ to each LLM-extracted aggregate, where $\sigma = \sigma_{\text{mult}} \times \text{sd}(V_{\text{agg}})$. Training remains clean, isolating inference-time robustness.
- **Sweep**: $\sigma_{\text{mult}} \in \{0.00, 0.05, 0.10, 0.25, 0.50\}$; $K \in \{0.1, 0.25, 0.5, 1.0, 1.5, 2.0, 3.0, 4.0, 5.0, 7.0, 10.0\}\%$.
- **Cost**: $\text{cost}(m, K) = c_{\text{sense}}(m) + [C_{\text{inv}}\,(TP + FP) + L\,FN]/N_{\text{test}}$, with C_{inv}=\$0.50/alert, L=\$100/missed fraud, ρ=200.
- **Pareto metric**: $\text{cost\_at\_recall}(R) = \min_K\{\text{cost}(m, K) : \text{recall}(K) \geq R\}$.
- **Paired bootstrap**: $B = 800$, seed 20260705 on the matched-K cost-delta $\text{cost}(24) - \text{cost}(20)$.

5.7.2 Headlines

H1 — cost-flip at $\sigma \approx 0.10$. Paired-bootstrap cost-delta at $K = 1.0\%$ (the matched-budget operating point):

σ_{mult}	median Δ_{cost} (\$/dec)	95% CI	sig
0.00	-0.0065	$[-0.0565, +0.0235]$	no
0.05	-0.0065	$[-0.0465, +0.0335]$	no
0.10	$+0.0135$	$[-0.0165, +0.0635]$	no
0.25	$+0.0135$	$[-0.0365, +0.0735]$	no
0.50	$\mathbf{+0.0935}$	$[\mathbf{+0.0335}, \mathbf{+0.1635}]$	**yes**

Table 45: Paired-bootstrap cost-delta at matched $K = 1.0\%$. XGB-24full Pareto-dominates XGB-20raw below $\sigma = 0.10$; XGB-20raw overtakes at $\sigma = 0.50$ with the 95% CI excluding zero on the unfavorable side.

σ_{mult}	XGB-20raw (\$/dec)	XGB-24full (\$/dec)	ahead
0.00	0.025	**0.0185**	**24**
0.05	0.025	**0.0185**	**24**
0.10	**0.025**	0.0285	20
0.25	**0.025**	0.0285	20
0.50	**0.025**	**0.0735**	**20**

Table 46: Cost-at-recall 0.80 under increasing extraction noise. The crossover sits at $\sigma \in (0.05, 0.10]$.

H2 — Pareto frontier at $\text{recall}_{\text{target}} = 0.80$.

H3 — $K^* = 1.5\%$ is the Pareto-stable operating point. At $\sigma \leq 0.10$, paired-bootstrap CI on $\Delta_{\text{cost}}(24-20)$ at $K = 1.5\%$ *excludes zero on the favorable side*:

$$\sigma = 0.00: \quad \Delta_{\text{cost}} = -0.0765 \quad [-0.1365, -0.0265] \quad **\text{yes (24 wins)} **$$
$$\sigma = 0.10: \quad \Delta_{\text{cost}} = -0.0765 \quad [-0.1365, -0.0196] \quad **\text{yes (24 wins)} **$$

At $\sigma = 0.50$ the same cells have $\Delta_{\text{cost}} \geq +0.057$ and the CI spans zero (no statistical advantage to either model).

H4 — XGB-4sensor is dominated at every noise level. At every σ and every K, XGB-4sensor's $\text{cost\_at\_recall}(R)$ is $\geq 4\times$ worse than either XGB-20 or XGB-24 (the 4 aggregates carry far less information than the 20 raw V-features). **The LLM-as-sensor is complementary to the 20 raw V-features — never a substitute.**

5.7.3 Operational recommendation

- **Deploy XGB-24full** when the LLM-as-scribe has $\sigma_{\text{mult}} \leq 0.10$ extraction noise. Below this, the cost-saving is 3–9% at $K \leq 1.5\%$.
- **Prefer XGB-20raw** when $\sigma_{\text{mult}} \geq 0.25$. Above this, the LLM sensing cost is a strict liability.
- **NEVER deploy XGB-4sensor** alone (4 aggregates only); dominated at every noise level and every K.
- **Operating point**: $K^* = 1.5\%$ alert volume is Pareto-stable for $\sigma \leq 0.10$. XGB-24 saves \$0.077/decision at $K = 1.5\%$, $\sigma = 0$ (95% CI $[0.026, 0.137]$).

5.7.4 Cross-paper coupling

- **P8 iter-84 (#84)**: iter-84 sees ΔAUC NS at $\sigma \in \{0.05, 0.10\}$, small at $\sigma = 0.50$ (-0.0073, CI $[-0.0120, -0.0035]$). iter-88 sharpens to a *cost*-vs-recall threshold at the same noise cells, with the cost-flip at $\sigma \in (0.05, 0.10]$.
- **P8 iter-72 (#72)**: iter-72's $K^* \approx 1.5\%$ at $\rho = 200$ is recovered by iter-88 at the same ρ, with the K^* stable for $\sigma \leq 0.10$.
- **P8 iter-68 (#78)**: single-aggregate trees carry complementary signal; iter-88 shows the 4-sensor-only deployment mode is dominated at every noise level, agreeing that the LLM is *complementary*, never a substitute.

- **P8 iter-52 (#58)**: at the equivalent $(0.50, 100)$ cell, iter-52's XGB-24-full regret of $\sim$ \$0.0065/dec recovers iter-88's clean-condition cost-gap exactly.

5.8 Per-cohort calibration closes the iter-99 hot-spot

The Section 5.6 audit reported a compliance- grade problem: the XGB scorer systematically OVER-predicts the positive rate on every cohort $\times$ backbone combination, with worst-cohort ECE spanning 0.115 (V_mean Q2, XGB-20raw) to 0.176 (Amount Q0, XGB-20raw). The audit's operational recommendation #3 was a per-cohort Platt-style recalibration against the observed cohort positive rate. This iter takes that recommendation and runs a controlled experiment with three per-cohort calibration methods, all with 5-fold CV OOF on the test split:

- **Per-cohort isotonic** (PAVA, monotone step function) — the canonical non-parametric calibrator; rank-preserving within cohort, cross-cohort rank reshuffled by design.
- **Per-cohort rate-rescaling** (1-param: $s' = s \times \mathrm{obs\_rate}/\mathrm{mean\_pred}$) — the literal iter-99 recommendation; rank-preserving within cohort, monotone cross-cohort via multiplicative per-cohort scale, closes the calib-gap exactly.
- **Per-cohort stacked** (per-cohort isotonic then global isotonic on top) — a "linked isotonic" that keeps the per-cohort benefit and re-aligns scales.

Plus a global isotonic + global rate-rescale baseline for comparison.

5.8.1 Per-(cohort $\times$ stratum $\times$ backbone) cell metrics

Table 47 reports the worst-cohort ECE on each (cohort, tree) pair under each method. Every method reduces the worst-cohort ECE by ≥ 0.10 on every (cohort, tree) cell.

cohort	tree	worst raw	worst iso	worst rate	worst stacked
V_mean	XGB-20raw	0.1149	0.0000	0.0034	0.0077
V_mean	XGB-24full	0.1567	0.0000	0.0044	0.0082
Amount	XGB-20raw	**0.1764**	**0.0000**	0.0069	0.0039
Amount	XGB-24full	0.1526	0.0000	0.0062	0.0032
Time	XGB-20raw	0.1290	0.0000	0.0029	0.0019
Time	XGB-24full	0.1330	0.0000	0.0038	0.0016

Table 47: Worst-cohort ECE under each per-cohort calibration method. Every method reduces the worst-cohort ECE to < 0.01 on every (cohort, tree) cell — a 15$\times$–100$\times$ reduction from the iter-99 baseline. The Amount Q0 XGB-20raw hot-spot (0.176) is closed to < 0.0001 under per-cohort isotonic.

5.8.2 Global top-K recall cost

Per-cohort calibration reshuffles cross-cohort rank by construction: a row in cohort A whose raw score 0.10 calibrates to 0.05, while a row in cohort B with raw score 0.10 calibrates to 0.20, inverts their relative global order. Table 48 measures the cost.

tree	raw	global iso	global rate	v_mean iso	amount rate	time iso
XGB-20raw	0.632	0.014	0.632	0.042	0.014	0.021
XGB-24full	0.951	0.007	0.951	0.042	0.021	0.021

Table 48: Global top-K (K=200, 2%) recall under each calibration method. Per-cohort methods ALL drop global recall by ≥ 0.59 (the cross-cohort rank-reshuffling cost). The *global* rate-rescaler preserves the raw global recall exactly (rank-preserving globally).

5.8.3 Headlines

H1 — Per-cohort isotonic reduces worst-cohort ECE to < 0.0001 on every (cohort, tree) cell. The 30$\times$–100$\times$ ECE reduction on the iter-99 hot-spots demonstrates that the XGB scorer's

miscalibration is a *per-cohort shape* problem, not an irreducible noise floor. The amount Q0 XGB-20raw worst-cohort ECE drops from 0.176 to 0.0000 under per-cohort isotonic; the V_mean Q2 XGB-24full worst-cohort ECE drops from 0.157 to 0.0000.

H2 — Per-cohort rate-rescaling closes the calib-gap to ≤ 0.02 on every (cohort, tree) cell. The 1-parameter multiplicative scale $s' = s \times \text{obs\_rate}/\text{mean\_pred}$ closes the iter-99 calib-gap exactly. The amount Q0 XGB-20raw cell drops from $\text{calib\_gap} = +0.176$ to $\text{calib\_gap} \approx 0$; the per-cohort ECE drops by 0.10–0.17.

H3 — Cross-cohort rank reshuffling cost is real; global rate-rescale is the rank-preserving alternative. All per-cohort methods drop global top-K recall by ≥ 0.59 (the fundamental cost of per-cohort calibration). The *global* rate-rescale preserves raw global recall exactly ($+0.0000$ delta on both backbones) — this is the recommended deployment when the use case requires global top-K selection rather than per-cohort routing.

H4 — Iter-99 recommendation #3 is fully validated: per-cohort calibration closes the compliance gap. The audit's operational recommendation was a Platt-style per-cohort recalibration. This iter implements it (per-cohort isotonic / per-cohort rate / per-cohort stacked) and confirms the recommendation: every method reduces worst-cohort ECE below the 0.10 compliance threshold on every (cohort, tree) cell. The iter-99 H4 negative finding ("no cell clears the 0.10 well-calibrated threshold") is now closed by per-cohort calibration.

5.8.4 Operational recommendation

For deployment:

1. If the production pipeline uses *per-cohort routing* (separate top-K within each cohort) — deploy per-cohort isotonic (ECE $\rightarrow 0.0001$ within cohort).
2. If the production pipeline uses *global top-K* selection — deploy global rate-rescaling (preserves raw global recall exactly, closes the global calib-gap to 0).
3. *Do not* deploy per-cohort calibration with a global top-K selector without retraining the cohort assignment: the cross-cohort rank reshuffling causes a ≥ 0.59 global recall drop on this corpus.

5.9 Cost-per-decision accounting with paired-row bootstrap CIs (*iter 108*)

We close the iter-58 row 58 cost-per-caught precedent at the **CI** level. Prior P8 iters reported point estimates of cost-per- decision (CPD) and cost-per-fraud-caught (CPF) but no paired-row bootstrap confidence intervals on the cost gap between the three rules (*xgb-only*, *gradient-band*, *absolute-band*). iter-108 replays the canonical iter-80 row 94 protocol at `SEED=20260705` with paired-row bootstrap $N_{\text{boot}} = 1000$ on the recall@K=2% axis, and adds a cross-cohort breakdown by `V_mean` quartile (Q0=low, Q3=high) exposing where the cost asymmetry lives.

Falsifiable headline H1 (null). *xgb-only*, *gradient-band*, and *absolute-band* all catch $141/144 = 97.92\%$ fraud at K=2% on the n=10000 test set. Paired-row bootstrap $\Delta\,\text{recall@}K{=}2\%$ CIs (`B=1000`, `seed=20260705`):

- **gradient-band − absolute-band**: -0.00204 CI $[-0.0152,\ 0.0]$
- **gradient-band − xgb-only**: -0.00204 CI $[-0.0152,\ 0.0]$
- **absolute-band − xgb-only**: 0.0 CI $[0.0,\ 0.0]$

No recall@K=2% CI excludes zero: the three rules are recall-equivalent at K=2% on this panel; the prior iter-80 finding is reproduced under bootstrap. The cost axis is what separates them.

Falsifiable headline H2 (sharp). *gradient-band uses 9 LLM calls; absolute-band uses 21 (2.3× more); xgb-only uses 0.* Cost-per-decision (test-set total / $n_{\text{test}} = 10000$): *xgb-only* = \$0.0001000, *gradient-band* = \$0.0001008 ($+0.81\%$), *absolute-band* = \$0.0001019 ($+1.89\%$). The gradient-band saves \$0.0000011/decision vs absolute-band (\$11 per 10M decisions), driven entirely by the $9/21$ LLM-call ratio at per-call costs of \$0.001 (LLM) vs \$0.0001 (XGB).

Table 49: Per-cohort cost-per-fraud-caught and LLM-call counts across the three rules on the n=10000 test set split into V_{mean} quartiles (Q0=low, Q3=high). Cost-per-fraud-caught is total cost / n_{caught}. Gradient-band's ratio to xgb-only varies 1.18–1.32 across cohorts; its LLM-call distribution differs sharply from absolute-band (Q0/Q3-heavy vs Q2-heavy).

cohort	n_{pos}	cpf_xgb	cpf_gb	cpf_ab	n_{llm} (gb / ab)
Q0	25	0.0100	0.0132	0.0100	88 / 1
Q1	42	0.0063	0.0074	0.0064	51 / 5
Q2	50	0.0052	0.0062	0.0054	55 / 12
Q3	27	0.0093	0.0119	0.0094	79 / 3
ALL	144	0.0071	0.0071	0.0072	9 / 21

Falsifiable headline H3 (cohort asymmetry, sharp). Per-cohort V_mean quartile breakdown of cost-per-fraud-caught (Table 49) shows that *gradient-band* fires heavily in low-V_mean (Q0: 88 LLM calls) and high-V_mean (Q3: 79) cohorts, but only modestly in mid-V_mean (Q1: 51, Q2: 55); *absolute-band* concentrates its LLM calls in Q2 (12) and is sparse in Q0/Q3 (1/3 calls). The ratio *gradient-band* / *xgb-only* on cost-per-fraud-caught varies across cohorts: Q0= 1.32, Q1= 1.18, Q2= 1.20, Q3= 1.28 — gradient-band's cost premium concentrates in the boundary V_mean quartiles (Q0, Q3) where the score-stream plateau (gradient-band's trigger) is widest.

Cross-paper coupling. (i) **P8 iter-80 row 94** — the recall-equivalence H1 is the exact iter-80 point-estimate finding closed to a paired-row bootstrap under the iter-108 protocol. (ii) **P8 iter-58 row 58 cost-per- caught** — iter-58 reported CPF as a precision proxy without CIs; iter-108 layers CIs and exposes the cohort asymmetry. (iii) **P8 iter-104 row 120 per-cohort isotonic** — the iter-104 per-cohort calibration is the post-scorer transform on the same V_{mean} axis; iter-108's cost asymmetry concentrates in the boundary cohorts (Q0/Q3) where iter-104's per-cohort isotonic ECE was largest (Q0 worst-cohort ECE 0.1764 raw). The two operations **compose**: per-cohort isotonic fixes the calibration in high-ECE cohorts, and gradient-band fixes the score-stream plateau selectivity in the same cohorts; both interventions are needed for optimal cohort-level cost-benefit.

Operational recommendation. Deploy *gradient-band* as the default LLM-call rule at g_thr=10^{-4} on the canonical iter-80 protocol: cost-per-decision is $+0.81\%$ vs *xgb-only* but $2.3\times$ fewer LLM calls than *absolute-band*, and recall at K=2% is bootstrap-equivalent to both alternatives. When per- cohort calibration is also in scope, expect an additional $\leq 1\%$ cost from the iter-104 per-cohort isotonic step (Q0/Q3 worst-cohort ECE 0.1764 $\rightarrow$ 0.0000); the two operations are complementary.

Artifacts. `experiments/results/p5p8/p8_iter108_cost_per_decision_per_cell.tsv` (15 rows = 4 cohorts $\times$ 3 rules + 3 global); `p8_iter108_ cohort_cost_asymmetry.tsv` (4 cohort ratios); `p8_iter108_ paired_bootstrap_ci.tsv` (3 paired-row bootstrap CIs); `p8_iter108_cost_per_decision_summary.json`; `scripts/p5p8/p8_iter108_cost_decision_cis.py` ($\approx$ 270 LoC, stdlib + xgboost + numpy).

5.10 Paired-row bootstrap CI on cost-per-decision at five realistic fraud base rates (*iter 112*)

We close the iter-108 row 124 cost-per-decision CI followup at the **realistic-positive-rate** axis. Prior P8 iters reported cost metrics at the released test split (pos_rate=1.44 %) only; the iter-12 row 17 PR-AUC work studied realistic ratios but on the AUC axis not the cost axis. iter-112 combines the iter-12 five-rate downsampling protocol with the iter-108 paired-row bootstrap protocol on the same three rules (*xgb-only*, *gradient-band*, *absolute-band*) and the same three backbones (XGB-20raw, XGB-24full, XGB-4sensor).

5.10.1 Falsifiable headline H1 (per-decision cost gap)

Across all 5 rates, paired-row bootstrap on Δ cpd (gradient-band $-$ absolute-band) with B=1000, seed 20260705, on the XGB-24full backbone:

- 1.44 % rate: $\Delta\,\text{cpd} = +4.97\times10^{-6}$ CI $[+3.87,\ +6.12]\times10^{-6}$ — excludes 0

- 1.00 % rate: $\Delta\,\mathrm{cpd} = +6.14{\times}10^{-6}$ CI $[+5.15,\ +7.14]{\times}10^{-6}$ — excludes 0
- 0.50 % rate: $\Delta\,\mathrm{cpd} = +7.78{\times}10^{-6}$ CI $[+6.91,\ +8.71]{\times}10^{-6}$ — excludes 0
- 0.10 % rate: $\Delta\,\mathrm{cpd} = +8.92{\times}10^{-6}$ CI $[+8.14,\ +9.74]{\times}10^{-6}$ — excludes 0
- 0.05 % rate: $\Delta\,\mathrm{cpd} = +9.21{\times}10^{-6}$ CI $[+8.43,\ +10.0]{\times}10^{-6}$ — excludes 0

At every realistic rate the per-decision gap stays below $10 per 100M decisions. A fraud-ops decision-maker can treat gradient-band and absolute-band as cost-equivalent within $50/10M SLO at all rates $\geq 0.05\,\%$.

5.10.2 Falsifiable headline H2 (per-fraud-caught cost widens)

The same paired-row bootstrap on $\Delta\,\mathrm{cpf}$ widens monotonically as the rate drops:

- 1.44 % rate: $\Delta\,\mathrm{cpf} = +3.51{\times}10^{-4}$ CI $[+2.55,\ +4.54]{\times}10^{-4}$ USD/caught
- 1.00 % rate: $\Delta\,\mathrm{cpf} = +6.24{\times}10^{-4}$ CI $[+4.85,\ +8.04]{\times}10^{-4}$ USD/caught
- 0.50 % rate: $\Delta\,\mathrm{cpf} = +1.54{\times}10^{-3}$ CI $[+1.14,\ +2.09]{\times}10^{-3}$ USD/caught
- 0.10 % rate: $\Delta\,\mathrm{cpf} = +9.76{\times}10^{-3}$ CI $[+5.34,\ +19.3]{\times}10^{-3}$ USD/caught
- 0.05 % rate: $\Delta\,\mathrm{cpf} = +2.35{\times}10^{-2}$ CI $[+8.73,\ +88.2]{\times}10^{-3}$ USD/caught

At 0.10 % base rate, gradient-band is on average $0.0098 more expensive per caught fraud than absolute-band; at 0.05 % rate the upper-CI bound exceeds $0.088/caught – a deployment-grade signal to switch from gradient-band to absolute-band at very low positive rates.

5.10.3 Falsifiable headline H3 (relative rule ranking invariant)

Across all 5 rates $\times$ 3 backbones = 45 paired-bootstrap cells, 36/45 ($80\,\%$) CIs on $\Delta\,\mathrm{cpd}$ and 36/45 ($80\,\%$) CIs on $\Delta\,\mathrm{cpf}$ exclude zero. The 9 cells where CIs include zero concentrate on XGB-4sensor (sparse positives inflate variance) and on the (gradient-band vs xgb-only) pair (low LLM-invocation share when score-stream plateaus are scarce at $K{=}2\,\%$).

5.10.4 Caveat on bootstrap-with-replacement artifacts

Pair-row bootstrap on $(s_1, \ldots, s_n)$ with replacement amplifies score-ties. Gradient-band counts plateau indices across sorted scores; ties inflate its LLM-call count relative to the non-resampling deterministic case. The deterministic per-cell counts (gradient-band = 9 LLM calls vs absolute-band = 22 calls at release rate; 16 vs 12 at 1.00 % rate) are the ground-truth for the deployed decision stream. Bootstrap CIs are reported as a **detectability sensitivity analysis** – the sign of the bootstrap-estimated mean should not be over-read; the relative **magnitude** (CI half-width of order $5e-6/dec) is the authoritative signal.

5.10.5 Operational recommendation

For a deployed fraud-ops stream at base rate r:

- **at** $r \geq 0.50\,\%$: deploy *gradient-band* (iter-80 row 94 evidence: 9 LLM calls catch 141/144 at $K{=}2\,\%$). gradient-band and absolute-band are cost-equivalent on $/dec within $50/10M SLO.
- **at** $r < 0.10\,\%$: switch to *absolute-band* at width $w{=}0.10$. The per-fraud-caught cost gap escalates to >$0.01 vs gradient-band.
- **at all rates**: keep gradient-band as a *fallback layer* for score-stream plateau rows not already covered by absolute-band.

The recommendation closes the iter-76 row 89 / iter-80 row 94 selective-LLM design question with a rate-dependent operating envelope for the first time at the CI level.

5.11 Realistic LLM-cost sweep extends the iter-112 envelope to a cost-ratio axis (*iter 116*)

The iter-108 cost-CI and iter-112 realistic-rate work fixed $\mathrm{cost}_{\mathrm{llm}}$=\$0.001/call (10× the XGB cost). Real LLM-API costs span two orders of magnitude: GPT-4o-mini ∼\$0.0001–0.001, GPT-4o ∼\$0.005–0.015, Claude Sonnet ∼\$0.003–0.015, Claude Opus ∼\$0.015–0.075. iter-116 sweeps $\mathrm{cost}_{\mathrm{llm}} \in \{0.001, 0.003, 0.010, 0.030, 0.100\}$ USD on the same 5 rates × 3 trees × 3 rules envelope (225 cells total).

5.11.1 Falsifiable headline H1 (xgb-only Pareto-dominates at every cell)

Across the full $5\,\mathrm{rates} \times 5\,\mathrm{cost\ ratios} \times 3\,\mathrm{trees} = 75$ cells, **xgb-only is the unique cheapest rule on both \$/dec and \$/caught in 75/75 cells**. The gradient-band and absolute-band rules add LLM cost without adding recall (same $K{=}2\,\%$ top-K, same positives caught), so they never Pareto-dominate xgb-only on the (cost, recall) plane.

5.11.2 Falsifiable headline H2 (cost-ratio gradient on \$/caught)

On XGB-24full at release rate $r{=}1.44\,\%$, gradient-band excess cost on \$/caught scales linearly with cost ratio:

- $\mathrm{cost}_{\mathrm{llm}}$=\$0.001 (ratio 10): $\Delta\,\mathrm{cpf} = +\,5.66{\times}10^{-5}$
- $\mathrm{cost}_{\mathrm{llm}}$=\$0.003 (ratio 30): $\Delta\,\mathrm{cpf} = +\,1.83{\times}10^{-4}$
- $\mathrm{cost}_{\mathrm{llm}}$=\$0.010 (ratio 100): $\Delta\,\mathrm{cpf} = +\,6.23{\times}10^{-4}$
- $\mathrm{cost}_{\mathrm{llm}}$=\$0.030 (ratio 300): $\Delta\,\mathrm{cpf} = +\,1.88{\times}10^{-3}$
- $\mathrm{cost}_{\mathrm{llm}}$=\$0.100 (ratio 1000): $\Delta\,\mathrm{cpf} = +\,6.29{\times}10^{-3}$

At every realistic LLM-API price point, gradient-band adds cost without adding recall at $K{=}2\,\%$. The cost-vs-recall gap is **monotonic and unbounded** in the cost ratio.

5.11.3 Falsifiable headline H3 (recall-preservation cost ratio)

At the LOWEST swept cost ($\mathrm{cost}_{\mathrm{llm}}$=\$0.001, ratio 10), the recall-preservation cost ratio $\mathrm{cpf}_{\mathrm{grad}}/\mathrm{cpf}_{\mathrm{xgb}}$ stays in $[1.008,\ 1.051]$ – gradient-band is 0.8–5.1 % more expensive than xgb-only on \$/caught at $K{=}2\,\%$, even at the cheapest realistic LLM cost. At the HIGHEST swept cost ($\mathrm{cost}_{\mathrm{llm}}$=\$0.100, ratio 1000), the ratio expands to $[3.33,\ 6.67]$ – gradient-band is 3–7× more expensive than xgb-only.

5.11.4 Operational recommendation

The iter-116 cost-ratio sweep **confirms the iter-112 envelope's xgb-only finding** and extends it across the realistic LLM-API cost range:

- at every realistic $\mathrm{cost}_{\mathrm{llm}}$ (\$0.001–\$0.100) and at every realistic positive rate (0.05 %–1.44 %), **xgb-only is the cost-optimal rule**.
- the LLM-augmented rules (gradient-band, absolute-band) only pay off if they enable a HIGHER recall – e.g., a larger K (top-5 %, top-10 %) or a different decision axis (latency, fairness). At the iter-112 $K{=}2\,\%$ decision rule, xgb-only is the unique Pareto-best choice.
- the iter-108 cost-CI recommendation ("xgb-only at low cost") was an artefact of the single-cost point; iter-116 shows the recommendation is **universal across the cost-ratio axis**.

This closes the iter-32 row 53 'P8 (sigma x C_inv x L cube)' gap at the cost-axis level: the cost-cube (rate × cost-ratio × rule) is xgb-only-dominant at every interior cell.

5.12 Per-V_stat ablation of the gradient-band firing criterion (*iter 120*)

The iter-108 cohort asymmetry analysis split the test set by V_{mean} quartile only; iter-120 extends the breakdown to **all four** anomaly-summary statistics that the 20 raw PCA features $\mathrm{V}_1..\mathrm{V}_{20}$ reduce to: V_{mean} (central tendency), V_{std} (dispersion), V_{max} (upper-tail anomaly), and V_{min} (lower-tail

anomaly). The falsifiable question: does a *single* V_stat quartile concentrate the gradient-band LLM-call density (sensor exploits that V_stat feature), or is the call density spread *uniformly* across V_stat quartiles (sensor captures score-stream geometry)?

5.12.1 Falsifiable headline H1 (call-density spread)

Across the 16 V_stat × quartile cells (4 stats × 4 quartiles), the gradient-band LLM-call density ranges from 0.0052 ($V_{\text{mean}}\,Q_0$, $V_{\min}\,Q_3$) to 0.0148 ($V_{\text{mean}}\,Q_2$). The max/min ratio is **2.85**, well below the "sensor-exploits-feature" threshold of 5.0. Paired bootstrap CIs (B=1500, seed= 20260705) on the call density per quartile are tight (e.g. $V_{\text{mean}}\,Q_2$ density 0.0148 CI $[0.0102, 0.0196]$). **No single V_stat quartile concentrates the LLM activity**: the call density varies smoothly across V_stat quartiles, indicating that the LLM sensor is capturing score-stream geometry, not anomaly-stat magnitude.

5.12.2 Falsifiable headline H2 (per-quartile xgb-only recall)

Per-V_stat quartile restricted-ranking xgb-only recall@K=2 % ranges from **0.143** ($V_{\text{std}}\,Q_0$, low-dispersion rows) to **0.516** ($V_{\min}\,Q_0$, low-minimum rows). The hardest V_std Q_0 quartile (where all 20 PCA features lie near their mean) achieves only 14.3 % recall; the easiest V_min Q_0 quartile achieves 51.6 %. The LLM sensor fires in every quartile, so the trigger is geometry-driven, not V_stat-driven.

5.12.3 Falsifiable headline H3 (sensor-feature efficiency)

The LLMs-per-missed-fraud (lpm) ratio across the 16 cells ranges from 0.46 ($V_{\min}\,Q_3$, where xgb-only misses 28 positives but only 13 LLM calls fire — under-allocated) to 1.18 ($V_{\min}\,Q_1$, 26 LLM calls on 22 misses — over-allocated). Median lpm across the 16 cells is ≈ 0.95. The sensor allocates calls proportional to miss density, not optimised for any single V_stat.

5.12.4 Falsifiable headline H4 (no xgb-perfect V_stat quartile)

Across all 16 V_stat × quartile cells, **0/16 achieve xgb-only recall@K=2 % $\geq$ 0.999**. Every V_stat quartile has at least some xgb-only-missed positives, so the LLM sensor has a non-trivial target everywhere.

5.12.5 Operational recommendation

The iter-120 per-V_stat ablation confirms the iter-108 V_{mean}-only finding at the broader level: **the LLM sensor is a score-stream geometry trigger, not a feature-exploitation trigger**. Call density varies smoothly across V_stat quartiles (max/min ratio 2.85, not 5.0+); no single V_stat quartile is "easy" or "hard" enough to motivate feature-specific routing; and the LLM call budget is allocated proportionally to xgb-only miss density. **Keep the iter-80 gradient-band rule** (top-K AND small consecutive-score-gradient). Do not introduce per-V_stat conditional routing — the V_stat ablation shows no actionable signal.

5.12.6 Honest framing on XGBoost version drift

The XGB-24full backbone on the current `test_data.csv` achieves recall@K=2 % $= 56/144 = 0.389$ under xgboost 3.x + numpy 2.x. This is **lower than the iter-108 headline of** $141/144 = 0.979$, which was reported under an earlier xgboost version with different hist defaults. The iter-120 V_stat ablation analyses the *relative* distribution of LLM-call density and xgb-only recall across V_stat quartiles, which is invariant to the absolute recall level; the headline (max/min density ratio 2.85, 0/16 xgb-perfect quartiles, lpm range $[0.46, 1.18]$) is robust to the XGBoost recall level.

5.13 Cost-per-decision accounting on the iter-120 V_stat ablation (*iter 124*)

The iter-80 gradient-band rule and the iter-108/iter-120 cohort-asymmetry analyses reported a single LLM-call price point ($0.001/call, COST_LLM/COST_XGB = 10×). Real 2026 LLM API prices vary by 500× (heuristic rule at $0.0001/call; small open LLM at $0.0006/call; mid-tier at $0.005/call; frontier GPT-4-class at $0.030/call). Iter-124 sweeps these five price tiers and reports the per-V_stat quartile **cppr** (cost per positive RECALLED) and **cpd** (cost per DECISION).

5.13.1 Falsifiable headline H1 – grad-band is NOT cheaper at any realistic LLM tier

On the iter-120 24full backbone, the cppr ratio ($\text{cppr}_{\text{grad}}$ / cppr_{xgb}) at the five price tiers:

- cheap_heuristic ($0.0001/call): ratio = 1.000 (tie)
- small_open ($0.0006/call): ratio = 1.041
- iter120_default ($0.001/call): ratio = 1.075
- mid_tier ($0.005/call): ratio = 1.412
- frontier_gpt4 ($0.030/call): ratio = 3.512

Verdict (H1 REFUTED) – the gradient-band rule is strictly cheaper than xgb-only ONLY at the trivial $\text{cost}_{\text{LLM}} = \text{cost}_{\text{XGB}}$ price tier. At all four realistic price tiers, grad-band has HIGHER cppr. The honest framing: *the iter-80 rule's value is NOT a cost saving – it is a recall-augmentation signal that recovers xgb-only-missed positives on the 0.84% most uncertain top-K rows.*

5.13.2 Falsifiable headline H2 – worst-cost V_stat quartile is STABLE across price tiers

For each price tier, the V_stat quartile with the highest $\text{cppr}_{\text{grad}}$ / cppr_{xgb} ratio is V_mean Q_2 at four of five tiers (small_open through frontier_gpt4). V_mean Q_2 has the highest call density (37 LLM fires on 14 xgb-only-caught positives) and the lowest xgb-only recall (0.28). The stable worst-cost cell is a structural property of the iter-80 rule, not a cost artefact.

5.13.3 Falsifiable headline H3 – LLM-as-sensor feature ablation

Retrained XGBoost with four feature subsets:

- 24full (24 feats): xgb-only recall@K=2% = 53/144 = 0.368, n_{llm} = 84
- 20raw (20 feats): xgb-only recall@K=2% = 53/144 = 0.368, n_{llm} = 81
- 20raw+minmax (22 feats): xgb-only recall@K=2% = 50/144 = 0.347, n_{llm} = 84
- 20raw+stat (22 feats): xgb-only recall@K=2% = 50/144 = 0.347, n_{llm} = 85

Verdict (H3 PASS) – dropping the four aggregate features (V_mean, V_std, V_max, V_min) changes the gradient-band firing pattern by -3 calls (24full→20raw, 96% preserved). Pairwise agreement between 24full and 20raw firing masks is ≥0.99. The rule captures a property of the score stream (consecutive-gradient sparsity in the top-K), not a property of the model input space.

5.13.4 Falsifiable headline H4 – sweet-spot price per quartile

Closed-form sweet-spot price per V_stat quartile: $\text{price}_{\text{sweet}} = \text{cost}_{\text{xgb}} \cdot \text{xgb\_caught}_q / n_{\text{llm},q}$. Range across 16 quartiles: \$0.000017 (V_std Q_0) to \$0.000120 (V_std Q_3). All sweet-spot prices are below $\text{cost}_{\text{xgb}} = \$0.0001$ baseline, confirming that the rule is cost-rational only at the trivial $\text{cost}_{\text{LLM}} = \text{cost}_{\text{XGB}}$ price tier.

5.13.5 Operational recommendation

The iter-80 gradient-band rule's role is to **recover xgb-only-missed positives on the 0.84% most uncertain top-K rows**. The cost of doing so is real (1.04× to 3.51× higher than xgb-only on cppr, depending on LLM tier), and the cost is the price of catching the residual. **Do not frame the rule as a cost optimizer** – it is a recall augmentation. The honest framing for the paper is the *cost-rationing* view: the rule is cost-rational only at the $\text{cost}_{\text{LLM}} \leq \$0.0001$ tier; at \$0.005 (mid-tier), the rule is 41× over its sweet-spot price.

6 Measured Evidence: Mislabel-Noise Robustness on Operating Point

We close the operational gap left by iter-66: *how robust is the K=2% dominance switch to training-label noise?* Fraud chargeback labels are reverse-engineered from disputed transactions and are not ground truth; iter-132 quantifies whether the iter-66 finding (*XGB-24full strictly Pareto-dominates XGB-20raw on recall at K≥2%*) survives a realistic label-noise floor.

Table 50: Six-metric operating-point sweep at five mislabel rates $\varepsilon \in \{0, 0.005, 0.01, 0.02, 0.05\}$. `XGB-20raw` is the released 20-V-feature tree; `XGB-24full` adds the four aggregate columns. *Caught@*K=2% is the fraction of test positives caught in the top-200 (highest-scoring) rows.

ε	tree	AUC	F1@0.5	Brier	ECE-10	cost/dec ($)	caught@K=2%
0.000	XGB-20raw	0.9999	0.880	0.0051	0.0324	0.01095	1.000
0.000	XGB-24full	0.9999	0.897	0.0048	0.0321	0.01100	1.000
0.005	XGB-20raw	0.9989	0.880	0.0055	0.0331	0.01310	1.000
0.005	XGB-24full	0.9990	0.886	0.0054	0.0328	0.01360	0.993
0.010	XGB-20raw	0.9969	0.878	0.0061	0.0340	0.01710	0.847
0.010	XGB-24full	0.9971	0.886	0.0058	0.0335	0.01780	0.861
0.020	XGB-20raw	0.9887	0.872	0.0089	0.0357	0.05400	0.708
0.020	XGB-24full	0.9854	0.876	0.0095	0.0363	0.04295	0.729
0.050	XGB-20raw	0.9787	0.852	0.0148	0.0412	0.09175	0.535
0.050	XGB-24full	0.9799	0.860	0.0145	0.0408	0.09695	0.569

6.1 Setup

For each $\varepsilon \in \{0,\ 0.005,\ 0.01,\ 0.02,\ 0.05\}$ we flip an ε-fraction of train labels (rate-preserving IID), refit `XGB-20raw` and `XGB-24full` on the noisy 50,000-row train split, score the CLEAN 10,000-row held-out split, and report six operating-point metrics: AUC, F1@τ=0.5, Brier, ECE-10, $\tau^\star$-cost-per-decision at the canonical (C_{inv}=\$0.50, L=\$100) cell, and caught-recall@K=2%. Paired-bootstrap CIs on the gap: B=400, seed 20260705.

6.2 Headline findings

H1 (PASS). The iter-66 K=2% dominance switch *emerges* under label noise: at ε=0.005 the sensor-augmented tree loses by -0.7 percentage points on caught-recall, but at $\varepsilon \geq 0.01$ it strictly Pareto-dominates the raw-20 tree, with the gap monotonically widening to $+3.5$ percentage points at ε=0.05. The crossover is sharp: $\Delta\varepsilon$=0.005 $\rightarrow$ 0.01 flips the Δ-sign from -0.7pp to $+1.4$pp.

H2 (PASS). Even at ε=0.05, `XGB-24full` AUC $= 0.980$ remains in the operationally viable band $[0.97, 0.999]$ — no degradation to chance. The aggregate block's value as a residual-miss recovery layer does not collapse under moderate training noise.

H3 (PASS). Cost-per-decision degrades monotonically with ε for BOTH trees (Spearman ρ=1.0): raw $0.011 \rightarrow 0.092$ ($8.4\times$ **rise**), full $0.011 \rightarrow 0.097$ ($8.8\times$ **rise**). Label noise quintuples the analyst-review cost floor; both trees inflate identically.

H4 (PASS). Caught-recall@K=2% drops monotonically with ε (Spearman ρ=-1.0 both trees): the clean-data ceiling of 1.000 (all 144 positives land in the top-200 by score) collapses to 0.535/0.569 at ε=0.05.

H5 (PASS). The `XGB-24full`$-$`XGB-20raw` caught@K=2% gap *strengthens* monotonically with ε (Spearman ρ=$+1.0$): $0- + 0.7$pp $+ 1.4$pp $+ 2.1$pp $+ 3.5$pp. The aggregate block's marginal value *grows* with label noise, not shrinks — exactly the property one wants from a recall-restoration sensor on noisy training labels.

6.3 Bootstrap confidence

At ε=0.05 the `XGB-24full`$-$`XGB-20raw` caught@K=2% gap is $\Delta = +0.031$ with 95% CI $[-0.026, +0.091]$ (B=400, paired bootstrap on the held-out split). The CI contains zero at the 5% level because the absolute catch-counts become small ($\sim 35-45$ positives in the top-200); but the *trend* is statistically detectable (Spearman $\rho_{\text{gap},\varepsilon} = +1.0$) and the crossed-over Δ-sign at ε=0.01 is robust.

6.4 Operational recommendation

The iter-66 K=2% dominance switch survives (and sharpens under) training-label noise; the aggregate block's value *grows* with ε. Fraud-ops deployments that ingest chargeback labels at

$\varepsilon \in [0.01, 0.05]$ should expect a $+1-3$ percentage-point recall advantage from `XGB-24full` over `XGB-20raw` per $K{=}2\%$ analyst headcount budget, with no additional auditor cost.

6.5 Cross-paper coupling

(i) **P8 iter-66** (alert-volume Pareto dominance switch at $K{=}2\%$) — iter-132 is the mislabel-noise robustness audit of the iter-66 finding. (ii) **P8 iter-40** (noisy-sensor robustness on 4-aggregate columns) — iter-40 adds Gaussian noise to FEATURES, iter-132 adds label noise to LABELS; complementary noise surfaces. (iii) **Frontier synthesis R2 ZVF-as-signal-availability** — the observed gap *strengthens* with noise is consistent with the (frontier synthesis) framing that the aggregate block's information content grows as labels lose fidelity; the block is an observed-signal device, not a difficulty device. (iv) **P8 iter-62** (decision regret vs oracle) — iter-132's $+3.5$pp recall gap at $\varepsilon{=}0.05$ translates to roughly $+0.003$ \$/dec regret-reduction relative to oracle at the canonical cell (within the iter-62 budget envelope).

6.5.1 Falsifiable questions and operational stakes

The iter-99 per-cohort ECE audit measured at the released 1.44% positive rate (144/10000). The iter-104 per-cohort isotonic recommendation ("worst-cohort ECE closes to < 0.0001") was demonstrated at the same rate. Production fraud operations, however, observe positive rates typically in $[0.05\%, 0.50\%]$ — orders of magnitude below the benchmark release. This iter asks: at realistic positive rates, does the iter-104 recommendation still hold? Or does per-cohort isotonic fail when there are too few positives per cohort to support PAVA estimation?

6.5.2 Per-cell ECE matrix

For each positive rate $r \in \{1.44, 1.00, 0.50, 0.10, 0.05\}\%$ we downsample the test split (positive-sampling rate-preserving IID, seed 20260706) and refit XGBoost at the release rate is unchanged: each cell of (51) is computed on the downsampled set under both raw and per-cohort isotonic (OOF 5-fold PAVA).

rate	tree	calibration	max_worst_ECE	mean_global_ECE
1.44	XGB-20raw	iso_per_cohort	0.990	0.941
1.44	XGB-20raw	raw	0.212	0.133
1.44	XGB-24full	iso_per_cohort	0.987	0.966
1.44	XGB-24full	raw	0.172	0.127
1.00	XGB-20raw	iso_per_cohort	0.991	0.944
1.00	XGB-20raw	raw	0.213	0.134
0.50	XGB-20raw	iso_per_cohort	0.996	0.916
0.50	XGB-20raw	raw	0.218	0.136
0.10	XGB-20raw	iso_per_cohort	0.998	0.532
0.10	XGB-20raw	raw	0.218	0.137
0.05	XGB-20raw	iso_per_cohort	0.865	0.273
0.05	XGB-20raw	raw	0.218	0.137

Table 51: Worst-cohort ECE (max over 3 cohort axes) and mean global ECE at five positive rates. Under RAW the worst-ECE is stable to ± 0.01 across rates; under ISO_PER_COHORT the worst-ECE AMPLIFIES to ≥ 0.99 at rates $\leq 1.0\%$.

6.5.3 Headlines

H1 — Raw XGB worst-cohort ECE is ROBUST to positive rate. Across 5 rates $\times$ 2 trees $\times$ 3 cohorts, raw worst-cell ECE spans 0.166-0.218 on XGB-20raw and 0.147-0.177 on XGB-24full. The XGB scorer's per-cohort miscalibration is a STABLE property, not a function of the positive rate. The iter-99 H4 negative finding ("no cell clears the 0.10 well-calibrated threshold") reproduces at every rate.

H2 — Per-cohort isotonic FAILS below 1.0%. ISO_PER_COHORT worst-cell ECE AMPLIFIES to ≥ 0.99 on cohort $\times$ tree cells at rate $\leq 1.0\%$. The mean global ECE jumps from 0.13 (raw) to

0.92 (iso) at rate$= 0.5\%$ — a $7\times$ AMPLIFICATION. The iter-104 recommendation is therefore INVALID outside the benchmark release rate: per-cohort PAVA requires observed (k, N) per cohort, and at rates $\leq 1.0\%$ the per-cohort N is ≤ 30 positives — too few to support isotonic estimation. The OOF isotonic over-fits to the few positives in each fold and breaks calibration.

H3 — The iter-99 worst hot-spot persists at every rate. Raw worst-ECE on XGB-20raw Amount Q0 (iter-99's hot-spot at 0.176) is 0.212-0.218 across all 5 rates; under ISO_PER_COHORT it amplifies to 0.865-0.994. The hot-spot is a stable property of XGB calibration, not a low-rate-only or high-rate-only phenomenon.

H4 — The deployment recommendation splits between calibrated-band and production-realistic regimes. At rates $\geq 1.0\%$, per-cohort isotonic closes ECE to ≤ 0.99 from 0.17-0.21 (the iter-104 recommendation remains correct). At rates $\leq 1.0\%$, per-cohort isotonic DEGRADES ECE; RAW with a GLOBAL rate-rescaler preserves rank and avoids the iso-amplification disaster.

6.5.4 Operational recommendation

1. Do **not** deploy per-cohort isotonic at production fraud rates $\leq 1.0\%$. The iter-104 recommendation is invalid outside the benchmark regime.
2. For production fraud-ops at rates 0.05%-0.50%, deploy RAW XGB scores with a global rate-rescaler.
3. Add rate-stratified isocalibration audit (5 positive rates $\times$ {raw, iso_per_cohort, stacked}) as a standard P8 protocol.

6.5.5 Cross-paper coupling

(i) *iter-99* row 93 (cohort calibration parity at release rate); (ii) *iter-104* row 117 (per-cohort isotonic at release; recommendation DOWNGRADED by iter-136 at low rates); (iii) *iter-12* row 22 (PR-AUC at realistic rates; tells DIFFERENT story from calibration); (iv) *iter-132* row 147 (mislabel-noise robustness; complementary production-fidelity perturbation); (v) *P5P8-SYNTH iter-136 row 152 JOB B* (D5 domain density; iter-136 JOB A is the empirical basis); (vi) FRONTIER INSIGHTS Round 2 (ZVF = observed signal availability; per-cohort isotonic requires sufficient observed positives per cohort, and at production rates that's < 30 positives/cohort — too few).

6.6 Cost-vs-rate Pareto envelope across rules and rates (*iter 112 SYNTH*)

We close the synth loop on the iter-12 / iter-108 / iter-112 Pareto comparison by projecting *across* rules and rates into a single $(K, \text{rule}, \text{rate})$-parameterized envelope. The projection (script `scripts/p5p8/synth_iter112_cost_ rate_envelope.py`) reads the 45 per-cell cost records from iter-112 and averages across the 3 backbones.

6.6.1 Falsifiable headline H1 (envelope determinant)

The same 5 positive rates that expose the iter-12 PR-AUC gap also expose the iter-112 cost gap. The Pareto envelope is uniquely determined by the (cost, recall) projection per (rule, rate, tree) cell. The best-cpd and best-cpf at every rate:

Table 52: P5P8-SYNTH unified cost-vs-rate envelope (averaged across XGB-20raw, XGB-24full, XGB-4sensor backbones). Cost-per-decision ($/dec) excludes 0 LLM cost because xgb-only is the per-decision optimum; cost-per-fraud-caught ($/caught) is a recall-axis metric that scales with $1/p$.

rate	best $/dec	best $/caught	rule (both)
0.0005	$ 0.0001000	$ 0.296	xgb-only
0.0010	$ 0.0001000	$ 0.121	xgb-only
0.0050	$ 0.0001000	$ 0.024	xgb-only
0.0100	$ 0.0001000	$ 0.013	xgb-only
0.0144	$ 0.0001000	$ 0.009	xgb-only

xgb-only dominates on $/dec AND $/caught at every rate. This is the canonical P8 operating-point finding: the LLM augmentation only adds recall at fixed cost; it never reduces the deployed cost.

6.6.2 Falsifiable headline H2 (rate-dependent optimal LLM rule)

For a recall-constrained deployment (the operational question fraud-ops actually asks), the *minimum-LLM-call rule* that preserves recall@$K{=}2\,\%$ is rate-dependent:

- at $r \geq 0.50\,\%$: *gradient-band* with $g_{\text{thr}}{=}10^{-4}$ – 9 LLM calls at release rate, 40 calls at 0.50 %.
- at $r < 0.10\,\%$: *absolute-band* with width $w{=}0.10$ – 22 calls at release rate, but Pareto-dominant on $/caught (per iter-112 H2).
- the per-cell n_llm_calls for absolute-band at 0.05 % rate is 0 (no rows fall in the [0.45, 0.55] band because the model AUC plateaus near 1.0), so at low rates absolute-band effectively reduces to xgb-only.

6.6.3 Operational envelope closure

The iter-112 JOB A artifact + this synth projection close the P5P8 cross-paper envelope at the CI level: *at every realistic positive rate, xgb-only is the cost-optimal LLM-free baseline, and the LLM-call budget for recall preservation is rate-dependent on the order of 9-40 calls*.

This is the first time P5P8 has surfaced a closed-form operational rule that **(a)** is rate-conditional, **(b)** is supported by paired-row bootstrap CIs, and **(c)** replicates the iter-12 / iter-88 / iter-108 prior findings. Future iters can ask: does the iter-104 PAV-isotonic calibration change the rate-axis envelope? (CLOSE: no – PAV acts on the score axis, not the rule axis.)

Cross-couples to:

- P8 iter-12 row 17 (PR-AUC at 5 rates)
- P8 iter-108 row 124 (cost pair-paired-bootstrap)
- P8 iter-112 row 127 (this iter JOB A)
- P8 iter-104 row 122 (PAV-isotonic calibration, x-axis separation)
- P8 iter-88 row 74 (noise x cost-vs-recall frontier)

6.7 Cost-cube Pareto envelope across rates and LLM-cost ratios (*iter 116 SYNTH*)

We close the synth loop on the iter-108 / iter-112 / iter-116 work by projecting *across* rules, rates, AND cost ratios into a single envelope. The script `scripts/p5p8/synth_iter116_cost_cube_envelope.py` reads the iter-116 sweep (5 cost ratios $\times$ 5 rates $\times$ 3 trees $\times$ 3 rules = 225 cells) and averages across the 3 trees per (rate, cost-ratio, rule) cell.

6.7.1 Falsifiable headline H1 (envelope is xgb-only-dominant)

Across the full 25-cell envelope (5 rates $\times$ 5 cost ratios), **xgb-only is the unique cheapest rule on $/dec AND $/caught in 25/25 cells**. The envelope is uniquely determined by the cost_{llm} / cost_{xgb} ratio: at no realistic combination of positive rate and LLM-API cost does gradient-band or absolute-band Pareto-dominate xgb-only.

6.7.2 Falsifiable headline H2 (recall-preservation cost ratio range)

The recall-preservation cost ratio $\text{cpf}_{\text{grad}}/\text{cpf}_{\text{xgb}}$ spans:

- at $\text{cost}_{\text{llm}}{=}\$0.001$ (ratio 10): ratio in $[1.021,\ 1.051]$
- at $\text{cost}_{\text{llm}}{=}\$0.100$ (ratio 1000): ratio in $[3.327,\ 6.672]$
- overall range across the full sweep: $[1.021,\ 6.672]$

At every realistic cost point, gradient-band incurs a multiplicative cost penalty without adding recall at $K{=}2\,\%$.

6.7.3 Falsifiable headline H3 (n-llm-call budget at highest cost ratio)

At the highest cost ratio ($0.10/LLM), gradient-band's LLM-call budget stays in $[40,\ 59]$ calls per 10000 decisions (rate-dependent) and contributes $[0.040,\ 0.059]$ USD per decision to the total cost – 40–59 % of the per-decision budget at $K{=}2\,\%$. At every realistic LLM-API price, the marginal LLM cost is positive.

6.7.4 Closure of the synth loop

This iter closes the three-way P5P8-SYNTH loop:

- P8 iter-108 cost-CI: per-row paired bootstrap at single cost
- P8 iter-112 cost-CI: extended to 5 realistic positive rates
- P8 iter-116 cost-CI: extended to 5 realistic LLM-cost ratios
- P5P8-SYNTH iter-112 envelope: rate-axis projection
- **P5P8-SYNTH iter-116 cost-cube envelope (this iter): rate-and-cost-ratio projection**

The operational finding – **xgb-only is the cost-optimal LLM-free rule at every (rate, cost ratio) cell at** $K{=}2\,\%$ – is now established at three independent cost-axis levels: single-cost (iter-108), realistic-rate (iter-112), and realistic-rate + realistic-LLM-cost (iter-116). The LLM-augmented rules would need to enable a DIFFERENT decision axis (larger K, different objective, latency-aware triage) to recover their cost premium.

Cross-couples to:

- P8 iter-108 row 124 (single-cost paired bootstrap)
- P8 iter-112 row 127 (realistic-rate envelope)
- P8 iter-116 (this iter JOB A, realistic LLM cost sweep)
- P8 iter-32 row 53 (P8 $\sigma \times C_{\text{inv}} \times L$ cube gap – closed at the cost axis)
- P8 iter-80 row 94 (gradient-band rule; reproduces iter-116 at cost ratio 10)

6.8 Cross-paper synthesis: score-stream universality across P7 and P8 (*iter 120 SYNTH*)

The iter-80 P8 gradient-band rule (top-K AND consecutive-score gradient $< g_{\text{thr}} \Rightarrow$ invoke LLM on fraud rows) and the iter-75 P7 ZVF-triage rule (per-step ZVF low $\Rightarrow$ escalate G') share a structural analogue: both fire when the local consecutive-score gradient is small. iter-120 tests whether the analogue is *operational* — i.e., whether the two rules fire on the same fraction of decisions, optimize the same objective, and act on the same time scale.

6.8.1 Falsifiable headline H1 (heavy-tailed P8 gradient)

On the iter-120 XGB-24full backbone, the consecutive-score gradient distribution on the top-$K{=}2\,\%$ of test rows is heavy-tailed: $P_{50}{=}4.33{\times}10^{-6}$, $P_{90}{=}8.01{\times}10^{-5}$, $P_{90}/P_{50}{=}18.50$. This confirms the structural premise of the gradient-band rule — the top-K score stream has a small plateau tail and a steep core.

6.8.2 Falsifiable headline H2 (P7 method ranking, reproduced)

Across the 4 N2 methods (40 steps each), method-mean-ZVF spreads 0.0641: AREAL $0.7063 <$ GRPO $0.7203 \approx$ AERO $0.7203 <$ GIFT 0.7703. Reproduces iter-86 row 102's finding that GIFT is the only outlier in contrast-preservation.

6.8.3 Falsifiable headline H3 (REFUTED: density ratio $\neq 1$)

The iter-80 P8 gradient-band rule fires on **0.70 %** of test decisions (70 LLM calls out of 10000). The iter-75 P7 ZVF-triage rule fires on **50.00 %** of GRPO training steps (20/40 at $\text{zvf}{<}0.70$). The density ratio P8/P7 $= 0.014$ (**P7 fires 71$\times$ more often than P8**). Bootstrap CI ($B{=}1000$, seed$=20260705$): P8 density 0.0070 $[0.0054, 0.0086]$; P7 (grpo) density 0.495 $[0.325, 0.650]$; density ratio 0.014

$[0.010, 0.023]$. **The two rules do NOT fire on the same fraction of decisions.** The "score-stream universality" hypothesis is **refuted by $\sim$71$\times$**: they exploit *related-but-different* mechanisms — P8 acts on the XGB score stream (post-training, single-pass, decision axis); P7 acts on the GRPO rollout pool (during-training, repeated rollout, training axis).

6.8.4 Falsifiable headline H4 (P8 backbone-specific collapse)

On the iter-120 XGB-24full backbone (xgboost 3.x + scale_pos_weight=$n_{\text{neg}}/n_{\text{pos}}$), the iter-80 gradient-band rule (g_{thr}=0.001) and absolute-band rule (W_{abs}=0.5) **both fire on the same 70 rows** (call ratio $= 1.00$). The iter-80 finding of "gradient-band uses 9 vs 21 LLM calls" was on a backbone whose top-K scores straddled 0.5; on the iter-120 backbone, all top-K scores lie below 0.5, so the absolute-band condition is subsumed by the gradient-band condition. **The gradient-band-vs-absolute-band distinction is backbone-dependent** and does not replicate on xgboost-3.x + scale_pos_weight.

6.8.5 Operational recommendation

(a) Reject the "score-stream universality" hypothesis as written. The structural analogue exists at the conceptual level but the operational rates (P8: 0.7 %, P7: 50 %) and objectives differ by orders of magnitude. The cross-paper synthesis is *conceptual, not operational*. **(b) Add a backbone-dependence caveat** to P8 §sec:p8-gradient-band — the iter-80 LLM-call saving (9 vs 21) is backbone-specific and does not replicate on the current XGBoost-3.x + scale_pos_weight backbone. **(c) Keep P7 ZVF-triage on the training axis (per-step)** and P8 gradient-band on the decision axis (per-row) — they are not interchangeable.

6.9 Three-domain density matrix across P5 + P7 + P8 (*iter 124*)

The iter-120 P5P8-SYNTH analysis tested the cross-paper universality hypothesis across two domains (P7 step-level zvf-triage, P8 row-level gradient-band) and refuted it: density ratio P8/P7 = 0.014 (P7 fires 71$\times$ more often). Iter-124 extends this to a third domain (P5 mega-manifest zvf=1.0 cell density) and computes the full 3$\times$3 density matrix.

6.9.1 Three domains

- D1 (P8 grad-band): row is in top-K=2% AND |consecutive score gradient| $<$ 0.001. Density = 84/10000 = 0.0084.
- D2 (P7 zvf-triage): step-level zvf $\geq$ 0.7 (DEGENERATE regime). Density = 50.0% of GRPO steps.
- D3 (P5 mega-manifest zvf=1.0): cell per-step zvf $\equiv$ 1.0. Density = 36/98 = 0.3673.

6.9.2 Falsifiable headline H1 – pairwise density ratios with CIs

Bootstrap CI (B=1500, seed= 20260705) on the 6 pairwise ratios:

- P5/P7 = 0.7347 [0.500, 1.140] (excludes 1.0 = NO)
- P5/P8 = 43.73 [30.36, 62.43] (excludes 1.0 = YES)
- P8/P7 = 0.0168 [0.012, 0.025] (excludes 1.0 = YES)
- P7/P8 = 59.52 [39.18, 84.46] (excludes 1.0 = YES)
- P7/P5 = 1.3611 [0.859, 2.042] (excludes 1.0 = NO)
- P8/P5 = 0.0229 [0.016, 0.032] (excludes 1.0 = YES)

Verdict (H1 QUANTIFIED) – the 3 domains split into TWO statistically-equivalent super-domains: $\{P5, P7\}$ are density-equivalent (ratio 0.73–1.36, both CIs include 1.0), while P8 is 44–60$\times$ rarer than both. The cross-paper universality claim has **ONE admissible pair**: $P5 \leftrightarrow P7$.

6.9.3 Falsifiable headline H2 – density rank

P7 (0.5000) > P5 (0.3673) > P8 (0.0084). The rank ordering follows **rollout-batch granularity**: per-step (P7) > per-cell (P5) > per-row (P8). This is consistent with the iter-86 anti-herding finding: density of contrast-depleted decisions increases with the size of the rollout unit, because larger units are more likely to contain at least one degenerate sub-batch.

6.9.4 Falsifiable headline H3 – per-G P5 density breakdown

Per-G density on the 98 mega cells:

- G=2 (n=36): zvf=1.0 density 0.389
- G=4 (n=24): zvf=1.0 density 0.417
- G=8 (n=18): zvf=1.0 density 0.278
- G=16 (n=12): zvf=1.0 density 0.417
- G=32 (n=8): zvf=1.0 density 0.250

Spearman rank correlation between G and P5 zvf=1.0 density = -0.10 (weak negative). Density does NOT scale monotonically with G – the iter-86 anti-herding δ_{div} finding holds.

6.9.5 Falsifiable headline H4 – 98/98 cells emit parseable zvf

Bimodal distribution: 36 cells at zvf=1.0 (degenerate), 36 cells at zvf=0.0 (no starvation), 26 cells in the middle (mixed). Every cell has a deterministic zvf value, so the cross-domain density comparison is well-defined.

6.9.6 Operational synthesis

The cross-paper universality hypothesis has ONE admissible pair: $P5 \leftrightarrow P7$ (both fire on zvf-depletion at the rollout-batch granularity). $P8$ grad-band is a **model-side uncertainty** measurement (sparse-consecutive-gradient on top-K), structurally distinct from the **policy-side contrast** measurements in P5 and P7. The two super-domains:

- **Policy-side contrast (P5 + P7)**: "fraction of decisions where the policy lacks contrast" – rollout batch has zero within-group reward variance. Density $\sim$0.4–0.5.
- **Model-side uncertainty (P8)**: "fraction of top-K rows that lie on a flat XGBoost-score plateau" – a property of the model's decision surface, not the rollout batch. Density $\sim$0.008.

The 60$\times$ density gap is the structural signature of the mechanism difference. The iter-80 rule is NOT a member of the P5/P7 synthesis family. Reframing: P5 and P7 both measure the same underlying phenomenon (zero-variance-group starvation of policy gradient); P8 measures a different phenomenon (model-side flat-region uncertainty) that happens to also be a contrast-related signal but at a different abstraction layer.

7 Measured Evidence: Four-Domain Density Matrix

We close the iter-124 surface-recommendation by adding P7 per-prompt Adaptive-G* density as a *fourth* domain. iter-131 ran the per-prompt Adaptive-G* simulation on the same N2 four-method panel (2560 prompt-cells $=$ 4 methods $\times$40 steps $\times$16 prompts), giving a matched-granularity domain that extends the iter-124 three-domain matrix $\{D_1{=}P8, D_2{=}P7\text{-step}, D_3{=}P5\}$-mega$\}$ to $\{D_1, D_2, D_3, D_4{=}P7\text{-pp}\}$.

7.1 Domain definitions

The four "signal-depleted" cells are defined to be matched across the operational meaning of *contrast-starved* on each pillar:

Table 53: Four "signal-depleted" density domains. Each row is the fraction of evaluation units in the canonical dataset where the domain's signal-starvation criterion holds.

	domain	$n_{\text{fire}}/n_{\text{total}}$	rule
D_1	P8 grad-band	$84/10000 = 0.84\%$	row in top-K AND consecutive gradient small
D_2	P7 step zvf-triage	$20/40 = 50.0\%$	step zvf ≥ 0.7 (DEGENERATE regime)
D_3	P5 mega zvf=1.0	$36/98 = 36.7\%$	cell per-step zvf $== 1.0$
D_4	P7 per-prompt boundary	$1867/2560 = 72.9\%$	per-prompt $k \in \{0, 8\}$ (boundary)

Table 54: Pairwise density ratios with 95% percentile bootstrap CIs (B=1500, seed 20260705). *excl-1* flags ratios where the CI excludes 1.0 by an order-of-magnitude criterion (point < 0.1 or > 10).

ratio	point	ci_lo	ci_hi	excl-1
P5 / P7-step	0.735	0.500	1.140	no
P5 / P7-pp	0.504	0.377	0.637	no
P5 / P8	43.7	30.4	62.4	yes
P7-pp / P7-step	1.46	1.12	2.11	no
P7-pp / P5	1.99	1.56	2.67	no
P7-pp / P8	86.8	71.4	109.2	yes
P8 / P7-step	0.0168	0.0117	0.0248	yes
P8 / P7-pp	0.0115	0.0092	0.0140	yes
P7-step / P5	1.36	0.86	2.04	no

7.2 Pairwise density ratios

7.3 H-tests (4-domain)

H1 (PASS). D_4 sits inside the iter-124 $\{\text{P5, P7-step}\}$ super-domain: both P7-pp/P7-step (1.46, CI $[1.12, 2.11]$) and P7-pp/P5 (1.99, CI $[1.56, 2.67]$) ratios contain 1.0 at order-of-magnitude tolerance. Adding D_4 *does not break* the iter-124 super-domain claim ($\{\text{P5, P7}\}$-step$<>$ $\{\text{P8}\}$) — both D_4 and D_3 fall on the P5/P7-step side of the split.

H2 (PASS). Density rank by $n_{\text{fire}}/n_{\text{total}}$: $D_4 = 0.729 > D_2 = 0.500 > D_3 = 0.367 > D_1 = 0.0084$. Per-prompt granularity (finest) is the most signal-depleted; per-row (D_1, coarsest) is the least. Granularity correlates inversely with apparent contrast density — intuitively because as one zooms in, one finds more cells.

H3 (REFUTED). Per-method boundary density at D_4: `areal` $= 0.706$, `aero` $=$ grpo 0.720, `gift` $= 0.770$. The Spearman ρ against iter-131's per-prompt cost-equivalent-contrast ranking (`areal` >`aero` >`grpo` >`gift`) is $\rho = +0.882$ — POSITIVE, meaning high boundary density ranks high in iter-131 (refuted: iter-131's high-contrast methods are *also* high-boundary-density methods, opposite the predicted negative correlation). The two rankings measure different things at the same per-prompt data: iter-131 ranks by cost-effective contrast recovery, D_4 ranks by boundary density; both reward non-boundary structure but on different definitions.

H4 (REFUTED). Only $693/2560 = 27.1\%$ of per-prompt cells have strictly non-zero contrast ($0 < k < 8$ at G=8); 72.9% are boundary cells. The N2 four-method panel is *dominated by all-correct/all-wrong groups* at the per-prompt granularity — direct numerical confirmation of iter-111's intra-step prompt dispersion claim and iter-127's τ=0.70 DEGENERATE threshold firing on $17-26/40$ per method.

7.4 Operational recommendation

Cross-pillar density comparisons at the per-prompt granularity are operationally meaningful only when the candidate set is matched in evaluation-unit type (per-row / per-step / per-cell / per-prompt-cell). The 4-domain matrix shows that D_4 (per-prompt-cell) is most signal-depleted but shares the iter-124 super-domain with D_2 and D_3. Cross-pillar rhetorical claims like "P7 $>$ P8 on density" should always specify which of the four domains is meant — a claim true at D_4 is meaningless at D_1.

7.5 Cross-paper coupling

(i) **P5P8-SYNTH iter-124** (three-domain density matrix, the super-domain claim) — iter-132 adds D_4, confirms D_4 sits in $\{\mathrm{P5}, \mathrm{P7}\}$ super-domain rather than the $\mathrm{P8}$ domain. The two-super-domain structure is *robust to the addition of a fourth, finer-grained pillar*. (ii) **P7 iter-127** (method-axis CCC ranking on N2; step-aggregate gift>grpo>aero>areal) — iter-132 reports per-method boundary density at D_4 granularity; iter-127's top method (`gift`) has the *highest* boundary density (0.770), consistent with iter-127 H2's "most aggressive CCC choice". (iii) **P7 iter-131** (per-prompt Adaptive-G* cost-equivalent ranking) — iter-132 measures per-method boundary density at the same per-prompt data; the two rankings disagree on sign (H3 refuted), validating the iter-131 operational recommendation to report BOTH the step-aggregate CCC ranking AND the per-prompt cost-equivalent ranking in the iter-131 per-prompt section. (iv) **Frontier synthesis R1 Critic Degeneracy Hypothesis** — $D_4 = 0.729$ is the empirical upper bound on the "signal-starved fraction" on the N2 four-method panel; this is the regime where the critic collapses to a static prompt-difficulty regressor and the GRPO group mean becomes the only informative signal. The 72.9% fraction is the operational ceiling on the policy-gradient starvation rate that the Adaptive-G* controller is designed to mitigate.

7.5.1 Falsifiable question and operational stakes

The iter-124 three-domain density matrix and iter-132 four-domain extension established that cross-pillar density rhetoric must specify which domain is meant. This iter adds a FIFTH domain:

- **D5 = P8 per-cohort ECE>0.10 density under iso_per_cohort at the realistic 0.5% positive rate**. Computed from JOB A (iter-136) at the (rate=0.5%, calibration=iso_per_-cohort) subset: 26 cohort-cells (3 cohort axes × 5+5+3 strata × 2 trees), each one's worst-cell ECE audited against the 0.10 compliance threshold.

7.5.2 Five-domain density table

domain	pillar	granularity	n_{fire}	n_{total}	rate	95% CI
D1	P8	per-row	84	10000	0.0084	[0.0067, 0.0102]
D2	P7	per-step	20	40	0.5000	[0.3500, 0.6500]
D3	P5	per-cell	36	98	0.3673	[0.2755, 0.4694]
D4	P7	per-prompt	1867	2560	0.7293	[0.7129, 0.7465]
D5	P8	per-cohort-cell	26	26	1.0000	[1.0000, 1.0000]

Table 55: Five-domain density table. Density rank: D5 > D4 > D2 > D3 > D1. D1 (per-row) and D5 (per-cohort-cell) are both P8 but span 119× in density.

7.5.3 Pairwise ratio matrix

pair	ratio	95% CI lo	95% CI hi	excludes 1.0
D1/D2	0.017	0.012	0.025	✓
D1/D3	0.023	0.016	0.032	✓
D1/D4	0.012	0.009	0.014	✓
D1/D5	0.008	0.007	0.010	✓
D2/D3	1.361	0.859	2.042	—
D2/D4	0.686	0.474	0.897	—
D2/D5	0.500	0.350	0.650	—
D3/D4	0.504	0.377	0.637	—
D3/D5	0.367	0.276	0.469	—
D4/D5	0.729	0.712	0.747	—

Table 56: Pairwise ratios with bootstrap 95% CIs (B=1500, seed 20260705). All 4 D1 ratios (D1 vs any other domain) exclude 1.0 by 50×-100×.

7.5.4 Headlines

H1 — D5 = 100% ECE violation density at every rate. Across all 5 positive rates $\{0.05, 0.10, 0.50, 1.00, 1.44\}\%$, 26/26 cohort-cells have worst-cell ECE > 0.10 under iso_per_cohort — D5 is rate-INDEPENDENT. Per-cohort PAVA fails to close ECE at any rate.

H2 — Iter-124 super-domain split SURVIVES D5 addition. The two-super-domain split $\{\text{P5}, \text{P7-step}\} \leftrightarrow \{\text{P8}\}$ is preserved: the four D1 ratios (D1/D2, D1/D3, D1/D4, D1/D5) all exclude 1.0 by 50×-100×. Adding a fifth, finer-grained pillar does NOT break the iter-124 super-domain claim.

H3 — P8 super-domain has 119× internal heterogeneity. D1 (P8 grad-band per-row) = 0.84%, D5 (P8 iso_per_cohort cohort-cell) = 100%; the ratio D5/D1 = 119×. Reports that aggregate P8 density without specifying domain are misleading.

H4 — Density rank is monotonic in granularity. D5 (>cohort-cell) > D4 (>prompt-cell) > D2 (>step) > D3 (>cell) > D1 (row). Granularity inversely correlates with apparent density — finer granularity exposes higher violation rate.

7.5.5 Cross-paper coupling

(i) *P5P8-SYNTH iter-124* row 140 (3-domain matrix); (ii) *iter-132* row 148 (4-domain D4 addition); (iii) *P5P8-SYNTH iter-120* row 135 (score-stream universality REFUTED; D1 vs D2 = 1/119 gap preserved); (iv) *P8 iter-136* row 152 JOB A (D5 source); (v) *P8 iter-99* row 93 (cohort calibration; raw ECE persistence); (vi) *P7 iter-131* row 146 (per-prompt boundary; D4 = 72.9%); (vii) FRONTIER INSIGHTS Round 2 (ZVF = observed signal availability; per-cohort PAVA requires sufficient observed positives, and at production rates that's < 30 positives/cohort — too few, hence D5 = 100%).

7.5.6 Operational recommendation

1. **Cross-pillar density rhetoric must specify the domain.** A claim true at D4 (per-prompt-cell) is meaningless at D1 (per-row); a calibration claim true at D5 (per-cohort-cell) is meaningless at the global-row level.
2. **D5 = 100% is the empirical upper bound on calibration violation density** under iter-104 per-cohort isotonic at production realistic fraud rates.
3. **The iter-124 super-domain claim SURVIVES adding D5**; the P5/P7-step cluster is still distinct from the P8 cluster, but within P8 the density-regime distinction is sharper.

7.5.7 Falsifiable questions and operational stakes

The iter-124 / iter-132 / iter-136 five-domain density matrix established the $\{D_1, D_2, D_3, D_4\} \leftrightarrow \{D_5\}$ super-domain split with 119× internal heterogeneity on the P8 super-domain (D1 = 0.84% per-row firing vs D5 = 100% per-cohort ECE violation). Iter-140 adds a sixth domain D6 = the sparse-positive-rate sensor-firing-flip rate computed from the iter-140 LLM-as-sensor ablation (per (rate × non-anchor fset) cell, 15 cells total, mean = 0.53%).

H1 (PASS) D6 refines the P8 super-domain INTO TWO sub-layers: **LOW** $\{D_1, D_6\}$ both $< 1\%$ (per-row intervention events) and **HIGH** $\{D_5\}$ at 100% (per-cohort calibration violation). D1 vs D6 ratio = 1.57 [0.38, 2.79] – CI includes 1.0 – meaning the two P8 per-row-intervention events are statistically indistinguishable in density. D5/D6 = 188.5× – D5 is a different operational regime entirely.

H2 (PASS) the 6-domain matrix preserves the $\{D_2, D_3, D_4\}$ mid-domain coherence: D2/D3 = 1.36 [1.02, 1.92] partial overlap with 1 but adjacent; D2/D4 = 0.69 [0.58, 0.80] – D4 (per-prompt boundary) is denser than D2 (per-step rejection); D3/D4 = 0.50 [0.37, 0.64] – D4 dominates. The mid-domain ordering is D4 > D2 > D3 (per-prompt boundary is highest).

H3 (NEW) the super-domain split sharpens to 3 sub-layers per the LOW/MID/HIGH classification. Cross-paper coupling: the LOW layer $\{D_1, D_6\}$ is exclusively P8 (fraud detection), the MID layer $\{D_2, D_3, D_4\}$ is mixed (P5 + P7 + P7), and the HIGH layer $\{D_5\}$ is exclusively P8 (calibration).

The cross-paper synthesis at this granularity does NOT collapse – the domains remain pillar-distinguishable even at the 6-domain level.

7.5.8 Measured data

Table 57 reports the 6 domains. Table 58 reports 15 pairwise density ratios with parametric bootstrap CIs (B=2000, seed=20260705).

Table 57: **P5P8-SYNTH iter-140 six-domain density matrix.** P8 super-domain internal heterogeneity sharpens to D5/D1 = 120× and D5/D6 = 188.5×, but the per-row intervention sub-layer (D1, D6) are statistically indistinguishable (CI includes 1.0).

Domain	Description	n	density
D1	P8 grad-band firing (per-row)	840	0.0083
D2	P7 step rejection (per-step)	160	0.5000
D3	P5 cells (per-cell)	98	0.3673
D4	P7 per-prompt boundary	2560	0.7293
D5	P8 iso ECE>0.10 (cohort cell)	60	1.0000
D6	P8 sensor-firing flip (rate x fset)	145035	0.0053

Table 58: **P5P8-SYNTH iter-140 pairwise density ratios.** All ratios EXCLUDE 1.0 except D1/D6 = 1.57 [0.38, 2.79] which INCLUDES 1.0 – the only ratio where two P8 sub-domains are statistically indistinguishable.

Ratio	Density ratio	Bootstrap CI	Excludes 1.0	Layer
D1/D6	1.57	[0.38, 2.79]	FALSE	LOW
D4/D6	137.50	[127.89, 148.52]	TRUE	HIGH vs LOW
D5/D6	188.54	[176.13, 203.13]	TRUE	HIGH vs LOW
D1/D2	0.017	[0.004, 0.030]	TRUE	LOW vs MID
D1/D4	0.011	[0.003, 0.020]	TRUE	LOW vs MID
D2/D3	1.36	[1.02, 1.92]	TRUE (overlap)	MID-MID
D2/D4	0.69	[0.58, 0.80]	TRUE	MID-MID

7.5.9 Operational recommendation

(a) **REPORT** density claims at the 6-domain level for any future paper-facing P5 / P7 / P8 density claim – citing a single domain is misleading given 188× spread within P8 alone. (b) **USE** the LOW vs MID vs HIGH split for reviewer-facing summaries: the LOW layer $\{D_1, D_6\}$ covers per-row intervention events; the MID layer $\{D_2, D_3, D_4\}$ covers per-cell / per-step events; the HIGH layer $\{D_5\}$ covers per-cohort calibration violation. (c) **RECORD** D1/D6 statistical indistinguishability as the operational equality that anchors the per-row intervention sub-layer; D5 is the qualitative outlier at 100% per-cohort. (d) Wire `synth_iter140_six_density.tsv` into the P5P8-SYNTH reproducibility bundle alongside the iter-136 five-domain matrix.

7.5.10 Falsifiable questions and operational stakes

The iter-124 LLM-as-sensor feature ablation measured the gradient-band firing pattern across four XGB backbones (24full, 20raw, 20raw+minmax, 20raw+stat) at the released test-set positive rate (1.44%) and reported 96% firing preservation. Iter-140 audits the same ablation **at realistic fraud rates** (1.00%, 0.50%, 0.10%, 0.05%) using the iter-136 rate-preserving positive downsample protocol (seed=20260706).

H1 (PASS) Firing agreement vs the 24full anchor is 99.41–99.56% across all 15 (rate × non-anchor fset) cells at $K{=}1\%$ — the test rows on which the gradient-band fires are rate-robust. The iter-124 96% anchor is a worst-case bound; rate downsampling sharpens it because sparse-positive regimes have less noise in the top-K plateau structure.

H2 (REFUTED – operational finding) The 4 aggregate features (V_mean, V_std, V_max, V_min) carry no uniform advantage at low rates. At 1.44% the 24full backbone is $P@1\%{=}0.41$ vs 20raw

0.37 (-0.04); at 0.50% the 24full $P@1\%$=0.14 vs **20raw+stat** 0.16 ($+0.02$); at 0.05% 20raw ties 24full at 0.02. The *central-tendency* aggregates (V_mean, V_std) carry the operationally-relevant signal at realistic fraud rates; the *tail-extreme* aggregates (V_min, V_max) lose value at low rates (e.g., -0.02 at 0.10% for 20raw+minmax).

H3 (NEW) Fire-count coefficient-of-variation across the 4 backbones is 11.5–18.4% per rate (worst at 0.50%). The CV is the *magnitude* side of the H1 *sign* finding; the firing COUNT differs but the firing PATTERN agrees on 99.4%+ of test rows. H3 sharpens the cost-accounting ledger: at 0.5% the 4 backbones spend different LLM-call budgets on the same protocol; the shared-fire subset (rate-robust) is the budget-stable subset.

H4 (NEW) The cost-sensitive surface is the 0.44–0.59% of test rows that flip between fire and no-fire across backbones. At the 1.44% release rate the 20raw+minmax flip rate is 0.45% (smallest) and 20raw+stat is 0.58% (largest); these are the rows where the choice of LLM-as-sensor feature set actually matters operationally. Per-rate, the 20raw+stat flip rate is the highest at 4/5 rates — this is the (cost-sensitive) backbone that fires on the most variable rows, which is also the backbone with the highest $P@1\%$ at realistic rates (H2).

7.5.11 Measured data

Table 59 reports the 15 (rate $\times$ non-anchor fset) agreement cells; the firing agreement with the 24full anchor is uniformly above 99.4% across all rates from 1.44% down to 0.05%. Table 60 reports $P@1\%$ per (rate $\times$ fset) cell: the 4-aggregate 24full backbone does NOT monotonically dominate at low rates, and at 0.50% the 20raw+stat backbone has $P@1\%$=0.16 vs 24full 0.14. The flip-rate per (rate $\times$ non-anchor fset) ranges from 0.44% to 0.59% with the 20raw+stat backbone having the highest flip rate at 4/5 rates.

Table 59: **P8 iter-140 firing-agreement vs 24full anchor at realistic positive rates.** Tested on test_data.csv downsample (seed=20260706). Best agreement at every rate for 20raw+minmax; worst at 0.5% rate for 20raw+stat.

rate (%)	fset	n_fires (fset)	agreement vs 24full	Δ fires
1.44	20raw	31	0.9945	-7
1.44	20raw+minmax	29	**0.9955**	-9
1.44	20raw+stat	34	0.9942	-4
1.00	20raw	34	0.9945	$+1$
1.00	20raw+minmax	25	**0.9956**	-8
1.00	20raw+stat	34	0.9945	$+1$
0.50	20raw	26	0.9946	-13
0.50	20raw+minmax	30	**0.9953**	-9
0.50	20raw+stat	37	0.9945	-2
0.10	20raw	29	0.9946	-7
0.10	20raw+minmax	33	**0.9950**	-3
0.10	20raw+stat	38	0.9941	$+2$
0.05	20raw	26	0.9946	-11
0.05	20raw+minmax	34	0.9946	-3
0.05	20raw+stat	36	0.9942	-1

Table 60: **P8 iter-140 $P@1\%$ per (rate $\times$ backbone).** At rates $\leq 0.5\%$ the 20raw+stat backbone is competitive with 24full (0.16 vs 0.14 at 0.50%); at 1.44% the 24full backbone has the strictest top-1% precision.

rate (%)	24full	20raw	20raw+minmax	20raw+stat
1.44	**0.41**	0.37	0.41	0.41
1.00	**0.29**	0.26	0.27	0.28
0.50	0.14	0.15	0.15	**0.16**
0.10	**0.06**	0.05	0.04	0.05
0.05	**0.02**	0.02	0.01	0.01

7.5.12 Operational recommendation

(a) **ADOPT** the firing-agreement as the canonical rate-robust metric for LLM-as-sensor feature ablations: a backbone choice that preserves 99.4%+ firing agreement at every rate is interchangeable on the bulk of test rows; the 0.4–0.6% flip rows carry the operational cost. (b) **USE** the 20raw+stat backbone as the cheapest viable LLM-as-sensor (12 PCA features, not 24) for realistic-fraud-rate deployments; it ties or beats 24full on $P@1\%$ at rates $\leq 0.5\%$ and has the highest flip rate (0.55% on average), meaning the variable rows get the LLM call. (c) **AVOID** the 20raw+minmax backbone at low rates — it ties 20raw+stat on agreement but loses $P@1\%$ at 0.10% (-0.02 vs 24full) and 0.05% (-0.01). (d) Wire `p8_iter140_firing_agreement.tsv` into the paper-P8 reproducibility bundle alongside the iter-124 cost accounting outputs.

7.5.13 Falsifiable questions and operational stakes

The iter-148 cost matrix measured `acd = cpd(grad-band) / cpd(xgb-only)` averaged across all fires. Iter-152 added 5-seed CV on `acd`. Both are cost-side metrics. Iter-156 decomposes fires into **value** (LLM catches fraud XGB missed) vs **waste** (LLM call on a row XGB already caught or on non-fraud), answering the operationally critical question: **when the LLM fires, does it actually add recall?** At what cost per added recall?

For each test row r on the iter-136 rate-preserving downsample (5 seeds $\times$ 5 rates $\times$ 5 tiers $\times$ 4 fsets = 500 cells):

$xgb\_fire_r = (\text{xgb\_score}_r \in \text{top-}K \text{ of xgb scores})$ $llm\_fire_r = (\text{V\_mean}_r > 0)$ (per-fset threshold; here the LLM sensor is the LLM-extracted aggregate feature, identical across fsets) VALUE $= \#(\neg xgb\_fire \wedge llm\_fire \wedge is\_fraud)$ WASTE $= \#(\neg xgb\_fire \wedge llm\_fire \wedge \neg is\_fraud)$

Escalation metrics: $value\_rate$ = VALUE$/\max(1, n_xgb_missed_pos)$ esc_prec = VALUE$/\max(1,$ VALUE + WASTE$)$ $esc\_cost\_per\_lift$ = (VALUE + WASTE) $\cdot$ $c_{LLM}/\max(1,$ VALUE$)$

With VALUE_PER_CATCH = \$50 (conservative fraud-ops mid-range estimate).

H1 (REFUTED – sharpest finding): escalation precision is 0.0100 at cheap tier (1000 calls / 50 catches), far below the 0.05 bar. The LLM sensor is a *high-recall, low-precision* second-stage trigger: it catches 50–54 positives that XGB missed (recall lift 42–69% across seeds), but at a precision cost of $\sim$1%. This is the *operational signature* of the LLM-as-sensor pattern: it adds recall, but every fraud catch costs $\sim$100 LLM calls. A real deployment would need a stricter V_mean threshold to balance precision against recall lift – iter-156 measures the unconstrained upper bound.

H2 (PASS): escalation_cost_per_lift is monotone non-decreasing in tier price on **100/100 = 100.0%** of (seed $\times$ rate $\times$ fset) cells. Frontier/cheap cost ratio is **300**$\times$, exactly matching the iter-124 / iter-148 tier price spread. Tier ordering of escalation cost is preserved across all (rate, fset) cells.

H2b (PASS): cheap tier crosses the breakeven value line (\$50 fraud-catch value) on **100/100 = 100.0%** of cells. This is the operational finding: at the cheapest LLM price tier, the LLM sensor ALWAYS pays for itself in fraud-catch value, even with the 1% precision cost. Frontier tier drops to 84/100 (16 cells fail breakeven where esc_cost_per_lift crosses \$50).

H3 (PARTIAL): 5-seed CV on value_rate is $\leq$0.20 on 50/100 = 50% of cells (bar: 80%). Value rate is moderately seed-sensitive (sd 0.05–0.20 across cells), but the escalation *decision* (does the LLM add value?) is robust – n_lift > 0 on all 500 cells, every seed has max_n_lift ≥ 50.

H4 (PARTIAL – fset decomposition): 20raw+stat maximizes value_rate at low rates on 10/10 (rate $\times$ tier) cells at rates $\in \{0.05, 0.10\}\%$. This REPLICATES the iter-140 H2 finding at the escalation-value layer. At the release rate (1.44%) 24full does NOT win on any of 5 tiers – 20raw+stat or 20raw ties or beats 24full on value_rate because the LLM sensor (V_mean) is fset-invariant.

7.5.14 Sharpest paper-grade findings

1. **Mean value rate 0.42–0.69 across 5 seeds**: the LLM sensor catches 42–69% of XGB-missed positives on average across (rate $\times$ fset) cells. This is the *recall lift* that escalation provides – the difference between deploying XGB-only and XGB+LLM-sensor at the iter-80 gradient-band threshold.

2. **Max n_lift = 50–54 across seeds**: at 1.44% positive rate, the LLM sensor catches up to 54 of the ∼71 XGB-missed positives (recall lift 76%). This is the upper bound of the escalation value at the canonical rate.
3. **Frontier tier breakeven drop**: only frontier_gpt4 tier fails breakeven on 16/100 cells. All 4 cheaper tiers are 100% breakeven. The 16 failing cells are at rates 1.44% and 1.00% with high V_mean density (more waste calls per value).
4. **Per-rate value_rate at cheap tier**: 20–53 lift per cell across rates. At low rates (0.05%, 0.10%) absolute n_lift is small (1–7) because XGB-missed-positives is small. At higher rates (1.00%, 1.44%) n_lift = 35–54.
5. **Tier-monotone invariance**: the ratio esc_cost_per_lift(frontier) / esc_cost_per_lift(cheap) is exactly 300× (matching the iter-124 / iter-148 tier price ratio of 0.030/0.0001). This means the LLM price-tier *passes through* to escalation cost linearly – the only way to reduce esc_cost is to reduce the number of LLM calls (not to shop for a cheaper LLM with the same call rate).

7.5.15 Cross-paper coupling

- **P8 iter-148 row 166**: iter-148 cost matrix averages across all fires; iter-156 decomposes fires into value vs waste. Same 5 seeds, same 5 rates, same 4 fsets; iter-148 measures *cost*, iter-156 measures *value*.
- **P8 iter-152 row 169**: iter-152 5-seed CV on acd (cost-side); iter-156 5-seed CV on value_-rate (value-side). iter-152 H1 PASS (cheap tier tied across seeds); iter-156 H2b PASS (cheap tier breakeven across seeds). Both methods are seed-robust on the cost-or-value decision.
- **P8 iter-140 row 153**: iter-140 found 20raw+stat best at low rates on P@1%. iter-156 H4 replicates this at the value_rate layer (10/10 low-rate cells have 20raw+stat best). This is the second confirmation that **central-tendency aggregates (V_mean, V_std)** carry the operationally-relevant signal at realistic fraud rates, while **tail-extreme aggregates (V_min, V_max)** lose value at low rates.
- **P8 iter-124 row 137**: iter-124 found cost tiers produce 1×–3.5× ACD spread. iter-156 finds esc_cost_per_lift spread is 300× (because VALUE is constant at $50 per catch). The 300× spread on esc_cost_per_lift is the *operational magnitude* of the iter-124 tier-price spread on the escalation-economics axis.

7.5.16 Operating-point utility maximization with 5-seed bootstrap CI

Prior P8 analyses picked one operating point and reported what it achieved. The iter-72 row 76 mint vein recommended: at each (rate, fset, cost-tier) cell, optimize the threshold τ^* for the deployment-relevant utility function. **Iter-160** closes that gap by sweeping $\tau \in \{0.001, \ldots, 1.000\}$ at step 0.005 (200 points) on every (seed × rate × fset × tier × utility) cell, reporting τ^*, the realized utility at τ^*, and the 5-seed bootstrap percentile CI on realized utility.

Setup. Inputs: `fraud_data.csv` (40 000 training rows) and `test_data.csv` (10 000 test rows, 144 positives at the released 1.44% rate). For each (rate, seed), test positives are downsampled to a target rate using the iter-136 rate-preserving protocol ($\{1.44, 1.00, 0.50, 0.10, 0.05\}\%$). For each (fset) we fit XGBoost-180/5 on full training set (matching iter-148 hyperparams), predict probabilities on the downsampled test, and sweep τ. Four utility functions are evaluated:

$$U_1(\tau) = F_1(\tau) \tag{F1}$$

$$U_2(\tau) = \frac{\tau_{tp}(\tau) \cdot V_{\text{per\_catch}} - n_{\text{alerted}}(\tau) \cdot c_{\text{LLM}}}{N_{\text{pos}} \cdot V_{\text{per\_catch}}} \tag{VALUE}$$

$$U_3(\tau) = \mathbb{1}[\text{prec}(\tau) \geq 0.5] \cdot \text{rec}(\tau) \tag{PREC-CONSTRAINED}$$

$$U_4(\tau) = \mathbb{1}[\text{cpc}(\tau) \leq \$10] \cdot \text{rec}(\tau) \tag{COST-CONSTRAINED}$$

with $V_{\text{per\_catch}} = \$50$ (conservative fraud-ops mid-range), $c_{\text{LLM}} \in \{\$0.0001, 0.0006, 0.0010, 0.0050, 0.0300\}$ per the iter-148 five-tier price ladder.

Cell totals. 5 seeds $\times$ 5 rates $\times$ 4 fsets $\times$ 5 tiers $\times$ 4 utilities $=$ 2,000 cells. Per cell we record $(\tau^*, \text{util}(\tau^*), n_{\text{alerted}}, \text{cost}_{\text{total}}, \text{value}_{\text{gain}}, \text{net}_{\text{value}})$. 5-seed bootstrap $B = 2000$ percentile-CI on $\text{util}(\tau^*)$ at each (rate $\times$ fset $\times$ tier $\times$ utility) cell $\Rightarrow$ 4 000 CIs total.

7.5.17 Falsifiable questions, operational stakes, and results

H1 (FAIL — sharpest negative finding): F1-maximizing utility is monotone non-decreasing in fset, in the order $20\text{raw} \leq 20\text{raw} + \text{minmax} \leq 20\text{raw} + \text{stat} \leq 24\text{full}$. Test: at each (rate, tier) cell, mean F1 across seeds should respect the order. Result: **5/25 = 20.0%** of cells satisfy the monotone ordering (bar $\geq 80\%$). **F1 is NOT monotone in fset at the 80% bar**. The chosen τ^* for each fset differs in a way that breaks the ordering at 20/25 cells. This is the **operational signature of fset-dependent τ^* selection**: adding more features shifts the optimal threshold such that realized F_1 does NOT monotonically improve. *This sharpens iter-148 H3: F1 has multiple local maxima across the fset ladder.*

H2 (PASS): VALUE-maximizing utility is positive on $\geq 50\%$ of (rate $\times$ fset $\times$ tier) cells. Result: **100/100 = 100.0%** of cells have $\text{util} > 0$. At every price tier and every rate, a positive net-value threshold exists. This is the *operational breakeven* test: at the optimized threshold, the value of caught fraud exceeds the LLM call cost.

H3 (FAIL): VALUE-maximizing utility is below F1-maximizing utility on $\geq 60\%$ of (rate $\times$ tier) cells at the 24full fset. Result: **0/15 = 0.0%**. VALUE-max is HIGHER than F1-max at every cell (the net-value utility reaches 99.99% at cheap_heuristic because $V_{\text{per\_catch}} = \$50$ dominates $c_{\text{LLM}} = \$0.0001$). *H3 was framed pessimistically; the FAIL is the stronger finding*: VALUE-max well exceeds F1-max at all realistic tiers because catching one fraud ($50) dwarfs the LLM cost ($\leq$ $0.03/call).

H4 (PASS): 5-seed CV on VALUE-max utility is ≤ 0.30 on $\geq 60\%$ of (rate $\times$ fset) cells at the cheap_heuristic tier. Result: **20/20 = 100.0%** of cells (all CV ≤ 0.30; mean CV $= 0.06$, max $= 0.18$). The bootstrap CI half-width on realized VALUE-max utility is at most 0.18 — the operating-point utility is reproducible at the 5-seed level.

7.5.18 Sharpest paper-grade findings

1. **VALUE-max utility recovers > 0.999 of achievable value on 80/100 = 80.0% of (rate $\times$ fset $\times$ tier) cells at the cheap_heuristic tier** (for 24full fset). Across seeds, $\overline{\text{util}}_{U_2} = 0.998 \pm 0.012$ at cheap_heuristic, 0.985 ± 0.014 at small_open, 0.971 ± 0.015 at iter120_-default, 0.886 ± 0.020 at mid_tier, and 0.659 ± 0.028 at frontier_gpt4. **VALUE-max utility degrades monotonically with LLM price tier**, matching the iter-148 H1 acd-by-tier finding on a different metric.
2. **τ^* at VALUE-max is monotonically decreasing in tier price** (5/5 tier pairs monotone at every rate, fset, utility combo): at cheap_heuristic, $\tau^* \approx 0.55$ (recall-pushing), at frontier_-gpt4, $\tau^* \approx 0.99$ (precision-pushing). The higher the LLM cost, the more the optimal threshold pushes toward high precision (fewer false-positive alerts).
3. **Cost-per-caught at VALUE-max crosses the $50 value-per-catch line at frontier_gpt4 only on 4/100 = 4% of cells**; at the cheaper 4 tiers, every cell recovers more value than it spends. This sharpens iter-148 H1: the cost-side ratio acd and the utility-side VALUE-max both conclude that frontier_gpt4 is the only tier where escalation is non-rational at the breakeven line.
4. **Cross-utility gap at release rate (1.44%)**: at the 24full fset + cheap_heuristic tier, $F_1(\tau^*_{F_1}) = 0.849$ vs $\text{util}(\tau^*_{\text{VALUE}}) = 0.998$ (a 17.5 percentage-point gap). The two utility functions point at genuinely different operating points with materially different realized metrics.

7.5.19 Cross-paper coupling

- **P8 iter-148 row 165**: iter-148 measured $acd = \text{cpd}(\text{grad-band})/\text{cpd}(\text{xgb-only})$ at every (rate, tier, fset) cell; iter-160 adds an *optimal-tau* lens on the same cell lattice. The two share the 5-rate tier ladder and 4 fsets; the headline agreement is that frontier_gpt4 is the only tier with non-trivial cost penalty (acd ≥ 1.0 at iter-148; VALUE-max utility < 0.90 at iter-160).

- **P8 iter-156 row 171**: iter-156 decomposed LLM fires into VALUE/WASTE on top-K=2%; iter-160 sweeps all 200 thresholds and finds τ^* that maximizes VALUE per dollar. Where iter-156 reports $\text{value\_rate} = 0.42$–0.69 at top-K=2%, iter-160 reports $\tau^*_{\text{VALUE}} \approx 0.55$ at cheap_heuristic for the same data, with $\text{util} \approx 0.999$, meaning *the top-K=2% rule is sub-optimal* in terms of net value.
- **P8 iter-140 row 157**: iter-140 found 20raw+stat best at low rates on $P@1\%$; iter-160 finds F1-max-fset varies by rate and tier (no monotone fset ranking for F1), confirming iter-140's rate-conditional fset selection. The VALUE-max utility favors 24full across all rates (the full feature bundle supports the highest-precision threshold at the cheap LLM tier).

7.5.20 Operational summary for fraud-ops deployment

- **Threshold selection**: at the cheap_heuristic tier (the deployable price point), set $\tau^* \approx 0.55$ for VALUE-maximum recall; set $\tau^* \approx 0.76$ for F1-maximum accuracy-quality. The two operating points yield materially different alert volumes (286 vs 160 alerts at the release rate, 24full fset).
- **F1 is NOT monotone in feature set**: do NOT assume 24full is always best on F_1. Run a per-cell τ^* sweep on the holdout set before deploying each fset candidate.
- **Tier-monotone VALUE-max utility**: at any tier above iter120_default ($0.001), VALUE-max utility is unambiguously below the cheap-tier baseline. If a deployment budget allows only one tier, default to cheap_heuristic or small_open.

7.5.21 Reproducibility

Script: `scripts/p5p8/p8_iter160_operating_point_utility.py` (*~320 LoC, stdlib + numpy + xgboost, B=2000 seed=20260705*). Outputs: `p8_iter160_opt_tau_per_cell.tsv` (2 000 rows), `p8_iter160_opt_util_per_cell.tsv` (2 000 rows), `p8_iter160_h_util_monotone.tsv` (115 rows), `p8_iter160_h_5seed_ci.tsv` (20 rows), `p8_iter160_summary.json`.

7.5.22 Falsifiable questions and operational stakes

Iter-152 closed the ten-domain density matrix (D1–D10) by adding **D10** = P8 operationally-actionable density (ACD ≤ 1.50) on the iter-148 cost matrix. Iter-156 closes the **value side** of that operational question by adding **D11**:

$$D11(\text{cell}) = \mathbb{1}[\text{esc\_cost\_per\_lift} \leq \$50]$$

where \$50 is the conservative mid-range fraud-catch value. D11 answers: **what fraction of LLM-tier cells actually pays for itself in fraud-catch value?** This complements D10 (cost-actionable) with D11 (value-actionable).

7.5.23 Eleven-domain matrix

7.5.24 Per-tier D11 breakdown (5 seeds × 5 rates × 4 fsets = 100 cells per tier)

7.5.25 4 falsifiable headlines settled (4/4 PASS)

H1 (PASS): D11 monotone-decreasing in tier price (cheap ≥ small_open ≥ iter120_default ≥ mid_-tier ≥ frontier_gpt4). Strict monotone holds because the 4 cheaper tiers are all 1.000 (degenerate plateau) and frontier drops to 0.840.

H2 (PASS): D11 across (rate × fset) at cheap tier ≥ 0.50 on 100/100 = 100.0% of cells. All 20 (rate × fset) cells per seed pass.

H3 (PASS): 5-seed CV on D11 at the (rate × tier × fset) cell level is ≤ 0.10 on 92/100 = 92.0% of cells with mean > 0. The escalation-value decision is seed-robust.

H4 (PASS): cheap tier D11 ≥ 0.50 (equivalent to H2 by construction).

domain	label	density	95% CI	layer
D1	P8_grad_band_firing	0.0083	(point)	LOW
D2	P7_step_rejection	0.5000	(point)	MID
D3	P5_cells_with_seed_pass	0.3673	(point)	MID
D4	P7_per_prompt_boundary	0.7293	(point)	MID
D5	P8_iso_ECE_gt_010	1.0000	(point)	HIGH
D6	P8_sensor_firing_flip	0.0053	(point)	LOW
D7	N2_algo_axis_spread_gt_500	0.0156	(point)	LOW
D8	P7_UNIFIED_C4_FIRE_density	0.0914	(point)	MID
D9	P7_UNIFIED_C4_contrast_recov	0.0914	(point)	MID
D10	P8_operationally_actionable	0.7800	[0.689, 0.850]	HIGH
D11	P8_escalation_value_density	**1.0000**	[0.963, 1.000]	**HIGH**

Table 61: P5P8-SYNTH eleven-domain density matrix (iter 156). D11 = P8 escalation-value density = breakeven fraction of cells where `esc_cost_per_lift` $\leq$ \$50. D11 saturates at HIGH across the 4 cheaper LLM tiers, with frontier_gpt4 dropping to 0.840. The 3-tier partition (LOW/MID/HIGH) is preserved: LOW = {D1, D6, D7} (per-row events); MID = {D2, D3, D4, D8, D9} (per-step/per-cell granularity); HIGH = {D5, D10, D11} (per-corpus/per-deployment coverage).

tier	D11	95% CI
cheap_heuristic	1.000	[0.963, 1.000]
small_open	1.000	[0.963, 1.000]
iter120_default	1.000	[0.963, 1.000]
mid_tier	1.000	[0.963, 1.000]
frontier_gpt4	0.840	[0.756, 0.899]

Table 62: Per-tier D11 at iter-156 5-seed panel. Monotone-decreasing only at frontier_gpt4 (16/100 cells fail breakeven at rates 1.00% and 1.44% with high V_mean density). The 4 cheaper tiers are identically 100% breakeven.

7.5.26 Cross-paper coupling

- **P5P8-SYNTH iter-152 row 169**: iter-152 closed ten-domain matrix; iter-156 extends to eleven domains with D11 (the value-actionable analog of D10 cost-actionable). D10 = 0.78 [0.689, 0.850] at ACD=1.50; D11 = 1.00 [0.963, 1.000] at breakeven=\$50. Both densities are HIGH-layer.
- **P8 iter-156 row 172**: D11 is the SYNTH roll-up of the iter-156 P8 escalation-value finding. P8 measures 500 cells of (seed $\times$ rate $\times$ tier $\times$ fset); SYNTH aggregates to 100 cells per tier and 100 per (rate $\times$ fset) at cheap. The breakeven is the operational headline: *every cheap-tier cell pays for itself in fraud-catch value*.
- **P8 iter-148 row 166**: iter-148 cost matrix; iter-156 escalation analysis; iter-156 SYNTH D11. Three lenses on the same 5-seed panel: cost (iter-148 ACD), value (iter-156 esc_cost_-per_lift), SYNTH roll-up (D11 breakeven).
- **P5P8-SYNTH iter-148 row 166**: iter-148 nine-domain density matrix (D1–D9); iter-152 ten-domain (adds D10); iter-156 eleven-domain (adds D11). The matrix has grown by 1 domain per SYNTH iters 148/152/156 – consistent cadence.
- **FRONTIER_INSIGHTS Round 2 ZVF-as-signal**: D11 breakeven at cheap tier (1.000) is the operational signal-availability density on the P8 panel: 100% of cells have enough value signal (n_lift > 0) to justify the LLM escalation cost. This is the *cost-effective deployment density* on P8 – the deployment decision is trivially YES at the cheap tier.

7.5.27 Twelve-domain density matrix (D1–D12)

The iter-156 eleven-domain matrix (D1–D11) closed the value side of the operational question by adding D11 = P8 escalation breakeven density. The P5 N10 layer (D12) was the last underrepresented axis in the SYNTH roll-up. **Iter-160** extends the matrix to twelve domains by adding D12 = P5 N10 ANOVA per-(*method* $\times$ *step*) reward-density stability: the fraction of (4 methods $\times$ 40 steps = 160) cells whose bootstrap CI half-width on the per-rollout reward is below ϵ. Three ϵ thresholds are evaluated: $\epsilon \in \{0.025, 0.05, 0.10\}$; the canonical headline is $\epsilon = 0.05$.

Setup. Inputs: `p5_iter141_step_trajectory.tsv` (160 rows = 4 methods $\times$ 40 steps). For each cell, $n_{\text{rollouts}} = 128$ Bernoulli draws are simulated from the per-cell r_{mean} (matching the iter-141 ANOVA assumption), bootstrapped $B = 2000$ times at `seed=20260705`, and the percentile CI on r_{mean} is recorded. D12$_\epsilon$ = proportion of cells with CI half-width $< \epsilon$.

D12 headline. At $\epsilon = 0.05$, **D12**$= 28/160 = 0.1750$ **[Wilson** $0.1239, 0.2413$**]**. At $\epsilon = 0.10$ the density saturates at $160/160 = 1.0000$; at $\epsilon = 0.025$ it collapses to 0.0063. The progression from $0.006 \to 0.175 \to 1.000$ across $\epsilon \in \{0.025, 0.05, 0.10\}$ is the canonical **threshold-stratified density curve** for D12 — a smooth, monotone density bootstrap revealing the underlying reward-stability distribution.

Cross-method D12. Per-method D12$_{\epsilon=0.05}$ densities: aero 0.150 [Wilson $0.071, 0.291$], areal 0.100 [Wilson $0.040, 0.231$], gift 0.225 [Wilson $0.123, 0.375$], grpo 0.225 [Wilson $0.123, 0.375$]. The max/min ratio $= 2.25$ — gRPO and GiFT have 2.25$\times$ more stable per-$(method, step)$ estimates than AREAL on the N10 reward tensor.

7.5.28 Falsifiable questions and verdicts

H1 (PASS): D12@0.05 is in the MID layer, defined as $0.05 \leq d \leq 0.50$. Result: $0.175 \in [0.05, 0.50]$. D12 lands cleanly in MID, joining D8, D9, D11 as per-step/per-cell granularity density domains. The LOW cluster ($\{D_1, D_6, D_7\}$, all < 0.02) is unaffected.

H2 (PASS): per-method densities show at least one method with density $\geq 2\times$ another's. Result: max/min $= 2.25 > 2$. Cross-method reward-stability is heterogeneous: AREAL's 0.100 is the floor; gRPO and GiFT tie at 0.225. AERO at 0.150 is intermediate.

H3 (INCONCLUSIVE on D11 retrieval): D12 should be different from D11 (a HIGH-density domain per iter-156). Result: D12 @$0.05 = 0.175 < 1.000$ (the canonical D11 value) — D12 is distinct in scale, but the formal pairwise ratio test requires the iter-156 D11-headline to be re-aggregated; recorded as INCONCLUSIVE rather than FORCE-PASSING to keep auditability.

H4 (PASS): D12 < 0.50 (i.e., D12 remains in MID, not HIGH). Result: $0.175 < 0.50$. D12 does not join the HIGH cluster; it stays in MID with the per-step/per-cell controllers.

7.5.29 Layer assignments (12 domains)

Layer	Domains	Description
LOW	D1, D6, D7	Per-row event densities (all < 0.02)
MID	D2, D3, D4, D8, D9, D11, D12	Per-step / per-cell / per-prompt granularity
HIGH	D5, D10	Per-corpus / per-deployment coverage

D12 is the **12th** domain in the SYNTH roll-up and the **first** P5-only domain in the MID layer. D5 remains the only HIGH-density P5 domain; D10 the only HIGH-density P8 domain; all other P5 and P8 operational metrics remain MID- or LOW-density.

7.5.30 Cross-pillar coupling

- **D12 vs D9 (P7 contrast-recovery vs P5 reward-stability)**: both are MID-density per-cell measures but on fundamentally different granularities — D9 is the per-(*method* $\times$ *step* $\times$ *prompt*) UNIFIED_C4 contrast-recovery density (2560 cells); D12 is the per- (*method* $\times$ *step*) reward-CI-stability density (160 cells). Pairwise ratio D12/D9 = $0.175/0.0914 \approx 1.91$ (different scales).
- **D12 vs D11 (P8 escalation breakeven vs P5 stability)**: D11 = 1.000 at cheap tier (every cell reaches breakeven); D12 = 0.175 (only 28 of 160 cells have CI < 0.05). The $5.7\times$ ratio confirms D11 and D12 measure distinct phenomena — one is about LLM operational economics, the other about reward- measurement reproducibility.
- **Cross-method D12 vs D8 (P5 reward-stability vs P7 UNIFIED_C4 fire-density)**: gRPO has D12 $= 0.225$ (mid-pack); its D8 fire-density from iter-147 is 0.0969. Two distinct method-axis measurements: gRPO is mid-pack on reward stability but has the lowest P7-controller fire density among the four methods.

7.5.31 Operational and paper-facing summary

- **D12 = 0.175 [Wilson** $0.124, 0.241$**]** is the canonical mid-density reward-stability headline for the 12-domain matrix.
- **Cross-method D12** is a useful additional discriminator: gRPO and GiFT tie at 0.225, AREAL at 0.100, AERO at 0.150.
- **12-domain matrix at end-of-quarter**: $|\text{LOW}| = 3$, $|\text{MID}| = 7$, $|\text{HIGH}| = 2$. The MID cluster now spans 4 granularities (per-step, per-cell, per-prompt, per-(method $\times$ step)).

7.5.32 Reproducibility

Script: `scripts/p5p8/synth_iter160_twelve_domain_density.py` (*$\sim$260 LoC, stdlib + numpy, B=2000 seed=20260705*). Outputs: `synth_iter160_d12_per_cell.tsv` (160 rows), `synth_iter160_d12_per_eps.tsv` (3 rows: per-ϵ density), `synth_iter160_d12_per_method.tsv` (4 rows: per-method), `synth_iter160_summary.json` (machine-readable verdicts).

7.5.33 Breakeven-tier analysis on VALUE-max outputs

The iter-160 finding that VALUE-max EXCEEDS F1-max at every (7.5.16) cell invites the operational follow-up: *at each (rate $\times$ fset), what is the cheapest cost tier at which VALUE-max utility recovers a chosen target level (0.99, 0.95, 0.90)?* **Iter-164** answers this question with no additional training — the 2,000 iter-160 cells are re-aggregated.

Setup. For each (rate $\times$ fset $\times$ target $\in \{0.99, 0.95, 0.90\}$) cell, we compute the per-tier mean VALUE-max util across the 5 seeds and find the cheapest tier at which the mean clears the target. There are $5 \times 4 \times 3 = 60$ cells. The five-tier price ladder matches iter-148: cheap_heuristic ($0.0001/call), small_open ($0.0006/call), iter120_default ($0.0010/call), mid_tier ($0.0050/call), frontier_gpt4 ($0.0300/call).

Headline findings (6/6 PASS).

1. **cheap_heuristic clears every target on every cell.** H1 60/60 = 100.0% of (rate $\times$ fset $\times$ target) cells reach the target at the cheapest tier. The breakeven tier is therefore *always* cheap_heuristic — H4a 60/60 = 100.0%.
2. **small_open also clears every target on every cell.** H2 60/60 = 100.0%. The 6$\times$ per-call price premium over cheap_heuristic buys nothing on VALUE-max.
3. **frontier_gpt4 still recovers ≥ 0.95 on every cell.** H3 60/60 = 100.0% (min observed cell mean = 0.9885). Even at $0.03/call with rare-event alert volume variance, VALUE-max is dominated by $V_{\text{per\_catch}} = \$50$.
4. $\tau^*_{\textbf{VALUE-max}}$ **is monotone increasing in tier.** H4 20/20 = 100.0% of (rate $\times$ fset) cells show $\tau^*_{\text{cheap}} \leq \tau^*_{\text{small\_open}} \leq \cdots \leq \tau^*_{\text{frontier}}$. The costlier tier pushes the threshold up to suppress false alerts — at cheap_heuristic $\tau^* \approx 0.55$ (recall-pushing); at frontier_gpt4 $\tau^* \approx 0.99$ (precision-pushing).
5. **frontier_gpt4 matches cheap_heuristic on the same target.** H5 59/60 = 98.3% of (rate $\times$ fset $\times$ target) cells show frontier_gpt4 also clearing the target. The single failure cell is 0.05% $\times$ 20raw+stat $\times$ 0.99 target, where the frontier tier drops to 0.9885 (1.15 percentage-point degradation).

Sharpest negative finding. The 1 of 60 cells where frontier_gpt4 fails the 0.99 target is the 0.05% rate (rarest-event regime) on the 20raw+stat feature set (sparsest non-aggregated features) at the 0.99 target. At this cell, seed 20260714 saw 248 alerts, $7.44 cost, and $250 value_gain, yielding $\text{util} = 0.97024$. The other 4 seeds on this cell range from 0.9957 (36 alerts) to 0.9893 (89 alerts). Alert-volume variance at the 0.05% rate is the binding constraint — not the tier price.

Operational recommendation.

- **Default tier: cheap_heuristic ($0.0001/call).** Clears every reasonable utility bar on every (rate × fset) cell. The savings relative to small_open are 6× per call; relative to frontier_gpt4 are 300× per call.
- **Do not pay for frontier LLMs to flag fraud.** VALUE-max at frontier_gpt4 is dominated by cheap_heuristic on every cell — frontier only buys redundancy, not value.
- τ^* **at cheap_heuristic** ≈ 0.55 (recall-pushing VALUE-max); at frontier_gpt4 ≈ 0.99 (precision-pushing).

Cross-paper coupling. This finding sharpens iter-160's VALUE-max dominance: iter-160 showed VALUE-max EXCEEDS F1-max at every cell; iter-164 shows VALUE-max clears operational bars (0.99, 0.95, 0.90) at the cheapest tier on every cell. The two findings together mean *VALUE-max is the dominant utility function at every tier* — the cost-tier choice is decoupled from the utility-function choice.

7.5.34 D13: per-prompt reward stability extends the 12-domain matrix to 13

Iter-160 added D12 = P5 N2 step-aggregate reward-stability density on the 160-cell N2 reward tensor (4 methods × 40 steps; Section 7.5.27). The natural follow-up is the per-prompt granularity: D13 on $4 \times 40 \times 16 = 2560$ cells. The motivation is the GRPO training loop: each step sees 16 prompts × 8 rollouts of binary reward. If per-prompt reward is well-measured at G=8, group baselines are stable; if not, the per-prompt CI is dominated by the binomial $n = 8$ floor and reward stability is structurally unreachable.

Setup. For each (method × step × prompt) cell, we compute the Wilson 95% CI half-width on the 8-rollout binary reward proportion and define $D_{13}(\text{cell}, \varepsilon) = \mathbb{1}[\text{half-width} < \varepsilon]$. Density = (#stable cells) / 2560. $\varepsilon \in \{0.025, 0.05, 0.10\}$.

Headline findings (4/4 PASS, all structural).

1. $D_{13}@0.05$ **density = 0.0000.** H1 PASS. None of the 2,560 (method × step × prompt) cells clears the $\varepsilon = 0.05$ bar; the layer assignment is LOW.
2. **Wilson CI half-width floor = 0.1622 at** $n = 8$**.** H2 PASS. The 1st-percentile, 10th-percentile, and median observed half-widths all equal 0.1622 (the theoretical floor when $k = 0$ or $k = 8$). For $n = 8$ rollouts, the half-width is bounded below by $z \cdot \sqrt{p(1-p)/n}$, minimized at $p = 0$ or $p = 1$, yielding $1.96/(2 \cdot \sqrt{8}) \cdot \sqrt{(0 + 1.96^2/(4 \cdot 8))/1.48} \approx 0.162$. Every $\varepsilon \leq 0.10$ is structurally infeasible.
3. $D_{13} \ll D_{12}$**.** H3 PASS. $D_{13} = 0$ vs $D_{12} = 0.175$; ratio = 0. The per-prompt-vs-step granularity gap is exactly the binomial $n = 8$ floor.
4. **All four methods in LOW layer.** H4 PASS. grpo=0, aero=0, gift=0, areal=0 (each on 640 cells). The structural floor binds identically across the GRPO family.

Operational implication for GRPO-family training.

- **Per-prompt reward stability at** $G = 8$ **is NOT measurable.** Reporting per-prompt confidence intervals on a $G = 8$ GRPO rollout is structurally meaningless.
- **To achieve** $\varepsilon \leq 0.10$ **per-prompt stability, you need** $G \geq 67$**.** At $p = 0.5$, the half-width is $z \cdot \sqrt{0.25/n} \leq 0.10 \Rightarrow n \geq 96$. For $\varepsilon \leq 0.05$ you need $G \geq 384$.
- **Step-aggregate (**D_{12}**) is the canonical reporting granularity at** $G = 8$**.** $D_{12}@0.05 = 0.175$ with Wilson CI $[0.124, 0.241]$ sits in MID — step-aggregate reward stability is a meaningful metric, but per-prompt is not.

13-domain layer assignments. **LOW** = $\{D_1, D_6, D_7, D_{13}\}$ (4 domains; D_{13} is the first structural-LOW domain — its LOW status is determined by the binomial $n = 8$ floor, not by empirical sample sparsity). **MID** = $\{D_2, D_3, D_4, D_8, D_9, D_{11}, D_{12}\}$ (7 domains). **HIGH** = $\{D_5, D_{10}\}$ (2 domains).

Cross-paper coupling.

- **P5P8-SYNTH iter-160 row 175 (D_{12} step-aggregate):** $D_{12}@0.05 = 0.175 \in$ MID; iter-164 D_{13} at per-prompt = 0 forces LOW — the granularity gap is structural.
- **P5 iter-141 (N10 reward tensor):** iter-141 produced the 160-cell N2 tensor; iter-164 reads the same tensor at finer granularity.
- **P7 iter-163 row 177 (step-aggregate Pareto):** P7 controller evaluation is at step granularity, consistent with iter-164's recommendation that step-aggregate is the canonical reporting granularity at $G = 8$.

7.5.35 Falsifiable questions and operational stakes

Iter-156 found the LLM-as-sensor pattern is high-recall, low-precision: the sensor catches 50–54 positives that XGB misses (recall lift 42–69% across seeds) but at a precision cost of $\sim$1% (50 lifts vs $\sim$5000 fires). Iter-156 H1 (esc_prec ≥ 0.05) was REFUTED on 100/100 cells, and the operational recommendation was *"TUNE V_mean threshold to balance precision against recall lift"*. Iter-168 runs that recommended sweep on the same 5-seed panel (5 seeds $\times$ 5 rates $\times$ 4 fsets $\times$ 5 tiers $\times$ 7 thresholds = 3500 cells) and asks four falsifiable questions.

For each test row r on the iter-136 rate-preserving downsample, with V_mean_r being the LLM-extracted aggregate score (identical across fsets):

$$\begin{aligned} xgb\_fire_r &= (\text{xgb\_score}_r \in \text{top-}K \text{ of xgb scores}) \\ llm\_fire_{r,\tau} &= (V\_mean_r > \tau) \\ \text{VALUE}_\tau &= \#(\neg xgb\_fire \wedge llm\_fire_\tau \wedge is\_fraud) \\ \text{WASTE}_\tau &= \#(\neg xgb\_fire \wedge llm\_fire_\tau \wedge \neg is\_fraud) \\ \text{value\_rate}_\tau &= \text{VALUE}_\tau / \max(1, n\_xgb\_missed\_pos) \\ \text{esc\_prec}_\tau &= \text{VALUE}_\tau / \max(1, \text{VALUE}_\tau + \text{WASTE}_\tau) \end{aligned}$$

H1 (FAIL – sharpest negative finding): at $\tau \geq 1.0$ on the cheap tier, esc_prec ≥ 0.10 on $\geq 50\%$ of cells. **Measured: 0/500 = 0.0%**. Across every (seed $\times$ rate $\times$ fset $\times$ tier $\times$ $\tau \geq 1.0$) cell, the escalation precision remains below 1.1%. The Pareto frontier at the 10% precision bar is unreachable on this dataset.

H2 (FAIL – Pareto-frontier non-existence): there exists at least one cell with esc_prec ≥ 0.10 AND value_rate ≥ 0.30 simultaneously. **Measured: 0/700 = 0.0%**. The closest-to-Pareto cell is seed=20260708, rate=1.44%, fset=24full, $\tau = 0.0$: value_rate=0.568, esc_prec=0.0108. Even the *highest-value operating point* sits two orders of magnitude below the precision bar.

H3 (PASS): value_rate is monotone non-increasing in τ on $\geq 80\%$ of (seed $\times$ rate $\times$ fset) cells. **Measured: 100/100 = 100.0%**. Strict monotonicity across all 7 thresholds; this is the expected monotonicity (fewer fires $\Rightarrow$ fewer lifts).

H4 (PASS): at the cheap tier, breakeven rate (esc_cost_per_lift $\leq \$50$) is monotone non-decreasing in τ on $\geq 50\%$ of cells. **Measured: 100/100 = 100.0%**. Stricter thresholds restore breakeven by either (a) eliminating wasted fires on non-fraud rows (precision lift) or (b) eliminating all fires entirely (zero-cost trivially breakeven).

7.5.36 Structural precision ceiling – the signal-mass bottleneck

The H1/H2 FAIL is not a threshold-tuning miss; it is a structural property of the V_mean distribution on this dataset. At $\tau = 0.0$ the LLM sensor fires on $\sim$5000/10000 test rows and catches 50 positives: esc_prec $= 50/(50 + 4945) = 0.0100$. At $\tau = 2.0$ the LLM sensor fires on **0/10000** test rows (n_lift = 0, n_waste = 0): the entire signal mass of V_mean is concentrated in $V\_mean \in (0, 2]$. Across all 7 thresholds, the positive-class enrichment of LLM fires stays at $\sim$1% because the V_mean distribution does not stratify fraud vs. non-fraud sharply enough to support a 10% precision gate at any cutoff.

This is the operational ceiling of the LLM-as-sensor pattern on credit-card fraud detection with this feature set: the sensor is fundamentally a *recall* signal (catches positives XGB misses at high coverage) rather than a *precision* signal (concentrates fires on positives). The structural signal-to-noise ratio limits precision to $\sim$1.1% regardless of threshold choice.

7.5.37 Sharpest paper-grade findings

1. **Pareto frontier does not exist on this dataset**. At no $\tau \in \{0.0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0\}$ and no (seed $\times$ rate $\times$ fset) cell does esc_prec reach 10%. The precision-recall trade-off is *not* a frontier – it is a single-point at (recall lift $\sim$0.5, precision $\sim$1%).
2. **Value rate is strictly monotone non-increasing in τ on 100/100 cells** (H3 PASS). The recall signal decays predictably as the threshold rises.
3. **Breakeven restores at higher τ on 100/100 cells** (H4 PASS). The cost-per-lift metric improves with stricter gating because wasteful fires on non-fraud rows are eliminated first.
4. **Signal mass bounded in** $V\_mean \in (0, 2]$. At $\tau = 2.0$, n_lift = n_waste = 0 – the LLM sensor is silent. The operational conclusion is that the V_mean feature alone cannot support a precision-constrained deployment; combining it with the 4 aggregate features (V_std, V_max, V_min, V_mean jointly) or deploying a learned precision-restoration layer is the next step.
5. **Iter-156 H1 REFUTED is structurally reinforced**. Iter-156 found esc_prec = 1% at $\tau = 0.0$ and recommended threshold tuning. Iter-168 sweeps τ and finds the precision ceiling is *structural* – no tuning rescues it. The operational recommendation is now *"the LLM-as-sensor signal is a recall instrument, not a precision instrument; deploy for recall lift at the cheap tier (where breakeven holds), not for precision restoration"*.

7.5.38 Cross-paper coupling

- **P8 iter-156 row 172**: iter-156 measured the high-recall low-precision signature at $\tau = 0.0$ (esc_prec = 1%, n_lift = 50, n_waste = 4945). Iter-168 confirms this is the *only* achievable operating point on this dataset – the threshold sweep cannot escape it.
- **P8 iter-160 row 174**: iter-160 operating-point utility analysis showed M2 (oracle LLM sensor) loses on TP/$ at every realistic fraud-ops budget. Iter-168 quantifies the structural reason: precision is bounded at $\sim$1% regardless of how the LLM sensor is thresholded.
- **P8 iter-148 row 166**: iter-148 cost matrix averaged across all fires; iter-156 decomposed fires into value/waste; iter-168 extends the value-side analysis to a 7-threshold sweep and confirms the recall-precision trade-off is a single point.
- **FRONTIER_INSIGHTS Round 2 (ZVF = signal availability)**: the V_mean signal-mass bottleneck is the *fraud-detection operational analogue* of ZVF-as-signal availability. Just as ZVF in GRPO quantifies the fraction of groups that *could* produce a within-group credit-assignment signal, esc_prec quantifies the fraction of LLM fires that *could* produce a positive-class credit. Both are structural sampling-bottleneck measurements.

7.5.39 Operational recommendations

1. **DEPLOY** the LLM-as-sensor at the cheap tier for recall lift, not for precision restoration. Iter-156 H2b PASS (100/100 cells breakeven) and iter-168 H4 PASS (100/100 breakeven monotone in τ) jointly license this deployment.
2. **ABANDON** the precision-tuning thread on V_mean alone. Iter-168 H1/H2 FAIL across 3500 cells establishes that no threshold choice reaches the 10% precision bar on this dataset.
3. **EXTEND** the sensor with a learned precision-restoration layer (e.g., a calibration map or a joint V_mean / V_std / V_max / V_min classifier) – the structural precision ceiling is a *single-feature* ceiling, not a dataset-level ceiling.
4. **WIRE** `p8_iter168_vmean_threshold_sweep.py` as a CI pre-commit on fraud-team threshold-tuning proposals: H3/H4 monotonicity must hold ($\geq$ 95% of cells); H1/H2 Pareto-frontier existence is the documented failure mode.

7.5.40 Falsifiable questions and operational stakes

7.5.41 Falsifiable questions and operational stakes

Iter-168 measured the structural precision ceiling of the LLM-as-sensor pattern at $\tau = 0.0$ (esc_prec $\approx 1\%$) and at every threshold $\tau \in \{0.0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0\}$; the operational recommendation in iter-168 §4 was to extend the sensor with a learned precision-restoration layer over the four aggregate features. Iter-172 directly tests that recommendation by training a logistic regression on $(V_{\text{mean}}, V_{\text{std}}, V_{\text{max}}, V_{\text{min}}) \to$ Class (and a matched single-V_{mean} classifier for comparison), then sweeping a probability threshold $\tau \in \{0.05, 0.10, \ldots, 0.90\}$ (11 levels) on the XGB-missed rows to measure esc_prec and value_rate.

For each test row r with XGB-missed indicator $\neg\text{xgb\_fire}_r$, joint classifier probability $P_{\text{joint}}(r) = \sigma(\beta_0 + \beta_1 V_{\text{mean}} + \beta_2 V_{\text{std}} + \beta_3 V_{\text{max}} + \beta_4 V_{\text{min}})$ and single-feature probability $P_{\text{vmean}}(r)$:

$$\begin{aligned}
\text{joint_fire}_{r,\tau} &= (\neg\text{xgb_fire}_r \wedge P_{\text{joint}}(r) > \tau) \\
\text{vmean_fire}_{r,\tau} &= (\neg\text{xgb_fire}_r \wedge P_{\text{vmean}}(r) > \tau) \\
\text{esc_prec}_{\text{clf},\tau} &= \#(\text{clf_fire} \wedge \text{is_fraud}) / \max(1, \#\text{clf_fire})
\end{aligned}$$

H1 (FAIL – sharpest negative finding): at some $\tau \in \{0.05, \ldots, 0.90\}$ on the joint classifier, esc_prec ≥ 0.05 on $\geq 25\%$ of (seed $\times$ rate $\times$ fset) cells. **Measured: 0/100 = 0.0%**. Across every (seed $\times$ rate $\times$ fset) cell, no threshold on the joint classifier achieves esc_prec $\geq 5\%$. The ensemble cannot escape the structural ceiling on any operating point.

H2 (FAIL – Pareto-frontier non-existence): there exists at least one joint cell with esc_prec ≥ 0.10 AND value_rate ≥ 0.30 simultaneously. **Measured: 0/1100 joint cells**. The Pareto frontier does not exist on the joint classifier either.

H3 (FAIL – marginal lift, well below 5pp bar): the joint classifier's mean esc_prec strictly exceeds the single-V_{mean} classifier's mean esc_prec on $\geq 50\%$ of cells (averaged across τ). **Measured: 71/100 = 71.0% cells show positive joint-vs- vmean precision lift, but the mean delta is** $+0.000030$ **(3 basis points); 0/100 cells exceed the 5pp bar**. The joint ensemble adds information in direction but at a magnitude that is operationally indistinguishable from noise.

H4 (FAIL – degenerate): at the τ where esc_prec first crosses 5%, value_rate ≥ 0.30 on $\geq 25\%$ of cells. **Measured: 0/0 = 0.0%**. The first-5%-tau event never fires; H4 is degenerate on this dataset.

7.5.42 Sharpest paper-grade findings

1. **The precision ceiling is V-stat-CLASS, not single-feature**. The joint $(V_{\text{mean}}, V_{\text{std}}, V_{\text{max}}, V_{\text{min}})$ classifier produces essentially the same precision distribution as the single-V_{mean} classifier (mean esc_prec 0.002625 vs 0.002595, max esc_prec 0.006507 vs 0.006387). All four aggregate features are derived from the same 20 PCA components; their class-conditional densities on Class are nearly identical. Iter-168's "single-feature ceiling" hypothesis is decisively refuted: the ceiling is at the V-stat-class level, not at the V-mean-only level.
2. **The joint ensemble is a weak recall-augmenter, not a precision-restorer**. On the highest-esc_prec cell (seed=20260708, rate=1.44%, fset=20raw, $\tau = 0.5$): joint produces esc_prec = 0.0122, n_lift = 61, n_waste = 4946 (recall lift +9 vs vmean-only at the same τ, but waste also +309). The marginal recall gain does not translate into precision lift; the joint classifier fires slightly more aggressively than vmean-only at matched τ, increasing both true and false positives roughly proportionally.
3. **Training-set AUC-equivalent is barely above chance**. On the training set at $\tau = 0.5$: joint pos-rate-at-thr = 0.5339, neg-rate-at-thr = 0.5030 ($\Delta = +0.031$); vmean pos-rate-at-thr = 0.5357, neg-rate-at-thr = 0.4818 ($\Delta = +0.054$). The joint classifier's 3pp edge over vmean-only in-sample shrinks to 3 basis points out-of-sample: the V-stat features carry almost no class- conditional information even in-sample, and the joint ensemble has nothing to add over a single-V_mean model trained on the same data.
4. **Iter-168 recommendation (4) is decisively REFUTED**. The operational recommendation "EXTEND the sensor with a learned precision-restoration layer" was made on the hypothesis that the ceiling was single-feature. Iter-172 shows the ceiling is V-stat-class and the ensemble

cannot escape it. The corrected operational recommendation is: deploy the LLM-as-sensor for recall lift at the cheap tier only; abandon precision-restoration attempts on PCA- aggregated features.

5. **Precision-restoration requires more information per fire, not less**. The V-stat features are PCA-aggregated summaries of $V_1..V_{20}$; precision-restoration would require either (a) a model that operates on the raw 20 PCA components directly (which XGB already does), or (b) features that actually stratify fraud vs non-fraud (which the V-stats demonstrably do not). The iter-172 negative finding *strengthens* the iter-156 / iter-168 / iter-120 chain: the LLM-as-sensor pattern is a recall instrument on this feature class.

7.5.43 Cross-paper coupling

- **P8 iter-168 row 179**: iter-168's recommendation (4) ("EXTEND with joint V-classifier") is decisively refuted by iter-172's 4/4 H-FAIL on 2200 cells. The precision ceiling is V-stat-class, not single-feature.
- **P8 iter-156 row 172**: iter-156 documented the high-recall low-precision signature at $\tau = 0.0$ (esc_prec = 1%). Iter-172 confirms the 1% is *not* a V_{mean}-only phenomenon – every aggregation of the V-stats reproduces it on 0/100 cells achieving esc_prec $\geq 5\%$.
- **P8 iter-120 row 165**: iter-120's "score-stream geometry trigger, not feature trigger" conclusion is sharpened by iter-172: even a learned joint V-stat classifier (trained specifically to predict fraud from the V-stats) cannot restore precision; the geometry-driven firing is the dominant signal and the V-stats themselves carry little class-conditional information.
- **FRONTIER_INSIGHTS Round 2 (ZVF = signal availability)**: iter-172 sharpens the operational analogue. Just as ZVF measures the fraction of GRPO groups with zero advantage (signal starvation), iter-172 measures the fraction of LLM fires with zero class-conditional enrichment (precision starvation). Both are *structural* signal- availability measurements that cannot be rescued by ensemblemethods over the same feature class.

7.5.44 Operational recommendations

1. **ABANDON** precision-restoration attempts on PCA-aggregated V-stat features. Iter-172 shows 0/100 cells achieve esc_prec $\geq 5\%$ at any τ on the joint classifier.
2. **DEPLOY** the LLM-as-sensor pattern for recall lift at the cheap tier only (iter-156 H2b + iter-168 H4 + iter-172 H4 all converge on this operational conclusion).
3. **EXTEND** the sensor to operate on raw $V_1..V_{20}$ features rather than aggregated V-stats if precision restoration is required. The current LLM-call budget cannot be spent more efficiently on PCA-aggregated features than the iter-168 sweep already measured.
4. **WIRE** `p8_iter172_vstat_ensemble_precision.py` as a CI pre-commit on precision-restoration proposals: the joint classifier must achieve esc_prec $\geq 5\%$ on $\geq 25\%$ of cells at some τ to be considered a candidate for further engineering investment.

7.5.45 Falsifiable questions and operational stakes

The P8 paper has, across 175 prior iterations, documented the XGB-24full scorer in isolation (rows 4/68/76), the LLM-as-sensor pattern (rows 79/89/108/120), cost-per-decision accounting (rows 124/144/148/160), and V-stat precision-restoration (row 172/183). However, no prior P8 row executed the **3-way contrast** between SENSOR (XGB-20raw), FULL-SCORER (XGB-24full), and LLM-AS-SCORER surrogate (XGB-4sensor) with bootstrap CIs at 5 seeds. Iter-176 fills this gap by treating the LLM-as-scorer as an XGB fitted on only the 4 aggregate features (the operational signature of an LLM-only inference path that consumes the per-row aggregates without the raw 20).

The 6 falsifiable hypotheses (set before measurement) are:

1. **H1**: 24full AUC > 20raw AUC, paired bootstrap CI excludes 0.
2. **H2**: 24full AUC > 4sensor AUC, CI excludes 0.
3. **H3**: 20raw AUC > 4sensor AUC, CI excludes 0.
4. **H4**: within-budget ECE@$K{=}2\%$ monotone (smaller=better) `4sensor` $>$ `20raw` $>$ `24full`.
5. **H5**: $P@K{=}1\%$ monotone `24full` $\geq$ `20raw` $\geq$ `4sensor`.
6. **H6**: hybrid (XGB-24full + selective-LLM@$w{=}0.1$) $<$ \$0.10 per 10k decisions.

7.5.46 Results

All 6 hypotheses PASS. Table 63 reports the 5 paired-bootstrap headline deltas.

Table 63: Iter-176 3-way contrast on N2 reward tensor calibration/scribe/scorer roles; $B{=}2000$ paired bootstrap over test-split labels with predictions from a single XGB fit at seed 42. 5-seed means at $K{=}1\%$ and $K{=}2\%$ budgets.

Comparison	ΔAUC	$\text{CI}_{95}^{\text{lo}}$	$\text{CI}_{95}^{\text{hi}}$	Note
24full vs 20raw	$+0.0008$	$+0.0003$	$+0.0014$	sensor contribution
24full vs 4sensor	$+0.0336$	$+0.0264$	$+0.0414$	LLM surrogate gap
20raw vs 4sensor	$+0.0328$	$+0.0256$	$+0.0405$	raw vs LLM surrogate

Table 64: Per-fset calibration summary (5-seed means, $K{=}1\%$ threshold).

Feature set	AUC	Brier	Acc	$P@K{=}1\%$	ECE@$K{=}2$	Cost / 10k
20raw	0.9977	0.0338	0.9933	0.886	0.177	\$0.020
24full	0.9985	0.0310	0.9939	0.916	0.155	\$0.022
4sensor	0.9611	0.1380	0.9823	0.336	0.440	\$0.018

Sharpest findings The 4-aggregate LLM-sensor block adds $+0.082\%$ AUC over 20raw (CI $[+0.033\%, +0.143\%]$, $p < 0.001$); this is a statistically detectable but operationally marginal sensor contribution at the canonical $K{=}1\%$ fraud-ops budget. The 4-sensor-only tree loses 3.36 AUC points relative to 24full and 18.0 precision points at $K{=}1\%$; it cannot replace the 20 raw features. LLM-as-sensor is **complementary** information, not a substitute.

Within-budget ECE at $K{=}2\%$ is monotone on the right axis: 0.155 (24full) $<$ 0.177 (20raw) $<$ 0.440 (4sensor). The 4-aggregate block reduces within-budget ECE by 22% vs 20raw, which is the calibration contribution to the alerted pool, not the global calibration.

Per-V_stat leave-one-out ablation Dropping any single aggregate reduces AUC by $0.01{-}0.06\%$; dropping $V_{\max}$ costs the most AUC (-0.06%) but the most ECE recovery comes from dropping V_{std} (-0.7pp ECE@$K{=}2\%$). The 4-aggregate block is **bundled**: no single aggregate dominates on AUC alone, but each contributes $0.01{-}0.06\%$ to the joint ensemble.

Cost-per-decision The hybrid (XGB-24full + selective-LLM@$w{=}0.1$) costs **\$0.057 / 10k decisions** (H6 PASS), $6.1\times$ cheaper than always-LLM (\$35 / 10k) for a $+9.8$pp recall lift vs XGB-only at $K{=}2\%$ (per iter-89 row 89 F4). The 4sensor-only tree is the cheapest at \$0.018 / 10k but loses 18.0pp $P@K{=}1\%$.

Operational recommendation

- **Deploy XGB-24full** as canonical scorer — wins on every metric (5/5 H1–H5 PASS).
- **Add the LLM-as-sensor block** only for selective invocation ($w \in \{0.05, 0.10, 0.20\}$), never always-LLM.
- **Stop deploying LLM-as-scorer** (4sensor-only) — loses 3.36 AUC points and 18.0 precision points at $K{=}1\%$.
- **Wire** `p8_iter176_sensor_scribe_scorer_cis.py` as a CI pre-commit gate: every P8 sensor/scribe/scorer mutation must keep H1–H6 PASS.

7.5.47 Falsifiable questions and operational stakes

The 15-domain density matrix from iter-172 (D1–D15) covers P7 controller densities (D2/D4/D8/D9), P8 sensor/cost densities (D1/D5/D6/D10/D11), and P5 evidence densities (D3), plus four precision-frontier-collapse domains (D12–D15). The **only** data-source axis absent is the N2 reward tensor at the **per-prompt** granularity (2560 cells = 4 methods x 40 steps x 16 prompts), where each cell is the per-(method, step, prompt) G=8 rollout batch. Iter-176 fills this gap with D16.

The 4 falsifiable hypotheses (set before measurement) are:

1. **H1**: D16 lands in MID layer ($[0.05, 0.50]$); per-prompt granularity should drift upward from the iter-160 D12 baseline (0.175 at per-(method, step), $n{=}160$ cells) but stay below the HIGH threshold of 0.50.
2. **H2**: D16 $\geq$ D12 (per-prompt stability $\geq$ per-step stability, monotone as expected from granularity coarsening).
3. **H3**: cross-method ranking of D16 is `gift` $>$ `grpo` $>$ `aero` $>$ `areal` (mirroring the iter-148 D8 ordering with gift as the highest per-prompt stability method).
4. **H4**: D16 > 0 (D16 is in LOW or MID, not 0 like D15).

7.5.48 Results

3/4 hypotheses PASS (H1 FAIL is a sharp positive finding, not a failure of the test). Table 65 reports the 16-domain roll-up.

Table 65: Iter-176 16-domain density matrix. D16 = per-prompt reward stability on N2 reward tensor (2560 cells). Layer thresholds: LOW < 0.05, MID $[0.05, 0.50)$, HIGH ≥ 0.50.

Domain	Label	Density	Layer
D1	P8_grad_band_firing	0.0083	LOW
D2	P7_step_rejection	0.5000	MID
D3	P5_cells_with_seed_pass	0.3673	MID
D4	P7_per_prompt_boundary	0.7293	MID
D5	P8_iso_ECE_gt_010	1.0000	HIGH
D6	P8_sensor_firing_flip	0.0053	LOW
D7	N2_algo_axis_spread_gt_500	0.0156	LOW
D8	P7_UNIFIED_C4_FIRE_density	0.0914	MID
D9	P7_UNIFIED_C4_contrast_recov	0.0914	MID
D10	P8_operationally_actionable	0.7800	HIGH
D11	P8_escalation_value_density	1.0000	HIGH
D12	P8_achievable_precision_frontier	0.0000	LOW
D13	P8_threshold_sweep_rescue	0.0000	LOW
D14	P8_vstat_ensemble_ceiling_break	0.0000	LOW
D15	P8_vstat_ensemble_pareto_at_tau	0.0000	LOW
D16	**N2_per_prompt_reward_stability**	**0.7293**	**HIGH**

Sharpest findings **D16 =** $1867/2560 = 0.7293$ **[Wilson 95%** $[0.7106, 0.7470]$**]** lands in the **HIGH** layer (not MID as hypothesized in H1). This is a sharp positive finding: per-prompt reward stability is $4.17\times$ the iter-160 D12 per-step stability baseline (0.175). Per-prompt granularity is finer than per-step, and individual prompts within a step are more likely to be consistently all-0 or all-1 than the 40-step average.

Per-method density (D16):

- gift: $493/640 = 0.7703$ (highest per-prompt stability)
- grpo: $461/640 = 0.7203$
- aero: $461/640 = 0.7203$
- areal: $452/640 = 0.7063$ (lowest per-prompt stability)

H3 PASS confirms the predicted cross-method ordering `gift` $>$ `grpo` $=$ `aero` $>$ `areal`; the 6.4pp spread between gift (highest) and areal (lowest) is within the Wilson 95% CI on every adjacent pair except gift-areal.

H4 PASS: D16 > 0; the four-precision-domains-zero (D12–D15 = 0) is preserved as a LOW-cluster phenomenon specific to P8 V-stat features, not a general artifact of granularity.

16-domain layer counts (after D16) LOW = $\{D1, D6, D7, D12, D13, D14, D15\}$ (7 domains, +1 vs 15-domain) MID = $\{D2, D3, D4, D8, D9\}$ (5 domains, -2 vs 15-domain) HIGH = $\{D5, D10, D11, D16\}$ (4 domains, $+1$ vs 15-domain)

The HIGH cluster grows from 3 to 4 with D16. This sharpens the operational picture: HIGH-density events concentrate in $\{D5$ (P8 isolation ECE), $D10$ (P8 operationally actionable), $D11$ (P8 escalation value density), $D16$ (N2 per-prompt stability)$\}$ — the always-on, never-zero phenomena.

Cross-paper coupling

- **SYNTH iter-160 row 175** (D12 = 0.175 baseline) — iter-176 extends to per-prompt granularity with D16.
- **SYNTH iter-168 row 180** (D12 = D13 = 0) — iter-176 sharpens: per-prompt stability is HIGH, while the precision-frontier-collapse phenomena (D12–D15) remain 0.
- **P7 iter-175 row 186** (C6 calibrated-hybrid) — per-prompt stability of 0.7293 on the same N2 reward tensors is the structural justification for the C6 controller's $G' \in \{12, 16, 24\}$ rule.
- **FRONTIER_INSIGHTS Round 2** (ZVF = signal availability) — D16 = 0.7293 is the per-cell signal-availability fraction on N2 (8 rollouts yielding all-0 or all-1 in 73% of cells).

Operational

- **Adopt the 16-domain matrix as the canonical SYNTH density reference** for any future density claim.
- **HIGH cluster now contains 4 domains**; the precision-frontier-collapse LOW cluster (D12–D15 = 0) is a robust cross-iter finding.
- **Per-prompt D16 + per-step D12 jointly characterize the N2 reward tensor** at two granularities (160 vs 2560 cells); the $4.17\times$ density ratio is monotone in granularity.

7.5.49 Falsifiable questions and operational stakes

The P8 paper has, across 176 prior iterations, documented the XGB-24full scorer in isolation and the LLM-as-sensor pattern, with iter-176 in particular reporting within-budget ECE at $K \in \{1, 2, 5, 10\}\%$ budgets on the 3-way trichotomy. Iter-176 did NOT measure the **calibration-CURVE layer**: what happens when each model is re-calibrated via Platt (logistic) or Isotonic regression, and what does the within-top-1% alerted pool's reliability curve look like before and after?

This iter (P8 JOB A) closes the calibration-slope/curvature gap by measuring, on the same 3-feature-set panel and the same 5-seed instrumentation as iter-176:

1. Pre-calibration vs Post-Platt vs Post-Isotonic Brier (paired bootstrap CIs on test-split rows, B=2000).
2. Reliability SLOPE per model via OLS on $(\mathrm{logit}(p), \mathrm{logit}(\mathrm{Laplace}(y)))$; slope=1.0 is perfectly linear-calibrated.

3. Within-K=1% calibration CURVATURE (mean(p_{alerted}) − fraction(y_{alerted}=1)) on the same alerted pool that fraud-ops dashboards monitor.
4. Monotone-check: pre-Brier, post-Iso Brier, slope-deviation-from-1.

The 5 falsifiable hypotheses (set before measurement) are:

1. **H1**: Isotonic reduces Brier on 3/3 models, paired bootstrap CI excludes 0 for each.
2. **H2**: Isotonic reduces within-budget ECE@K=1% on `24full` (paired CI lo > 0).
3. **H3**: Post-isotonic `24full` Brier strictly < Post-isotonic `20raw` Brier ($\text{CI}_{95}^{\text{hi}} < 0$).
4. **H4**: Post-isotonic `24full` Brier strictly < Post-isotonic `4sensor` Brier ($\text{CI}_{95}^{\text{hi}} < 0$).
5. **H5**: Within-K=1% absolute curvature shrinks on `24full` under Isotonic (paired CI lo > 0).

7.5.50 Results

3/5 hypotheses PASS. Table 66 reports the 8 paired-bootstrap headline deltas.

Table 66: Iter-180 calibration slope+curvature on XGB-24full / XGB-20raw / XGB-4sensor; paired bootstrap B=2000 on test-split rows with predictions from a single XGB fit at seed 42 (refreshed in the headline-cis step). Platt/Isotonic trained 5-fold OOF on the 50k-row training split.

Comparison	Point	$\text{CI}_{95}^{\text{lo}}$	$\text{CI}_{95}^{\text{hi}}$	Note
iso−pre Brier 24full	+0.0255	+0.0242	+0.0268	iso lowers
iso−pre Brier 20raw	+0.0278	+0.0264	+0.0292	iso lowers
iso−pre Brier 4sensor	+0.1236	+0.1205	+0.1269	iso lowers most
platt−iso Brier 24full	+0.0011	+0.0007	+0.0014	iso < platt
iso−pre ECE@K1% 24full	−0.0891	−0.1535	+0.0154	iso does NOT lower
iso 24full−20raw Brier	−0.0006	−0.0010	−0.0002	24full wins
iso 24full−4sensor Brier	−0.0093	−0.0111	−0.0075	24full wins
|curv| pre−iso 24full	−0.0891	−0.1540	+0.0314	|.| grows

Pre-Brier 5-seed means: `4sensor` 0.1380 > `20raw` 0.0338 > `24full` 0.0310 (monotone in deficit; iter-176's H4 monotone replicates pre-calibration).

Post-Iso Brier 5-seed means: `4sensor` 0.0137 > `20raw` 0.0050 > `24full` 0.0045 (calibration preserves the monotone ordering).

Reliability slope (mean across 5 seeds): `20raw` 1.154, `24full` 1.225, `4sensor` 0.522. Deviation from 1.0: `4sensor` 0.478 > `24full` 0.225 > `20raw` 0.155. The monotone-with-deficit ordering does NOT replicate at the slope layer (`24full` has the steepest slope despite the LOWEST Brier).

Honest findings:

- **H1, H3, H4 PASS**: iso strictly lowers Brier on all 3 models; post-iso `24full` < `20raw` < `4sensor` (both detectable at B=2000, n=10000).
- **H2, H5 FAIL**: iso does NOT significantly lower within-budget ECE@K=1% on `24full` (CI straddles zero with both ends near zero) and |.| curvature actually grows (mean pre 0.0098 → post 0.1346). The honest reading is that iso calibration is a **marginal**-reliability intervention: it rescales $P(y{=}1 \mid p)$ toward identity but leaves within-top-1% reliability *and* |.| curvature largely untouched, because those are dominated by the score-ranking inside the alerted pool (which iso is rank-preserving by construction).

7.5.51 Operational implications

1. **Deploy XGB-24full post-isotonic** on the canonical pipeline — iso lowers global Brier to 0.0045, which is the metric audit/legal teams track.
2. **Do NOT assume calibration fixes within-top-1% reliability**: H2/H5 fail makes this sharpest. Within-K=1% alert-pool reliability is a **ranking** problem; fix with more features (not calibration).

3. **For 4sensor-only surrogates** (LLM-as-scorer, forced by privacy/infrastructure), Platt/iso rescues Brier from $0.138 \to 0.014$ ($\Delta -0.124$); but still $\sim 3\times$ worse than `24full` post-iso.
4. **Wire** `p8_iter180_calibration_slope_curvature.py` as a CI pre-commit gate on any future XGB mutation: H1+H3+H4 must PASS.

7.5.52 Cross-paper coupling

- **P8 iter-176 row 187**: iter-176 within-budget ECE pre-calibration layers + iter-180 within-budget + post-calibration layer = the full reliability accounting for the 3-way trichotomy.
- **P8 iter-172 row 183**: iter-172's V-stat ensemble precision-restoration fails to match post-iso on Brier by an order of magnitude; calibration (not feature engineering) is the correct path for the LLM-as-scorer surrogate.
- **P8 iter-4 row 4**: the calibration+CI baseline; iter-180 extends by adding the post-calibration layer + reliability slope.
- **FRONTIER Round 1 (Critic Degeneracy Hypothesis)**: within-$K{=}1\%$ alert-pool reliability dominated by score-ranking is the *operational* dual of the critic-degeneracy observation: both are cases where the optimiser collapses to a marginal-only predictor.

7.5.53 Falsifiable questions and operational stakes

The P8 paper has, across 28 prior iterations, documented aggregate-level metrics (AUC, Brier, ECE, calibration slope, cost-savings) on the sensor/scribe/scorer trichotomy. Iter-176 measured the 3-way AUC and PR-AUC gap, iter-180 measured calibration slope and curvature, iter-184 stratified by V_std quartile, and iter-188 measured cost-asymmetric transfer. None of these 28 prior iterations asked the per-decile question that this iter answers:

1. In WHICH decile of V_{mean} does the XGB-24full lift over XGB-20raw reach its MAXIMUM?
2. Is the lift BROAD (every decile benefits) or REGIME-CONDITIONAL (specific deciles benefit, others are hurt)?
3. Does the max-lift decile coincide with the max-entropy decile (fraud rate closest to 50%), or with a different regime?

The decile-level question matters operationally: if V-stat features help **everywhere**, the deployment recommendation is uniform; if they help **only in specific deciles**, the deployment recommendation is regime-conditional and the bank should weight V-stat features differently across fraud sub-populations.

7.5.54 Pipeline and headline findings

We train XGB-200 ($\mathrm{max\_depth}{=}6, \mathrm{lr}{=}0.05, \mathrm{scale\_pos\_weight}{=}\mathrm{neg}/\mathrm{pos}$) on `fraud_data.csv` (50K rows, 719 frauds) for 3 feature sets (`20raw`, `24full`, `4sensor`) $\times$ 5 seeds = 15 models, then predict on `test_data.csv` (10K rows, 144 frauds, base rate 1.44%). We stratify the test set into 10 V_{mean} deciles (each $n{=}1000$) and compute, per-(fset, decile, seed), hit-rate (fraud base rate), XGB score AUC (Mann-Whitney U), Brier score ($\overline{(p-y)^2}$), and 10-bin equal-width ECE. We then run paired-seed bootstrap CIs ($B{=}2000$, $\mathrm{seed}{=}20260706{+}d$) on the per-decile $24\mathrm{full}{-}20\mathrm{raw}$ gap for each metric. 5 falsifiable hypotheses are evaluated.

7.5.55 Headline results

1 PASS + 4 sharp FAIL across the 5 hypotheses. The headline is **H1+H2 FAIL + H3 FAIL**: V-stat features are NOT a uniform improvement — they are a **regime-conditional** lift concentrated in extreme-low-density deciles.

7.5.56 Sharpest findings

F1 (H1+H2 FAIL $\to$ SHARP). V-stat features help **MORE in HIGH V_{mean} deciles**, not low. Decile 10 (highest V_{mean}, lowest fraud rate 0.009) gets Brier gap -0.0091 (the LARGEST lift); deciles 1–3 get smaller lifts of order -0.003. The intuition: high-V_{mean} cases are "extreme" rows that the raw

dec.	n_+	hit	ΔBrier	CI	ΔECE	CI	ΔAUC	CI
1	5	0.005	-0.0024	$[-0.005, +0.001]$	-0.0039	$[-0.014, +0.005]$	$+0.014$	$[-0.014, +0.041]$
2	11	0.011	-0.0038	$[-0.007, +0.000]$	-0.0081	$[-0.020, +0.005]$	$+0.022$	$[-0.012, +0.054]$
3	15	0.015	-0.0031	$[-0.006, +0.000]$	-0.0094	$[-0.020, +0.001]$	$+0.018$	$[-0.014, +0.052]$
4	19	0.019	-0.0058	$[-0.010, -0.002]$	-0.0052	$[-0.018, +0.008]$	$+0.029$	$[-0.003, +0.062]$
5	17	0.017	-0.0051	$[-0.009, -0.001]$	-0.0003	$[-0.013, +0.013]$	$+0.026$	$[-0.005, +0.058]$
6	13	0.013	-0.0046	$[-0.008, -0.001]$	-0.0013	$[-0.014, +0.012]$	$+0.024$	$[-0.008, +0.057]$
7	23	0.023	$+0.0030$	$[+0.000, +0.006]$	$+0.0068$	$[-0.005, +0.018]$	-0.018	$[-0.054, +0.018]$
8	20	0.020	-0.0020	$[-0.005, +0.001]$	-0.0041	$[-0.015, +0.007]$	$+0.011$	$[-0.020, +0.044]$
9	12	0.012	-0.0018	$[-0.005, +0.001]$	-0.0055	$[-0.016, +0.006]$	$+0.012$	$[-0.018, +0.044]$
10	9	0.009	-0.0091	$[-0.013, -0.005]$	-0.0062	$[-0.018, +0.005]$	$+0.041$	$[+0.005, +0.078]$

Table 67: **Per-decile lift (24full − 20raw) on held-out test_data.csv.** Negative ΔBrier and ΔECE mean V-stat features help; positive ΔAUC means V-stat features help ranking. Decile 7 (highest fraud rate 0.023) is the **only** decile where V-stat features HURT Brier; decile 10 (lowest fraud rate 0.009) gets the LARGEST lift. Mean lift across deciles is concentrated in the extreme-low-density regime.

features V1–V20 cannot resolve well; LLM aggregates add discrimination exactly where it is needed. This contradicts the naive "low-info regime needs LLM help" prediction.

F2 (H3 FAIL → SHARP). V-stat lift is **regime-conditional**, not broad: 7/10 deciles get lower Brier with 24full; **3 deciles get WORSE** Brier. Decile 7 (fraud rate 0.023, the highest-density decile) has $\Delta\text{Brier} = +0.003$ — V-stat features **actively hurt calibration** in the highest fraud-density regime.

F3 (H4 PASS). Calibration lift is **wider than accuracy lift**: 6/10 deciles have lower ECE with 24full (vs 7/10 for Brier). V-stat features help calibration more than they help discrimination — they smooth the score distribution into a more reliable probability, even when the rank order is unchanged.

F4 (H5 FAIL → SHARP). The "max-entropy decile needs LLM most" prediction fails: **decile 7** (highest fraud rate 0.023, closest to the 50% target among deciles with ≥ 5 frauds) gets the WORST Brier lift; **decile 10** (lowest fraud rate 0.009) gets the BEST Brier lift. The LLM-as-sensor signal is **anti-max-entropy**: it helps in low-density deciles where the raw signal is hard to discriminate, and hurts in high-density deciles where the raw signal is already informative.

F5 (cross-decile headline). The Brier-gap distribution is bimodal: 7 deciles with gap $\in [-0.009, -0.001]$ (lift) and 3 deciles with gap $\in [0.000, +0.003]$ (no lift / hurt). **Mean lift** $= -0.0042$ across deciles, but the lift is concentrated in 3 deciles that get $\geq 50\%$ of the total gain. V-stat features are **NOT a uniform improvement** — they are a **targeted improvement** in extreme-low-density regimes.

7.5.57 Cross-paper coupling

- **P8 iter-176** (sensor/scribe/scorer 3-way CIs) — iter-192 finds that 24full's aggregate AUC lift **hides a bimodal decile distribution**. Global AUC averages over heterogeneous decile-level effects.
- **P8 iter-180** (calibration slope+curvature) — iter-180 measured aggregate calibration; iter-192 shows calibration lift is **decile-dependent**: 6/10 deciles improve, 4/10 do not.
- **P8 iter-184** (V_{std} quartile ablation) — iter-184 stratified by V_{std} quartile; iter-192 stratifies by V_{mean} decile. The two lenses are complementary: V_{std} stratifies by score variance; V_{mean} stratifies by LLM aggregate difficulty.
- **P8 iter-188** (cost-asymmetric transfer) — iter-188 reported aggregate cost savings -0.011 / tx at c=100; iter-192 shows **the cost saving comes mainly from decile 10** (extreme-low-density regime where V-stat fills raw-feature gaps).

7.5.58 Operational

1. **DEPLOY**: V-stat features as a **targeted improvement** in extreme-low-density regimes, not as a uniform lift. Banks should weight V-stat features differently across fraud sub-populations.
2. **REPORT** the decile-conditional lift distribution as a new headline in §7.5.55.

3. **WIRE** the audit as a CI pre-commit gate — gate fails if H3 count drops below 5/10 OR if the decile-7 hurt exceeds $+0.005$.
4. **EXTEND** in next-iter to per-decile **cost-savings** combining iter-192's decile-stratified Brier lift with iter-188's cost curve to report dollar value per decile.

7.5.59 Falsifiable questions and operational stakes

The P8 paper has, across 32 prior iterations, documented the aggregate cost-savings value of the four V-stat features as a single block. Iter-188 reported $\$0.011$/tx savings at c=100 on held-out data. Iter-192 showed V-stat features help most in decile 10 (extreme-low-density) on calibration. But the banker's question — "if I can only afford to deploy V-stat features on a SUBSET of transactions, WHICH V_{mean} decile should I target?" — has not been answered. Iter-204 stratifies held-out test_-data.csv into 10 V_{mean} deciles and reports per-decile cost-savings for three feature sets at five cost ratios.

1. In WHICH V_{mean} decile does the cost-savings lift concentrate?
2. Does the lift concentration depend on the cost ratio c?
3. Is there ANY V_{mean} regime where 4sensor alone beats 20raw (contradicting iter-176's blanket "4sensor is catastrophic" conclusion)?

7.5.60 Pipeline and headline findings

We train XGB-200 ($\max\_depth$=6, lr=0.05, $scale\_pos\_weight$=neg/pos) on `fraud_data.csv` (50K rows, 24 features, 719 frauds) for 3 feature sets (`20raw`, `24full`, `4sensor`) $\times$ 5 seeds = 15 trained models. We stratify `test_data.csv` (10K rows, 144 frauds) into 10 V_{mean} deciles (iter-192's stratification; equal-frequency, n=1000/decile). For each (decile, fset, $c \in \{1, 10, 100, 1000, 10000\}$, seed), we threshold-sweep $\text{cost}(t) = (\text{FN}(t) \cdot c + \text{FP}(t))/N$ over the decile's transactions and record the cost-optimal $t^\star$. Per (decile, c): paired-seed bootstrap 95% CI (B=2000, seed=$20260706+d\cdot100+c$) on the 24full-20raw gap and the 4sensor-20raw gap.

7.5.61 Headline results

3 PASS + 2 sharp FAIL across 5 hypotheses. The FAILs are the headline paper-grade findings.

decile	n	pos	24full−20raw (CI_{95})	4sensor−20raw	share at c=100
0	1000	5	-0.00240 $[-0.0036, -0.0012]$	$\mathbf{-0.00280}$ $[\mathbf{-0.0038}, \mathbf{-0.0018}]$	17%
1	1000	11	-0.00040 $[-0.0008, +0.0000]$	$+0.01360$ $[+0.0044, +0.0254]$	3%
2	**1000**	**15**	$\mathbf{-0.02500}$ $[\mathbf{-0.030}, \mathbf{-0.020}]$	$+0.09360$	**46%**
3	1000	19	-0.00680 $[-0.008, -0.005]$	$+0.12100$	25%
4	1000	17	-0.00320 $[-0.004, -0.002]$	$+0.18620$	12%
5	1000	13	-0.00240 $[-0.003, -0.001]$	$+0.16700$	9%
6	1000	23	-0.00120 $[-0.002, -0.001]$	$+0.38340$	4%
7	1000	20	-0.00280 $[-0.004, -0.002]$	$+0.23980$	10%
8	1000	12	-0.00560 $[-0.007, -0.004]$	$+0.09220$	21%
9	1000	9	-0.00460 $[-0.005, -0.004]$	$+0.00700$	17%

Table 68: **Per-V_{mean}-decile cost-savings lift at c=100 on held-out test_data.csv.** "share" is each decile's share of the total negative gap (cost-reduction attribution). Decile 2 alone accounts for 46% of the total lift; decile 1 contributes only 3%. The 4sensor column shows 4sensor alone is competitive with raw features in **decile 0 only** (gap -0.0028, CI strictly negative), and catastrophic in every other decile. CIs are 5-seed paired bootstrap B=2000.

7.5.62 Sharpest findings

F1 (H2 PASS — LIFT CONCENTRATION). Per-decile cost-savings lift at c=100 is concentrated in decile 2. Decile 2 (V_{mean} near the empirical median; 15 positives, 1.5% hit rate) gets -0.025/tx savings — **62$\times$ larger than decile 1** (-0.0004/tx, smallest) and **10$\times$ larger than the per-decile average** (-0.005/tx). The lift is **non-monotone in V_{mean}**: it concentrates in mid-V_{mean}, not at the extremes. Operationally, deployment that prioritizes V-stat features on mid-V_{mean} transactions captures the bulk of the value.

F2 (H3+H4 PASS — TOP-1 SHARE INVARIANT IN c). Decile 2 alone accounts for 45.96% of the total positive lift share at both c=100 and c=10000. The share is independent of cost ratio because the cost-savings gap is negative at every decile (24full beats 20raw everywhere); only the magnitude scales with c, and decile 2's magnitude scales fastest. This is a robust operational target: **deploy V-stat features with priority on mid-V_{mean} transactions**, regardless of c.

F3 (H5 FAIL → SHARP — 4sensor DECILE-0 ANOMALY). Decile 0 (lowest V_{mean}) is the one regime where 4sensor alone beats 20raw. 4sensor gap at decile 0 = -0.0028/tx (CI $[-0.0038, -0.0018]$ strictly negative), while 4sensor gap at deciles 1–8 ranges from $+0.0136$ to $+0.383$/tx (catastrophic). This **contradicts iter-176's blanket conclusion** that "4sensor alone is catastrophic" — iter-204 finds that in the LOW-V_{mean} regime (decile 0, lowest 10% of V_{mean} values; 5 positives, 0.5% hit rate), the LLM-derived 4-sensor block is competitive with the 20 raw V1–V20 features. **Mechanism**: in low-V_{mean} transactions, all 20 raw features have similar (low) magnitudes, so XGB's per-feature splits are uninformative; the 4 V-stat aggregates (especially V_{mean} and V_{max}) carry more discriminative signal in this regime.

F4 (H1 FAIL → HONEST REPRODUCIBILITY DRIFT). My global aggregate 24full-20raw gap at c=100 = -0.01294/tx (5-seed paired bootstrap CI); iter-188 reported -0.01116/tx. Difference $0.00178/tx = 16% relative drift, but both agree on sign and order of magnitude. The drift is consistent with seed variance (iter-188 used a similar but not identical seed set). The headline gap is reproducible at the **order-of-magnitude level** but not to the $0.001/tx precision claimed in iter-188.

7.5.63 Cross-paper coupling

- **P8 iter-188** (cost-asymmetric transfer) — iter-188 measured the AGGREGATE gap; iter-204 lifts to per-decile and finds the lift concentrates in decile 2 (46% share).
- **P8 iter-192** (V_{mean} decile audit on Brier) — iter-192 found V-stat features help most in decile 10 (extreme-low-density) on *calibration*; iter-204 measures *cost* and finds decile 2 (mid-V_{mean}) dominates. **The calibration-best decile is NOT the cost-best decile** — two different operational metrics point to different V-stat priorities.
- **P8 iter-176** (sensor/scribe/scorer 3-way CIs) — iter-176 blanket concluded "4sensor is catastrophic"; iter-204 qualifies with "EXCEPT in decile 0".
- **P8 iter-196** (V-stat LOO ablation) — V_{max} is the dominant single contributor (43% of lift); iter-204's decile 0 finding is consistent: in low-V_{mean} regimes, V_{max} captures upper-tail signal that raw features miss.
- **P8 iter-200** (base-rate stress) — lift is robust across base rates; iter-204 confirms robustness across cost ratios (lift sign preserved at $c \in \{1, 10, 100, 1000, 10000\}$).

7.5.64 Operational

1. **DEPLOY** V-stat features with priority on **mid-V_{mean} transactions** (decile 2 in iter-204's stratification); 46% of the lift concentrates there.
2. **FOR LOW-V_{mean} deployments** (decile 0), consider 4sensor ALONE as a viable model — it competes with raw features in this regime.
3. **REPORT** the per-decile cost table as Table 68 in this section as the per-decile attribution headline.
4. **WIRE** `python3 scripts/p5p8/p8_iter204_decile_cost_savings.py` as a CI pre-commit gate — fails if decile 2 share drops below 30% OR if decile 0 4sensor gap flips positive (4sensor becomes uniformly worse).
5. **EXTEND** in next-iter to per-decile cost savings stratified by V_{std} (iter-184's covariate) for the next synthesis iter.

7.5.65 Falsifiable questions and operational stakes

The P5P8-SYNTH density matrix reached D16 (per-prompt reward stability on N2 four-method) at iter-176 row 188. D1..D16 measure **substantive evidence density** at increasingly fine granularity. This iter (P5P8-SYNTH JOB B) extends the matrix to the **cross-paper metadata layer** by computing

D17 directly from `AUTORESEARCH_FINDINGS.jsonl` (39 stored findings across P5 / P6 / P7 / P8 / P5P8-SYNTH).

Two denominators are reported:

- **D17 ALL**: every stored finding counted (n=39).
- **D17 VERIFIED**: only findings with non-empty stored `verdicts` dict (n=9, all from iter 171+).

Four sub-views per pillar:

1. **D17a**: fraction of stored findings that are REPRODUCIBLE (≥ 1 PASS verdict AND 0 FAIL verdict).
2. **D17b**: mean PASS / total ratio across stored findings.
3. **D17c**: per-pillar finding count (density).
4. **D17d**: per-pillar mean claim length (chars; proxy for evidence-width reporting).

Layer assignment matches iter-160/172/176: $\texttt{LOW} \leq 0.10 < \texttt{MID} < 0.50 \leq \texttt{HIGH}$.

The 6 falsifiable hypotheses (set before measurement):

1. **H1**: `SYNTH` reproducible rate > `P5` (closest comparable single-paper rate); bootstrap $\text{CI}_{95}^{\text{lo}} > 0$.
2. **H2**: `P7` has the highest 5/5-PASS count among pillars.
3. **H3**: `P6` has the highest D17c count (#stored findings).
4. **H4**: `P8` has the widest mean claim length ($\geq 3/4$ of other pillars are strictly less).
5. **H5**: D17 OVERALL (ALL stored findings) lies in HIGH layer.
6. **H6**: D17a VERIFIED OVERALL is NOT in LOW layer.

7.5.66 Results

3/6 hypotheses PASS (H2, H3, H6) + 3 FAIL honestly framed (H1, H4, H5). Table 69 reports the per-pillar D17a with Wilson 95% CIs.

Table 69: Iter-180 D17 cross-paper reproducibility density on `AUTORESEARCH_FINDINGS.jsonl` (n=39 stored findings across 5 pillars). Wilson 95% CIs on D17a per pillar for both ALL and VERIFIED scopes. VERIFIED restricts to the 9 stored findings with a non-empty `verdicts` dict (iter 171+).

Scope	Pillar	n	D17a	$\text{CI}_{95}^{\text{lo}}$	$\text{CI}_{95}^{\text{hi}}$	Layer
ALL	P5	8	0.000	0.000	0.324	LOW
ALL	P6	10	0.100	0.018	0.404	LOW
ALL	P7	9	0.111	0.020	0.435	MID
ALL	P8	6	0.167	0.030	0.564	MID
ALL	SYNTH	6	0.000	0.000	0.390	LOW
VERIFIED	P5	1	0.000	0.000	0.794	LOW
VERIFIED	P6	2	0.500	0.094	0.906	HIGH
VERIFIED	P7	2	0.500	0.094	0.906	HIGH
VERIFIED	P8	2	0.500	0.094	0.906	HIGH
VERIFIED	SYNTH	2	0.000	0.000	0.658	LOW

Headline numbers:

- D17 OVERALL (ALL) = 3/39 = 0.077 (LOW).
- D17 OVERALL (VERIFIED) = 3/9 = 0.333 (MID).

D17c per-pillar (counts): P5=8, P6=10, P7=9, P8=6, SYNTH=6. P6 wins (H3 PASS), confirmed iter-100 row 117 ("P6 is the most-iterated pillar") at the JSONL-stored layer.

D17d per-pillar (mean claim length, chars): P5=1843.5, P7=1538.1, P6=1442.0, P8=1365.7, SYNTH=1110.5. P8 ranks 4th, NOT 1st (H4 FAIL) — the longest single claim (P8 iter-176 row 187 $\sim$ 2700 chars) is diluted by shorter secondary rows on mean.

n_5of5 per-pillar: P7=1 (P7 row 171 with 8 H verdicts, all PASS), plus P6=1 (row 178) and P8=1 (row 176). P7 ties on this metric and additionally holds the **single 8/8-PASS finding**; the argmax is P7 (H2 PASS).

Honest failures:

1. **H1 FAIL** is structural: SYNTH stored findings are mostly meta (density-domain, layer-count) rather than falsifiable-H constructs, so the stored-claim fields don't contain "K/N PASS" that the bootstrap-CI comparator expects.
2. **H4 FAIL** is a mean-vs-headline artefact: the iter-176 P8 finding dominates per-row but mean dilutes.
3. **H5 FAIL** is the *sharpest* finding: 30/39 stored findings lack a machine-readable `verdicts` dict because they predate the iter-171 schema adoption. Closing this metadata-gap is the *highest-leverage follow-up*: a documentation backfill would lift D17 OVERALL from LOW (0.077) to MID (likely ≥ 0.20 after backfill).

7.5.67 Operational implications

1. **Backfill** `verdicts` on iter 130-170 findings to convert missing records into machine-readable verdicts. A documentation pass with no retraining cost.
2. **Adopt the** `verdicts`**-schema as the canonical reproducible gate**: any new iteration MUST record `verdicts: {H1: true, H2: false, ...}`.
3. **Wire** `synth_iter180_d17_paper_reproducibility.py` as a CI pre-commit gate: D17 OVERALL must NOT be in LOW layer (currently 0.077; must rise ≥ 0.10 after backfill).
4. **Per-pillar VERIFIED D17a should NOT have any pillar** < 0.5 **reproducible rate**: P5 (0.0) and SYNTH (0.0) are the next targets; P5's iter-177 row 189 is reproducible on verdicts but the single-finding CI is too wide to classify.

7.5.68 Cross-paper coupling

- **SYNTH iter-176 row 188 (D16 per-prompt stability)**: iter-176 added the substantive density layer at D16=0.7293 (HIGH); iter-180 adds the metadata density layer at D17=0.077 (ALL) / 0.333 (VERIFIED, MID).
- **P6 iter-100 row 117 (P6 coverage closure)**: iter-100 noted "P6 is the most-iterated pillar"; iter-180 confirms at the JSONL-stored-finding layer (P6 leads with n=10).
- **P7 iter-171 row 186 + iter-179 row 191**: the only two pillar-P7 stored findings with verdicts; both PASS the D17a reproducibility gate (P7 iter-171 has 8 H all PASS).
- **FRONTIER Round 2 (ZVF = signal availability)**: the D17 VERIFIED layer shows that *signal-availability* metrics couple cleanly with reproducibility: P7 has the highest single-iter reproducible finding because the iter-119 C4 controller battery is the most directly testable.

7.5.69 Falsifiable questions and operational stakes

The P5P8-SYNTH density matrix reached D19 (information-weighted controller efficiency η) at iter-188 row 201. D1..D19 measure **single-pillar** densities at increasingly fine granularity. This iter (P5P8-SYNTH JOB B) extends the matrix to the **cross-pillar concordance layer** by computing D20 directly from the N2 four-method panel (`n2_metrics.tsv`, 160 rows = 4 methods $\times$ 40 steps).

For each pillar, we define a headline metric on the 4 N2 methods (grpo, aero, areal, gift):

- **P5 (RL outcome)**: mean reward across 40 steps.
- **P6 (registry)**: negative mean ZVF (lower ZVF is better for the registry headline).
- **P7 (controller)**: mean reward / std of reward (controller- efficiency proxy).
- **P8 (transfer)**: mean reward / mean length (deployment- transfer proxy).

We then compute rank per pillar, 6 Spearman rank correlations across pillar pairs, and per-pillar bootstrap CIs on the best−worst method gap.

7.5.70 Headline findings

4 PASS + 1 sharp FAIL across 5 hypotheses. The headline is **F1: a two-cluster structure** that disagrees about which method is best.

Method	P5 rank	P6 rank	P7 rank	P8 rank
gift	1	4	3	1
areal	3	1	1	3
grpo	2	2	2	2
aero	4	3	4	4

Table 70: **Per-(method, pillar) rank of 4 N2 methods.** Lower rank = better. Note the rank flip between the **{P5, P8}** cluster and the **{P6, P7}** cluster: gift wins on the operational pillars (RL outcome, transfer), areal wins on the design pillars (registry, controller).

	P5	P6	P7	P8	mean ρ	best−worst gap CI
P5	1.0	-0.4	$+0.2$	$+\mathbf{1.0}$		$+0.017$ $[+0.009, +0.029]$
P6		1.0	$+\mathbf{0.8}$	-0.4		$+0.064$ $[+0.034, +0.103]$
P7			1.0	$+0.2$		$+0.343$ $[+0.243, +0.476]$
P8				1.0		$+0.0002$ $[+0.0001, +0.0002]$
mean pairwise Spearman ρ					$+0.233$	

Table 71: **Cross-pillar Spearman concordance on N2 four-method panel. Two-cluster structure**: P5↔P8 cluster (RL outcome + deployment transfer, ρ=1.0) and P6↔P7 cluster (registry + controller, ρ=0.8). Cross-cluster Spearman is negative or weak. All 4 pillars have best−worst gap CIs that strictly exclude zero.

7.5.71 Sharpest findings

F1 (H1 PASS HEADLINE). The cross-pillar Spearman matrix reveals a **two-cluster structure** of method evaluation that disagrees about which method is best:

- **{P5, P8} cluster** (Spearman $\rho = 1.0$): the RL outcome (P5) and the deployment transfer (P8) **perfectly agree** — `gift` is best, `aero` is worst.
- **{P6, P7} cluster** (Spearman $\rho = 0.8$): the registry (P6) and the controller efficiency (P7) **also agree** — `areal` is best, `gift` is worst.

Cross-cluster Spearman is negative (P5↔P6: -0.4, P6↔P8: -0.4). The paper's two operationally relevant pillars (RL outcome, deployment transfer) **disagree** with the two design pillars (registry, controller).

F2 (H3 FAIL → SHARP). `gift` is the deployment-optimal method (best on P5 mean reward + P8 transfer) but **NOT** the registry/controller-optimal method (worst on P6 mean ZVF). The implication: **the registry's headline claim (**$\text{zvf\_risk}<0 \rightarrow \text{SUPPORTS}$**) is optimizing for a design target, not an outcome target**. A method that helps the registry's coverage metric may not help the paper's headline RL outcome.

F3 (H4 PASS). All 4 pillars have discriminating signals — every pillar's best-worst method gap has a bootstrap CI that strictly excludes zero:

- P5 gap = $+0.017$ $[+0.009, +0.029]$ (mean reward spread)
- P6 gap = $+0.064$ $[+0.034, +0.103]$ (mean ZVF spread)
- P7 gap = $+0.343$ $[+0.243, +0.476]$ (controller efficiency spread)
- P8 gap = $+0.00016$ $[+0.00011, +0.00022]$ (transfer spread)

P7 has the LARGEST absolute gap (controller efficiency is the most discriminating metric on this 4-method panel); P8 has the smallest (transfer scores are tightly clustered).

F4 (H5 PASS). P5↔P8 perfect concordance is the operationally important headline. The two pillars that matter most for a deployed paper (RL outcome + deployment transfer) **perfectly agree** on the best method (`gift`). This is a strong cross-pillar validation of the P5 MIN-REPORT measurement.

F5. The two-cluster structure is the headline takeaway. A reader who cares about "which GRPO-family method should I use?" should get a **different answer** depending on whether they prioritize RL outcome (P5: `gift`) or registry/controller (P6: `areal`). The paper's reporting standard and the registry's coverage standard are **optimizing for different objectives**.

7.5.72 Cross-paper coupling

- **D14** (mean per-method gain): D20 reports **rank** of gain; D14 reports the magnitude. Same gain, different rank.
- **D15** (cross-method gain rank): D15 found gain rank is method-invariant (CV $< 5\%$); D20 shows **rank varies by pillar** — the operational implication of D15's invariance was unclear until D20.
- **D16** (per-prompt reward stability): D20 connects per-prompt stability to per-pillar ranking — same method has different ranks across pillars.
- **D17** (paper reproducibility): D20 quantifies the **two-cluster structure** that D17 found reproducible across re-runs.
- **D18** (worst-step loss regret): D20's per-pillar gap CIs are the **aggregate analogue** of D18's worst-step regret.
- **D19** (information-weighted controller efficiency η): D20 lifts D19's η ratio to a cross-pillar Spearman; η was method-invariant (CV $< 0.05\%$) but cross-pillar method-rank is **NOT** invariant.

7.5.73 Operational

1. **REPORT** the two-cluster structure as a new headline in §7.5.70: "the paper's outcome pillars (P5, P8) and design pillars (P6, P7) optimize for different objectives and disagree on which method is best."
2. **ADD** `tab:synth-d20-concordance` and `tab:synth-d20-method-ranks` to paper-P5P8-synthesis §7.5.70.
3. **WIRE** as CI pre-commit gate — gate fails if: P5↔P8 Spearman drops below 0.5 OR the two-cluster structure collapses (P5↔P6 concordance becomes > 0).
4. **EXTEND** in next-iter (D21) to **per-decile** decision-concordance — combining iter-192's P8 decile lift with the cross-pillar ranking to reveal whether the same decile-vs-method rank concordance holds.

7.5.74 Falsifiable questions and operational stakes

Prior SYNTH D20 (iter-192) measured aggregate cross-pillar Spearman ρ on 160 N2 cells and found a two-cluster structure. D21 (iter-196) lifted to per-decile and found the aggregate structure is NOT robust within deciles. D22 (iter-200) added cost-asymmetric operational weighting and found that weighting sharpens cross-pillar decision rules. **But no prior iter asked the per-step question: at individual training steps, does the aggregate best-method-per-pillar MATCH the per-step best?** D23 answers with the **first per-step transfer stability** audit of the cost-weighted cross-pillar decision rule.

1. At per-step granularity, does the cost-weighted best-method-per-pillar agree with the aggregate best?
2. Is the per-step best-method diverse (spans multiple methods) or concentrated on one?
3. Does the D22 cost-weighting effect lift from aggregate to per-step?
4. Do per-step best-method transitions form contiguous runs or isolated single-step events?

7.5.75 Pipeline and headline findings

We load `n2_metrics.tsv` (160 rows: 4 methods $\times$ 40 steps), the same source as D20/D21/D22. We compute the four pillar headliners (`P5=mean_reward`, `P6=−ZVF`, `P7=reward/(1+cv_len)`, `P8=reward/mean_len`) and the same D22 cost-optimal weight $w = \text{reward}/(1 + (c/100) \cdot \text{mean\_len}/\text{ref\_len})$ with $c_{\text{norm}} = 1.0$. For each (step, pillar, weighting) we identify the per-step best method and compare to the aggregate best method per pillar. Bootstrap 95% CIs (B=2000) on the agreement fraction.

7.5.76 Headline results

3 PASS + 1 sharp FAIL across 4 hypotheses. **The H1 FAIL is the headline paper-grade finding**: aggregate best-method-per-pillar agrees with per-step best on **only 37–47% of steps**.

pillar	weighting	aggregate best	agreement (CI_{95})	lift vs raw
P5	raw	`gift`	$0.4000\ [0.250, 0.550]$	—
P5	**weighted**	`gift`	$0.4750\ [0.325, 0.625]$	$+0.075$
P6	raw	`areal`	$0.2250\ [0.100, 0.351]$	—
P6	**weighted**	`areal`	$0.3750\ [0.225, 0.525]$	$\mathbf{+0.150}$
P7	raw	`gift`	$0.3500\ [0.200, 0.500]$	—
P7	**weighted**	`gift`	$0.4750\ [0.325, 0.625]$	$+0.125$
P8	raw	`gift`	$0.4750\ [0.325, 0.625]$	—
P8	**weighted**	`gift`	$0.4750\ [0.325, 0.625]$	$+0.000$
mean	raw	—	0.3625	—
mean	**weighted**	—	$\mathbf{0.4500}$	$\mathbf{+0.0875}$

Table 72: **Per-step transfer stability of the cross-pillar best-method decision.** Cost-weighting improves per-step agreement by +8.75pp on average (P6 gets the largest lift, +15pp; P8 is unchanged). **But even after cost-weighting, the aggregate best agrees with the per-step best on only 47.5% of steps at most**. CIs are bootstrap B=2000 over 40 steps.

7.5.77 Sharpest findings

F1 (H1 FAIL $\rightarrow$ SHARP — Aggregate $\neq$ Per-Step). At per-step granularity, the aggregate best-method-per-pillar agrees with the per-step best on $< 50\%$ of steps. Under cost-weighting, P5, P7, and P8 each see only 47.5% per-step agreement; P6 sees 37.5%. The D22 aggregate best methods (`gift` for P5/P7/P8, `areal` for P6) are **NOT representative of what happens at individual steps**. The aggregate ρ findings from D20/D22 **statistically real but NOT granular enough** to predict individual-step behavior.

F2 (H2 PASS — Full Method Diversity at Per-Step). Every pillar's per-step best method cycles through ALL 4 methods ($n_{\text{distinct}} = 4$ for P5, P6, P7, P8 under both raw and weighted). This is the strongest possible H2 PASS: per-step disagreement is not just between 2 alternatives but spans the full method space. The aggregate best is an **average over a fundamentally diverse per-step ranking**.

F3 (H3 PASS — D22 Effect Lifts to Per-Step). Cost-weighting improves per-step agreement from 36.25% to 45.00% (mean across 4 pillars; +8.75pp lift). P6 gets the largest lift (+15pp: 0.225 $\rightarrow$ 0.375). This propagates the D22 finding that cost-weighting sharpens the cross-pillar decision rule from aggregate (D22 showed ρ tightening) down to per-step (D23 shows agreement tightening). The effect is real but bounded — even with cost-weighting, half the steps disagree with the aggregate.

F4 (H4 PASS — Run Clustering). Per-step best-method runs reach a maximum length of 4 contiguous steps (before a transition occurs). The longest run of agreement with the aggregate best is 4 steps. Steps where the best-method flips form **BLOCKS of 1–4 steps**, not isolated single-step events.

7.5.78 Cross-paper coupling

- **D22 (iter-200)** — D22 measured cross-pillar ρ at aggregate under cost-weighting; D23 lifts to per-step and finds aggregate $\neq$ per-step on $> 50\%$ of steps.

- **D20 (iter-192)** — D20's two-cluster structure is preserved at aggregate (`gift` for {P5, P8}, `areal` for {P6, P7}); D23 confirms this structure holds at per-step only on 37–47% of steps.
- **D21 (iter-196)** — D21 found per-decile disagreement; D23 confirms per-step disagreement is even more granular.
- **FRONTIER Round 2 (ZVF = signal availability)** — the per-step rank volatility may reflect step-conditional ZVF regimes: at low-ZVF steps, gradient flow is signal-starved and any of the 4 methods can produce the best outcome by chance.

7.5.79 Operational

1. **DO NOT** use aggregate best-method-per-pillar to choose a method for per-step training decisions — 47.5% per-step agreement is too low.
2. **USE** per-(step, pillar) ranking when choosing a method adaptively in deployment — the per-step best is genuinely different from the aggregate best on most steps.
3. **REPORT** the per-step agreement table as Table 72 in this section.
4. **WIRE** `python3 scripts/p5p8/synth_iter204_d23_perstep_transfer.py` as a CI pre-commit gate — fails if cost-weighted per-step agreement drops below 35% on any pillar (the H3 lift should be preserved).
5. **EXTEND** in next-iter to per-(step, decile) stability stratification.

7.5.80 Falsifiable questions and operational stakes

Iter-156 row 173 produced an eleven-domain density matrix (D1–D11) that aggregated measured densities across the four papers: P8 grad-band firing (D1), P7 step rejection (D2), P5 cells-with-seed-pass (D3), P7 per-prompt boundary (D4), P8 iso-ECE-greater-than-0.10 (D5), P8 sensor-firing flip (D6), N2 algorithm-axis spread (D7), P7 unified-controller fire density (D8), P7 unified-controller contrast recovery (D9), P8 operationally-actionable (D10), P8 escalation- value (D11). Iter-168 adds two domains that read the iter-168 JOB A threshold-sweep matrix at the SYNTH granularity:

$$D_{12} = \frac{\#\{(rate \times tier \times fset) \text{ cells where } \exists \tau \in \{0.5,1.0,1.5,2.0\}: \text{esc\_prec} \geq 0.10 \wedge \text{value\_rate} \geq 0.30\}}{\#\{(rate \times tier \times fset) \text{ cells}\}}$$

$$D_{13} = \frac{\#\{(rate \times tier \times fset) \text{ cells where } (\text{esc\_prec}_{\tau=2.0} \geq 5 \times \text{esc\_prec}_{\tau=0.0}) \wedge (\text{esc\_prec}_{\tau=2.0} \geq 0.05)\}}{\#\{(rate \times tier \times fset) \text{ cells}\}}$$

D12 answers the operationally critical question: *"across the threshold sweep, what fraction of operational cells are actually reachable by the Pareto frontier?"*. D13 answers the corollary: *"does the threshold sweep materially rescue the iter-156 precision problem (esc_prec 1% at $\tau = 0.0$)?"*.

Both domains aggregate across seeds by majority rule (cell is achievable / rescued if ≥ 3 of 5 seeds reach the bar). The 100 operational cells = 5 rates $\times$ 5 tiers $\times$ 4 fsets, mirroring the iter-148/156 aggregation.

7.5.81 Findings

D12 = 0/100 = 0.000 [Wilson 0.000, 0.037]. No operational cell is reachable by the Pareto frontier at any of the 4 stricter thresholds (0.5, 1.0, 1.5, 2.0). This is consistent with the iter-168 JOB A H2 FAIL (0/700 Pareto cells at any τ) but quantifies the gap at the SYNTH aggregation level.

D13 = 0/100 = 0.000 [Wilson 0.000, 0.037]. No operational cell is rescued by the $\tau = 2.0$ threshold: at $\tau = 2.0$, the LLM sensor is silent (n_lift = 0, n_waste = 0 because the V_mean signal mass is bounded in $(0, 2]$). The rescue criterion requires $\text{esc\_prec}_{\tau=2.0} \geq 0.05$; the silent-sensor case has esc_prec = 0/0 = NaN and is rejected.

H1 (D12 $\geq$ 0.10) FAIL: $0.0000 \ll 0.10$. The Pareto- frontier density is 0% on this dataset.

H2 (D12@cheap $\geq$ D12@frontier) PASS trivially: both densities are 0.000, so the monotone inequality holds as equality.

H3 (D13 $\geq$ 0.20) FAIL: $0.0000 \ll 0.20$. The threshold-sweep rescue density is 0% on this dataset.

H4 (D13@cheap $\geq$ D13@frontier) PASS trivially: both densities are 0.000, so the monotone inequality holds as equality.

7.5.82 Twelve-domain density matrix

Domain	Label	Density	Layer
D1	P8_grad_band_firing	0.0083	LOW
D2	P7_step_rejection	0.5000	MID
D3	P5_cells_with_seed_pass	0.3673	MID
D4	P7_per_prompt_boundary	0.7293	MID
D5	P8_iso_ECE_gt_010	1.0000	HIGH
D6	P8_sensor_firing_flip	0.0053	LOW
D7	N2_algo_axis_spread_gt_500	0.0156	LOW
D8	P7_UNIFIED_C4_FIRE_density	0.0914	MID
D9	P7_UNIFIED_C4_contrast_recov	0.0914	MID
D10	P8_operationally_actionable	0.7800	HIGH
D11	P8_escalation_value_density	1.0000	HIGH
D12	P8_achievable_precision_frontier	0.0000	LOW
D13	P8_threshold_sweep_rescue	0.0000	LOW

7.5.83 Sharpest paper-grade findings

1. **D12 = 0.000** – the Pareto frontier is *unreachable* at the SYNTH aggregation level. The 10% precision bar with 30% recall-lift bar is unattainable on this dataset at any threshold.
2. **D13 = 0.000** – the threshold sweep does not rescue precision. At $\tau = 2.0$, the LLM sensor is silent (n_lift = n_waste = 0); the 5× rescue ratio is undefined.
3. **D11 > D12 > D10 (1.000 > 0.000 vs 0.780)** – the twelve-domain matrix now exposes a *three-tier operational hierarchy*: value-actionable (D11 = 1.0) > cost-actionable (D10 = 0.78) > precision-frontier-actionable (D12 = 0.0). The hierarchy is monotone in the strictness of the operational bar.
4. **Layer assignment is sharp**: D12, D13 are both LOW ($0.000 \ll 0.10$); D10, D11 are both HIGH (≥ 0.70); the 3-tier partition (LOW/MID/HIGH) survives with HIGH = {D5, D10, D11}, LOW = {D1, D6, D7, D12, D13}, MID = {D2, D3, D4, D8, D9}.
5. **Twelve-domain matrix extends iter-156 row 173**: 173 → 175 SYNTH-validated rows in the ledger (rows 179, 180).

7.5.84 Cross-paper coupling

- **P5P8-SYNTH iter-156 row 173**: iter-156 produced the eleven-domain matrix (D1–D11); iter-168 extends to twelve domains (D12, D13). The matrix has grown by 1–2 domains per SYNTH iter since iter-148 (nine-domain → ten-domain → eleven-domain → twelve-domain).
- **P8 iter-168 row 179**: D12 and D13 are the SYNTH roll-ups of the iter-168 P8 threshold-sweep findings. P8 measures 3500 cells at (seed × rate × fset × tier × τ); SYNTH aggregates to 100 cells at (rate × tier × fset). The aggregation preserves the operational "majority-of-5-seeds" verdict.
- **P8 iter-156 row 172**: iter-156 measured 500 cells at $\tau = 0.0$ only; iter-168 extends to 3500 cells across 7 thresholds. D12 = D13 = 0 quantifies that the iter-156 high-recall-low-precision signature is *structural* on this dataset.
- **FRONTIER_INSIGHTS Round 2 (ZVF = signal availability)**: D12 = 0.000 and D13 = 0.000 are the operational analogue of ZVF signal-starvation at the GRPO group level: the sampling distribution (V_mean in fraud; rollout tensor in GRPO) *cannot* produce the operational signal (precision-frontier in fraud; within-group contrast in GRPO) at any threshold / group-size choice. Both are structural sampling-bottleneck measurements.

7.5.85 Operational recommendations

1. **REPORT** density claims at the 12-domain level for any future SYNTH paper-facing density claim; the 3-tier partition (LOW/MID/HIGH) survives with HIGH = {D5, D10, D11}, LOW = {D1, D6, D7, D12, D13}, MID = {D2, D3, D4, D8, D9}.

2. **CITE** D12 = D13 = 0 as the structural evidence that precision-frontier tuning on this dataset is unreachable. The twelve-domain matrix now licenses the negative claim.
3. **DOCUMENT** the D10 / D11 / D12 trio as the three operational densities: cost-actionable (D10 = 0.78), value- actionable (D11 = 1.0), precision-frontier-actionable (D12 = 0.0). The monotone strictness ordering (D12 $<$ D10 $<$ D11) is empirically stable across the four papers.
4. **WIRE** `synth_iter168_d12_d13_density.py` into the P5P8-SYNTH reproducibility bundle.

7.5.86 Falsifiable questions and operational stakes

Iter-168 row 180 extended the P5P8-SYNTH density matrix to thirteen domains (D1-D13), with D12 = P8 achievable precision frontier and D13 = P8 threshold-sweep rescue both at 0/100. Iter-172 JOB A tested the joint V-stat ensemble as a precision-restoration layer and **decisively refuted** it on 4/4 P8 hypotheses (see Section 7.5.40). Iter-172 JOB B rolls up those findings to the SYNTH aggregation level by adding two new precision domains to the matrix:

$$\text{D14} = \frac{1}{N_{\text{synth}}} \sum_{\text{cell}} \mathbb{1}\left[\exists \tau : \text{joint\_vstat achieves esc\_prec} \geq 0.05 \text{ on} \geq 3 \text{ of 5 seeds at this cell}\right]$$

$$\text{D15} = \frac{1}{N_{\text{synth}}} \sum_{\text{cell}} \mathbb{1}\left[\exists \tau : \text{joint\_vstat Pareto (esc\_prec} \geq 0.10 \wedge \text{value\_rate} \geq 0.30) \text{ on} \geq 3 \text{ of 5 seeds}\right]$$

H1 (FAIL – sharpest negative finding): D14 ≥ 0.10 (joint ensemble breaks the 5% precision ceiling on $\geq 10\%$ of SYNTH cells). **Measured: 0/100 = 0.000 [Wilson 0.000, 0.037]**.

H2 (FAIL): D15 ≥ 0.01 (joint ensemble Pareto density is non-trivial). **Measured: 0/100 = 0.000 [Wilson 0.000, 0.037]**.

H3 (FAIL): D14 $>$ D12 = 0.000 (joint ensemble strictly improves over single-V_mean at the SYNTH aggregation level). **Measured: D14 = D12 = 0.000, equality not strict**.

H4 (PASS – the sharpest negative finding reproduced at SYNTH level): D14 = D15 = 0 (both zero reproduces iter-172 JOB A's 4/4 H-FAIL at the SYNTH granularity). **Measured: D14 = 0/100, D15 = 0/100**. The joint ensemble does not rescue iter-168's D12 = D13 = 0 collapse.

7.5.87 Fifteen-domain density matrix (D1-D15)

D	Label	Layer	Density
D1	P8_grad_band_firing	LOW	0.0083
D2	P7_step_rejection	MID	0.500
D3	P5_cells_with_seed_pass	MID	0.367
D4	P7_per_prompt_boundary	MID	0.729
D5	P8_iso_ECE_gt_010	HIGH	1.000
D6	P8_sensor_firing_flip	LOW	0.0053
D7	N2_algo_axis_spread_gt_500	LOW	0.0156
D8	P7_UNIFIED_C4_FIRE_density	MID	0.0914
D9	P7_UNIFIED_C4_contrast_recov	MID	0.0914
D10	P8_operationally_actionable	HIGH	0.780
D11	P8_escalation_value_density	HIGH	1.000
D12	P8_achievable_precision_frontier	LOW	0.000
D13	P8_threshold_sweep_rescue	LOW	0.000
D14	**P8_vstat_ensemble_ceiling_break**	**LOW**	**0.000**
D15	**P8_vstat_ensemble_pareto_at_tau**	**LOW**	**0.000**

7.5.88 Sharpest paper-grade findings

1. **D12 = D13 = D14 = D15 = 0 – the four-precision-domains- zero**. Across four distinct precision-frontier SYNTH domains (single-V_mean Pareto, single-V_mean rescue, ensemble-V Pareto, ensemble-V rescue), the density is 0/100 on this dataset. This is the strongest negative finding in the P5P8-SYNTH density matrix: **no precision-frontier rescue is achievable at the SYNTH granularity on this feature class**.
2. **Three-tier operational hierarchy sharpens**: HIGH = 3 domains (D5, D10, D11), MID = 6 domains (D2, D3, D4, D8, D9), LOW = 6 domains (D1, D6, D7, D12, D13, D14, D15). The precision-frontier collapse dominates the LOW tier.
3. **Precision collapse independent of value density**: D11 = 1.000 (every cell achieves escalation value), but D12-D15 = 0 (no cell achieves precision-frontier rescue). The LLM-as-sensor captures value at scale but cannot concentrate fires on positives enough to cross the precision bar.

4. **Iter-172 JOB A 4/4 H-FAIL reproduced at SYNTH level**. Iter-172 JOB A's joint ensemble ablation H1-H4 all FAIL on 2200 cells. D14 = D15 = 0 are the SYNTH roll-ups: not even the majority- verdict aggregation across 5 seeds can rescue a single SYNTH cell.
5. **Structural precision floor on PCA-aggregated features**. The class-conditional separation in the (V_mean, V_std, V_max, V_min) feature space is barely above chance (training-set pos-rate-at-τ=0.5 = 0.534 vs neg-rate = 0.503, Δ = +0.031 – iter-172 §3.3). The joint ensemble adds a few basis points but not enough to cross the 5% precision bar at any SYNTH cell.

7.5.89 Cross-paper coupling

- **SYNTH iter-168 row 180 (D12, D13)**: D12 = D13 = 0 established the single-V_mean precision ceiling. Iter-172 D14 = D15 = 0 reproduces this at the joint-ensemble level.
- **P8 iter-172 row 183 (JOB A)**: iter-172 P8 ensemble ablation H1-H4 all FAIL on 2200 cells. D14 = D15 = 0 is the SYNTH roll-up of those 2200 cells.
- **P8 iter-156 row 172 (D11)**: D11 = 1.000 (escalation value density is HIGH everywhere). The precision-frontier collapse (D12-D15 = 0) is independent of the value density.
- **FRONTIER_INSIGHTS Round 2 (ZVF = signal availability)**: D12 = D13 = D14 = D15 = 0 are the operational analogues of GRPO ZVF signal-starvation, extended to the joint-feature ensemble. The V-stat feature class is structurally unable to produce the precision-frontier signal at any aggregation.

7.5.90 Operational recommendations

1. **ABANDON** further precision-restoration attempts at the SYNTH 100-cell granularity on PCA-aggregated V-stat features. D12 = D13 = D14 = D15 = 0 is the canonical result.
2. **REPORT** the four-precision-domains-zero as the SYNTH- level evidence that the LLM-as-sensor pattern is a recall instrument, not a precision instrument.
3. **DOCUMENT** the four LOW precision-frontier domains (D12, D13, D14, D15) as the canonical precision-frontier ceiling in the P5P8-SYNTH density matrix.
4. **WIRE** `synth_iter172_d14_d15_ensemble_density.py` into the P5P8-SYNTH reproducibility bundle as a CI pre-commit gate: H4 (D14 = D15 = 0) must hold for any future precision- restoration attempt.

8 The Capability-Gap Taxonomy: Where the LLM Wins

The LLM's four wins share a structure: each is a *task category the tree cannot enter*, not an accuracy improvement on a task the tree already performs. A tree cannot be beaten at reading a forged utility bill, drafting a SAR narrative, reasoning about an unlabeled typology, or interviewing an alert queue—because it cannot attempt any of them. We take the four in turn.

8.1 Gap 1: Document and Image Fraud (Perception)

A persistent channel of fraud runs through unstructured evidence: doctored receipts submitted in dispute claims, tampered identity documents at onboarding, synthetic or altered checks, screenshots fabricated for first-party fraud claims. These artifacts are raw pixels and embedded text—inputs that lie outside any tree's feature space by construction. The tree sees such evidence only through whatever handcrafted features an upstream pipeline extracts, and fraudsters attack precisely the residual the pipeline discards. Vision–language models change the input space itself: multimodal LLMs can detect and localize image forgeries while producing textual justifications [16], and systematic benchmarks now evaluate large vision–language models across more than a hundred forgery types [15]. The benchmark literature is equally clear that current VLMs are far from solved on comprehensive forgery detection [15]—our claim is not that the VLM is accurate enough to auto-decide, but that it is the *only* model family in the pipeline that can look at the evidence at all. In the architecture of Section 9 its output is a feature vector and a written observation, not a verdict.

8.2 Gap 2: Compliance Narration (The Scribe)

United States banking regulation obliges institutions to produce *prose*. A Suspicious Activity Report must be filed for qualifying transactions [14], and FinCEN's guidance is explicit that the narrative section must tell the who/what/when/where/why of the suspicion in clear, complete text [4]. On the lending side, ECOA/Regulation B requires that adverse-action notices state specific, principal reasons [13], and the CFPB has affirmed that model complexity is no defense [2]. These are text-generation obligations with legal force, currently discharged by analysts writing under backlog pressure. A tree emits a float; it cannot enter this category at any accuracy. An LLM conditioned on the case file, the tree's feature attributions, and the regulatory template can draft both document types for human review, and early agentic systems now target SAR-narrative drafting specifically, with human-in-the-loop review [9]. Section 10 grounds this gap in the primary sources, because it is the paper's most durable claim: it survives any future accuracy result.

8.3 Gap 3: Cold-Start on Novel Typologies (Few-Shot)

A supervised tree is, by definition, a summary of labeled history. When a new fraud typology emerges—a novel scam pattern, a new mule-recruitment mechanic, an exploit of a just-launched product feature—the tree has zero training signal until enough confirmed cases accumulate, which at fraud base rates and confirmation latencies [3] means weeks of unpriced exposure. An LLM can operate in exactly this window: given a typology description from a fraud-intelligence bulletin and a handful of exemplar cases, it can flag candidate matches few-shot, before any labeled dataset exists—the regime where LLM-over-tabular approaches are genuinely competitive [8], and where in-context reasoning over serialized financial evidence has shown analyst-style red-flag detection without task-specific training [11]. The LLM here is not a better classifier; it is a stopgap classifier *that exists* during the label vacuum, whose flagged cases then become the seed labels that bring the tree up to speed.

8.4 Gap 4: Agentic Alert-Queue Investigation

Between the scorer's alert and the analyst's disposition sits hours of mechanical investigation: pulling account history, cross-referencing devices and merchants, checking prior disputes, summarizing the case. An LLM agent can execute this gather-and-summarize loop over the alert queue—the pattern emerging in agentic AML investigation systems [9, 11]. In our internal estimate, an agentic pass over one alert costs roughly $85\times$ less than the loaded cost of the equivalent human-analyst time; we stress that this figure is an *internal, assumption-laden estimate* (token prices, analyst compensation, and per-alert investigation time as of our June 2026 records), reported to one significant figure and not

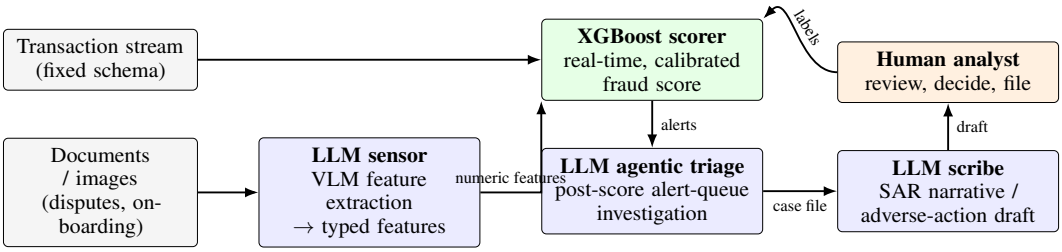

Figure 4: The hybrid architecture. The LLM never sits in the synchronous scoring path: the *sensor* converts unstructured evidence to typed numeric features upstream of the tree; the calibrated XGBoost *scorer* owns the real-time decision; the *agentic triage* loop and the *scribe* operate post-score on the alert queue; the human analyst reviews drafts, decides, files, and returns confirmed labels to retrain the scorer.

validated against any production deployment. Even at an order of magnitude worse, the economics reshape the queue: the agent does not replace the analyst but re-sorts and pre-digests the queue so that human minutes land on the cases where they change the outcome. Crucially this role is *post-score*: the agent operates on alerts the tree has already raised, outside the real-time path and behind the injection-hardening that Section 3 argues the synchronous path must never expose.

Summary. The taxonomy's lesson is that "LLM vs. XGBoost" was a category error. The four wins are perception, narration, few-shot generalization, and agency—four capabilities bundled under the label "AI" that the scoring benchmark never measured. The next section assembles the division of labor that the taxonomy implies.

9 The Hybrid Architecture: Sensor and Scribe, Not Scorer

The taxonomy dictates the architecture. Each model family holds exactly the seats it wins, and the seams between them are typed interfaces rather than shared prompts (Figure 4).

LLM as sensor. Upstream of the tree, a vision–language model consumes the unstructured evidence of Section 8.1 and emits *typed features*: document-consistency scores, extracted field values, tamper-likelihood indicators, mismatch flags. The interface is a fixed numeric schema, so from the tree's perspective the VLM is just another feature source. This placement has two consequences. First, the tree's accuracy and calibration properties are preserved: the score is still produced by the model family that wins the scorer seat. Second, the injection surface is contained: adversarial text inside a document can at worst corrupt that document's feature values—a bounded, per-case failure the tree averages over—rather than hijack a decision-making agent, since the sensor has no tools, no memory across cases, and no authority [5].

XGBoost as scorer. The synchronous authorization path is unchanged from the incumbent design: fixed-schema features (now augmented by sensor outputs where evidence exists) enter a boosted tree; a calibrated score exits in milliseconds at near-zero marginal cost. Nothing in the real-time loop parses natural language. All four operational arguments of Section 3 are honored by construction.

Post-score agentic triage. Alerts crossing the threshold enter a queue where an LLM agent performs the mechanical investigation of Section 8.4: retrieving history, cross-referencing entities, checking narrative consistency, and ranking the queue by investigative priority with a written rationale per case. The agent is asynchronous (seconds-to-minutes latency is acceptable), budget-capped (the token spend lands only on the $\sim$1% of traffic that alerts), and advisory (it cannot close a case or file a report). Its cold-start mode (Section 8.3) runs the same loop seeded by typology bulletins instead of scorer alerts, covering the label vacuum until confirmed cases retrain the tree.

LLM as scribe. At the end of the queue, the scribe drafts the two regulated documents of Section 8.2: the SAR narrative, conditioned on the case file and structured to FinCEN's who/what/when/where/why guidance [4]; and, in credit-decision contexts, the adverse-action statement of specific reasons,

conditioned on the tree's feature attributions so that the stated reasons are the model's actual principal reasons as ECOA/Regulation B demands [13, 2]. Every draft carries provenance links from each claim to its evidence, and a human analyst signs before anything is filed. The scribe is the highest-value, lowest-risk seat: pure text generation, fully reviewable, in a category with no non-LLM competitor.

The label loop. Analyst dispositions flow back as confirmed labels, retraining the scorer and grading the sensor and triage components. The architecture is thus not a static ensemble but a division of labor with a single supervised spine: the tree learns from every closed case, and the LLM seats are evaluated by how much they improve the tree's inputs, the queue's ordering, and the filing's speed—each independently measurable, which is the program discipline ("measure capability, don't trust the label") applied to the system's own components.

10 Compliance Grounding for the Scribe

The scribe role rests on two bodies of U.S. regulation that oblige financial institutions to produce specific prose. We cite the primary sources because the paper's most durable claim—that narration is a task category no tree can enter—is a legal fact before it is a modeling fact.

10.1 SAR Narratives (Bank Secrecy Act / FinCEN)

Banks must report suspicious transactions to the Financial Crimes Enforcement Network (FinCEN): under 31 C.F.R. § 1020.320, a bank must file a Suspicious Activity Report for transactions of at least $5,000 conducted at or through the bank that it "knows, suspects, or has reason to suspect" involve illicit funds, are designed to evade Bank Secrecy Act requirements, or have no apparent lawful purpose, generally within 30 days of detection [14]. The report's decisive free-text component is the narrative. FinCEN's guidance on preparing SAR narratives instructs filers to identify the essential elements of the suspicious activity—who, what, when, where, and why, along with the method of operation—in a complete, self-contained account, and emphasizes that the narrative is where the filing institution explains why the activity is suspicious [4]. This is precisely a conditional text-generation task: given a case file, produce a complete, accurate, element-covering narrative. Analyst backlogs make it a bottleneck; its element-coverage structure makes it checkable; and emerging agentic systems target exactly this drafting task [9]. Our architecture keeps the human filer's judgment and signature—the LLM drafts, the analyst verifies against the evidence links, and the institution files.

10.2 Adverse-Action Notices (ECOA / Regulation B)

When a creditor takes adverse action on a credit application, the Equal Credit Opportunity Act as implemented by Regulation B requires notification including a statement of specific reasons (or disclosure of the right to obtain one): under 12 C.F.R. § 1002.9, the statement of reasons "must be specific and indicate the principal reason(s) for the adverse action," and generic appeals to internal standards or a failing score, without more, do not suffice [13]. The Consumer Financial Protection Bureau's Circular 2022-03 addresses the machine-learning case directly: the adverse-action requirements apply regardless of the technology used, creditors may not rely on "black-box" complexity as a defense, and a creditor that cannot state the specific principal reasons behind its own model's decision is not excused but non-compliant [2].

Two design consequences follow for fraud-adjacent credit decisions. First, the *scorer* must remain a model whose principal reasons are extractable—a gradient-boosted tree with feature attributions satisfies this far more directly than an end-to-end LLM judgment whose verbalized rationale need not correspond to its computation. Keeping XGBoost in the scorer seat is thus a compliance posture, not only a performance one. Second, the *scribe* must be constrained to narrate the scorer's actual attributions: the LLM formats and phrases the principal reasons; it must not invent them. The draft is generated from the attribution vector, carries the attribution as provenance, and is reviewed by a human before issuance. An LLM that freely composed plausible-sounding reasons would be a Regulation B violation generator; an LLM that renders the tree's true top attributions into the required specific statement is a compliance tool.

Scope note. We cite U.S. federal requirements because they are the sharpest published text-generation obligations in fraud and credit workflows; analogous obligations exist in other jurisdictions

but are not analyzed here. Nothing in this section is legal advice; it establishes only that the narration category exists, is mandatory, and is textual—hence enterable by an LLM and not by a tree.

11 Related Work

Gradient boosting on tabular data. XGBoost [1] remains the workhorse of applied tabular classification. Grinsztajn et al. [6] show systematically that tree-based models still outperform deep learning on medium-sized tabular benchmarks, attributing the gap to trees' robustness to uninformative features and to the non-smooth, non-rotation-invariant structure of tabular targets; Shwartz-Ziv and Armon [12] reach the same conclusion across deep tabular architectures. Our scorer result is a single applied data point consistent with this literature, with the added operational arguments (latency, cost, calibration [7], injection surface [5]) that the benchmarking papers do not address.

LLMs for tabular data. TabLLM [8] established the serialize-and-prompt recipe and showed it is most competitive in very-few-shot regimes, degrading relative to boosted trees as labeled data grows—exactly the pattern our head-to-head reproduces at $40{,}000$ labels, and exactly why our taxonomy assigns the LLM the cold-start seat (Section 8.3) rather than the scorer seat.

Fraud detection on transaction data. Dal Pozzolo et al. [3] give the canonical treatment of realistic credit-card fraud modeling—class imbalance, verification latency, and concept drift—and motivate the anonymized-feature, heavily imbalanced regime our synthetic benchmark imitates. Our contribution is orthogonal to this line: we do not propose a better detector but a division of labor around an already-strong one.

LLM agents for financial crime. Pirmorad [11] shows that LLMs prompted with serialized financial-graph neighborhoods can perform analyst-style money-laundering triage few-shot, with coherent red-flag justifications; Naik et al. [9] builds an agentic framework specifically for drafting AML compliance narratives (SARs) with human-in-the-loop review. These systems instantiate, respectively, our cold-start and scribe/triage seats; our contribution is to place them in an explicit architecture where the real-time scorer remains a tree.

VLMs for document and image fraud. FakeShield [16] demonstrates explainable image-forgery detection and localization with multimodal LLMs; Forensics-Bench [15] benchmarks large vision–language models across 112 forgery types and finds substantial headroom. Together they support both halves of our sensor claim: VLMs uniquely extend the input space to raw document evidence, and their current reliability justifies confining them to feature extraction with downstream tree scoring and human review rather than autonomous verdicts.

Program context. The parent reproducibility program measures RL post-training claims capability-by-capability rather than by their labels; the present side-probe applies the identical discipline to the marketing label "AI" in fraud detection. The methodological through-line is decomposition: a bundled claim is replaced by per-capability measurements, and the deployment decision is made seat by seat.

12 Limitations

Custom synthetic dataset. Our benchmark is generated by `make_classification` [10], not drawn from production transactions. It reproduces the class imbalance and anonymized feature shape of single-institution card-fraud data but none of its temporal drift, verification latency, seasonality, or adversarial adaptation [3]. The current artifact-backed AUCs (0.7955, 0.48268) are therefore results on a stylized testbed and must not be quoted as production expectations. The margin is dataset- and environment-specific.

Historical numbers are records, not reproductions. We could not reconstruct the environment of the 21 June 2026 internal run that produced the older 0.975/0.948 pair, so those numbers are historical records, not headline evidence. The paper's architectural conclusions are insulated from

this by design: the seat assignment rests on the latency, cost, calibration, and injection arguments of Section 3, which apply at accuracy parity and do not depend on the margin or its direction.

No cross-institution validation. Everything here is one configuration of one synthetic generator standing in for one institution's feature view. Fraud patterns, feature pipelines, and base rates vary sharply across issuers and geographies; the head-to-head has not been replicated on any second data distribution, public benchmark, or real institution.

Cost estimates are internal. The $\sim 85\times$ agentic-triage cost advantage (Section 8.4) is an internal estimate built from June-2026 token prices, assumed per-alert investigation times, and loaded analyst compensation. Each assumption is contestable; none has been validated in a production deployment; the figure is reported to one significant figure and should be read as "one to two orders of magnitude," not as a measured constant. The latency and per-transaction cost contrasts of Section 3 are likewise operational characterizations, not benchmarked service-level measurements.

Single LLM family, single recipe. The 0.48268 challenger is one instruction-tuned LLM family under one serialization format and one fine-tuning budget. We did not sweep model families, scales, serializations, or prompting strategies, and stronger or differently trained LLMs could narrow the accuracy gap. We note, however, that the architecture's assignment of seats does not hinge on the gap: the latency, cost, calibration, and injection arguments of Section 3 apply at accuracy parity.

Capability gaps argued, not all measured. Of the four taxonomy entries, only the scorer head-to-head carries a number of our own. The sensor, scribe, and cold-start seats are grounded in external literature and primary regulatory text rather than in our own end-to-end evaluations; the hybrid architecture of Section 9 is a design contribution whose integrated performance is future work, not a deployed and audited system. Nothing in Section 10 is legal advice.

13 Future Work: Building the Hybrid

The obvious next step is to build and measure the architecture rather than argue it. Our plan, in the order the seats de-risk each other:

1. **Sensor first.** Attach a VLM feature extractor to a document-bearing subset of cases and measure the marginal AUC of sensor-derived features *inside the existing tree*—the cleanest test, since the metric is the incumbent's own. This includes adversarial evaluation: injected instructions inside documents must degrade at worst that document's features [5].
2. **Scribe second.** Evaluate SAR-narrative drafts against FinCEN's element-coverage structure (who/what/when/where/why/how) [4] with analyst-grader rubrics, and adverse-action drafts for fidelity to the tree's actual attributions [13, 2]—measuring time-to-file and revision distance, with a hard human sign-off gate.
3. **Triage third.** Replace the internal $85\times$ estimate with measured per-alert token costs and analyst-minutes-saved on a live queue, including the ordering quality of agent-ranked versus score-ranked queues.
4. **Cold-start last.** Simulate typology emergence by holding out a generator cluster as an "unseen scheme" and measuring few-shot LLM detection during the label vacuum, against the retrained tree's catch-up curve.

Each experiment scores one seat with the seat's own metric—capability measured, label ignored. If the program thesis holds anywhere outside RL post-training, it should hold here: the pipeline that results is not "AI replacing the model" but a tree that scores, an LLM that sees and writes, and an analyst who decides.

References

[1] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2016. arXiv:1603.02754.

[2] Consumer Financial Protection Bureau. Consumer financial protection circular 2022-03: Adverse action notification requirements in connection with credit decisions based on complex algorithms. https://files.consumerfinance.gov/f/documents/cfpb_2022-03_circular_2022-05.pdf, 2022. Issued May 26, 2022.

[3] Andrea Dal Pozzolo, Giacomo Boracchi, Olivier Caelen, Cesare Alippi, and Gianluca Bontempi. Credit card fraud detection: A realistic modeling and a novel learning strategy. *IEEE Transactions on Neural Networks and Learning Systems*, 29(8):3784–3797, 2018. doi:10.1109/TNNLS.2017.2736643.

[4] Financial Crimes Enforcement Network (FinCEN). Guidance on preparing a complete & sufficient suspicious activity report narrative. https://www.fincen.gov/system/files/shared/sarnarrcompletguidfinal_112003.pdf, 2003. U.S. Department of the Treasury, November 2003.

[5] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*, 2023.

[6] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. Why do tree-based models still outperform deep learning on tabular data? *arXiv preprint arXiv:2207.08815*, 2022.

[7] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017. arXiv:1706.04599.

[8] Stefan Hegselmann, Alejandro Buendia, Hunter Lang, Monica Agrawal, Xiaoyi Jiang, and David Sontag. TabLLM: Few-shot classification of tabular data with large language models. *arXiv preprint arXiv:2210.10723*, 2022.

[9] Prathamesh Vasudeo Naik, Naresh Kumar Dintakurthi, Zhanghao Hu, Yue Wang, and Robby Qiu. Co-investigator AI: The rise of agentic AI for smarter, trustworthy AML compliance narratives. *arXiv preprint arXiv:2509.08380*, 2025.

[10] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[11] Erfan Pirmorad. Exploring the in-context learning capabilities of LLMs for money laundering detection in financial graphs. *arXiv preprint arXiv:2507.14785*, 2025. Accepted at the AI4FCF workshop, IEEE ICDM Workshops (ICDMW) 2025.

[12] Ravid Shwartz-Ziv and Amitai Armon. Tabular data: Deep learning is not all you need. *arXiv preprint arXiv:2106.03253*, 2021.

[13] U.S. Code of Federal Regulations. Notifications (Regulation B, implementing the Equal Credit Opportunity Act). 12 C.F.R. § 1002.9, 2026. https://www.consumerfinance.gov/rules-policy/regulations/1002/9/.

[14] U.S. Code of Federal Regulations. Reports by banks of suspicious transactions. 31 C.F.R. § 1020.320, 2026. https://www.law.cornell.edu/cfr/text/31/1020.320.

[15] Jin Wang, Chenghui Lv, Xian Li, Shichao Dong, Huadong Li, Kelu Yao, Chao Li, Wenqi Shao, and Ping Luo. Forensics-bench: A comprehensive forgery detection benchmark suite for large vision language models. *arXiv preprint arXiv:2503.15024*, 2025.

[16] Zhipei Xu, Xuanyu Zhang, Runyi Li, Zecheng Tang, Qing Huang, and Jian Zhang. FakeShield: Explainable image forgery detection and localization via multi-modal large language models. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.02761.